Computer Vision

Bogdan Alexe

bogdan.alexe@fmi.unibuc.ro

University of Bucharest, 2nd semester, 2025-2026

Course structure

1. **Low-level vision: Features and filters**

   Linear filters, color, texture, edge detection, template matching

2. **Mid-level vision: Grouping and fitting**

   Fitting curves and lines, robust fitting, RANSAC, Hough transform, segmentation

3. **Mid-level vision: Multiple views**

   Local invariant feature and description, epipolar geometry and stereo, object instance recognition

4. **High-level vision: Object recognition**

   Object classification, object detection, part based models, bovw models

5. **High-level vision: Video understanding**

   Object tracking, background subtraction, motion descriptors, optical flow

Local features

A local image feature is a region in the image that:

- has distinctive content (strong intensity / color variation)
- is locally unique and repeatable (can be reliably detected across images)
- has a precise localization (well-defined position and scale)
- has a robust descriptor (can be matched to similar regions)
- is invariant to geometric (translation, rotation, scale) and photometric (illumination, contrast) transformations.

Local features

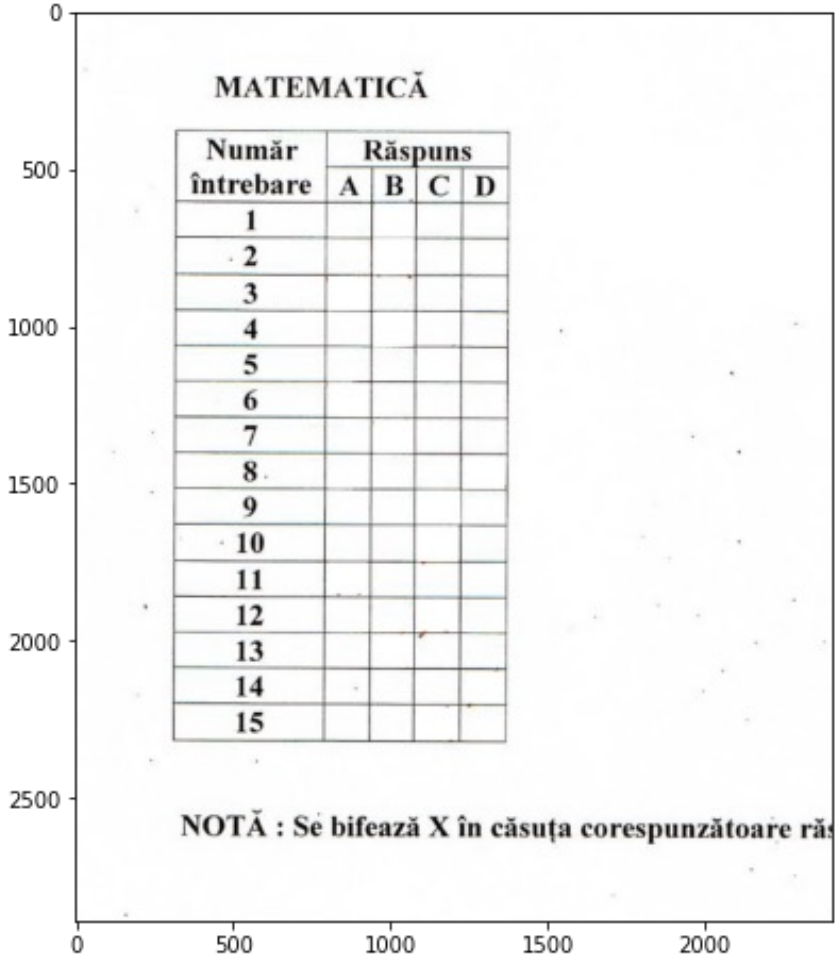
A local feature = keypoint (location + scale) + descriptor (local appearance)

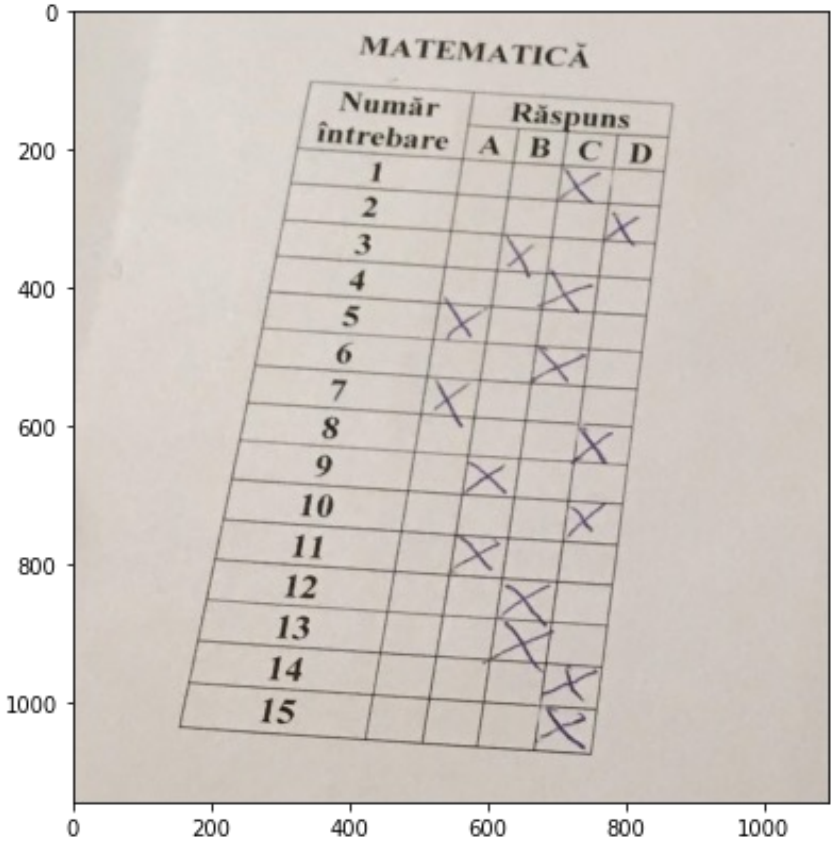
Good features: distinctive, repeatable, invariant to geometry and illumination

→ used for matching and recognition

Matching local features – past P1

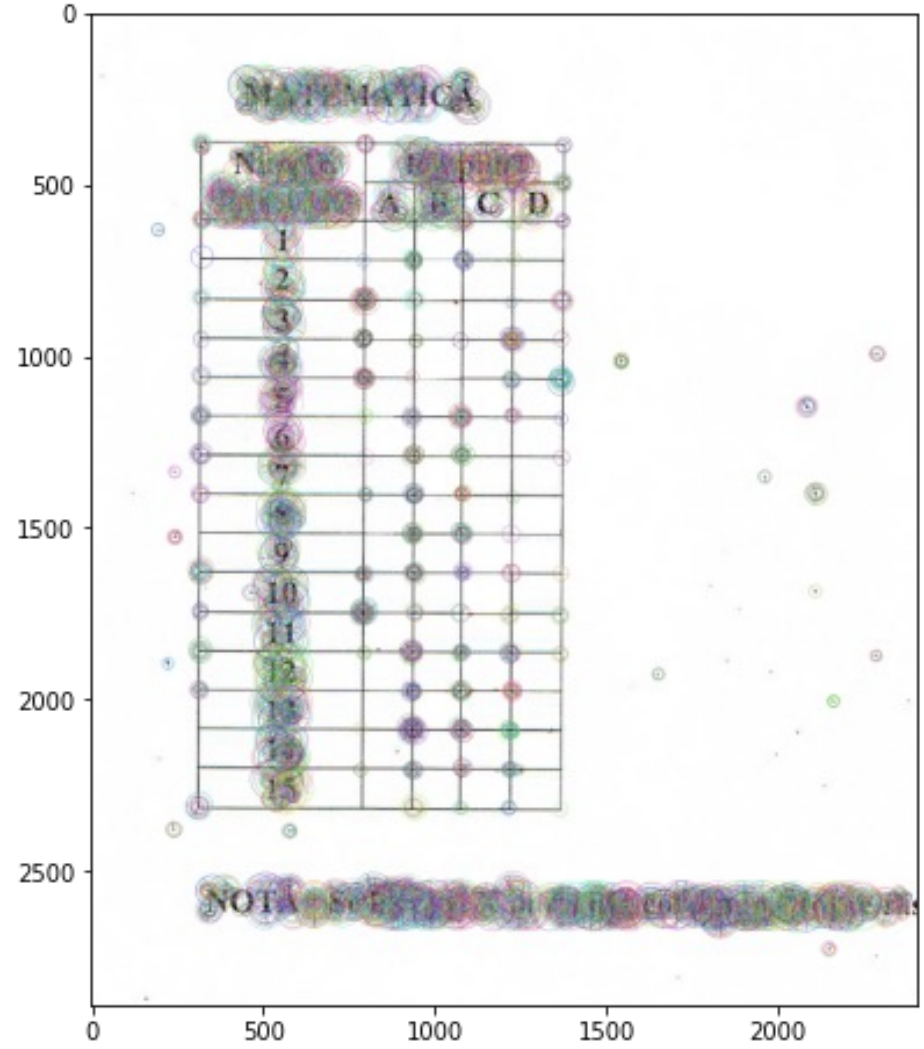
- matching local image feature = find correspondence points between images based on local features that look the same

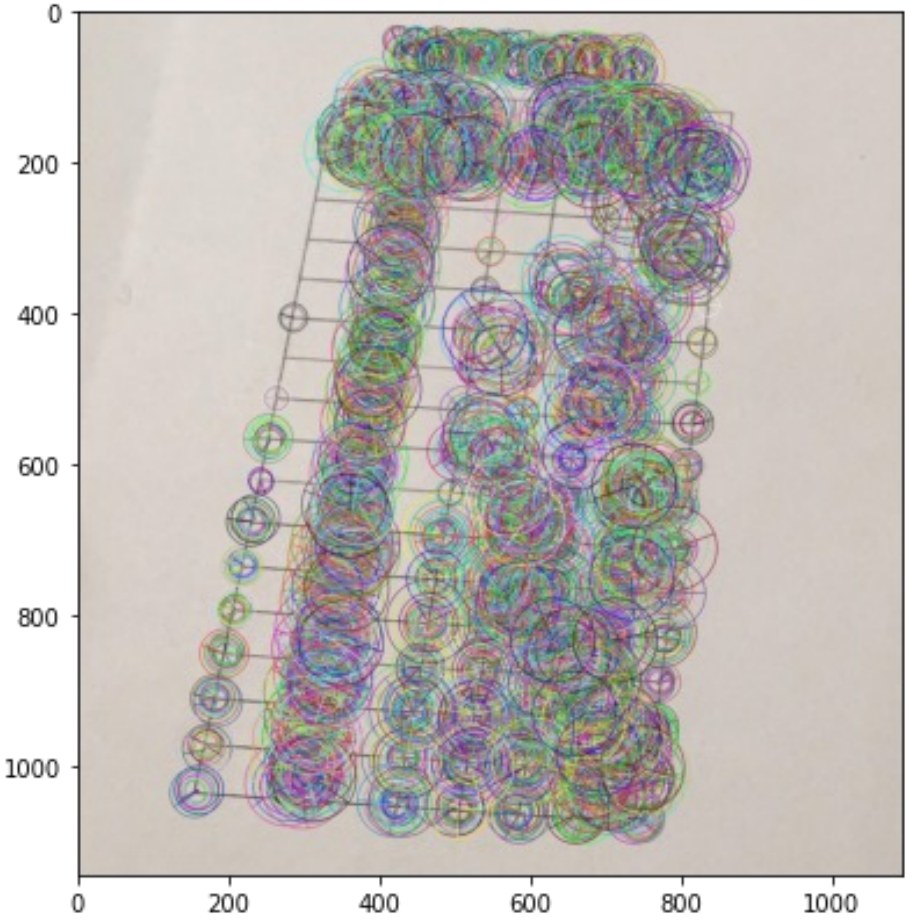




Matching local features – past P1

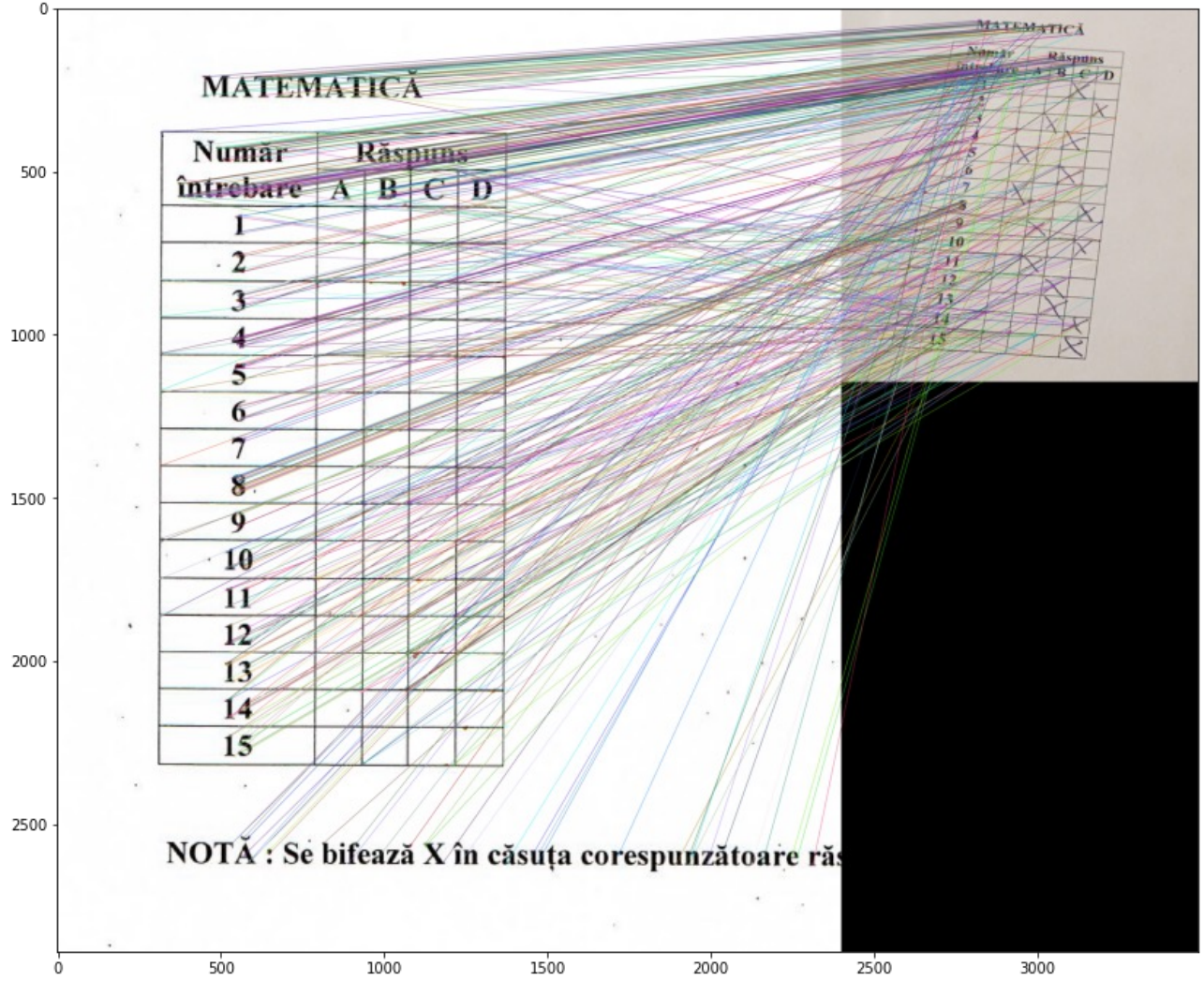
- matching local image feature = find correspondence points between images based on local features that look the same





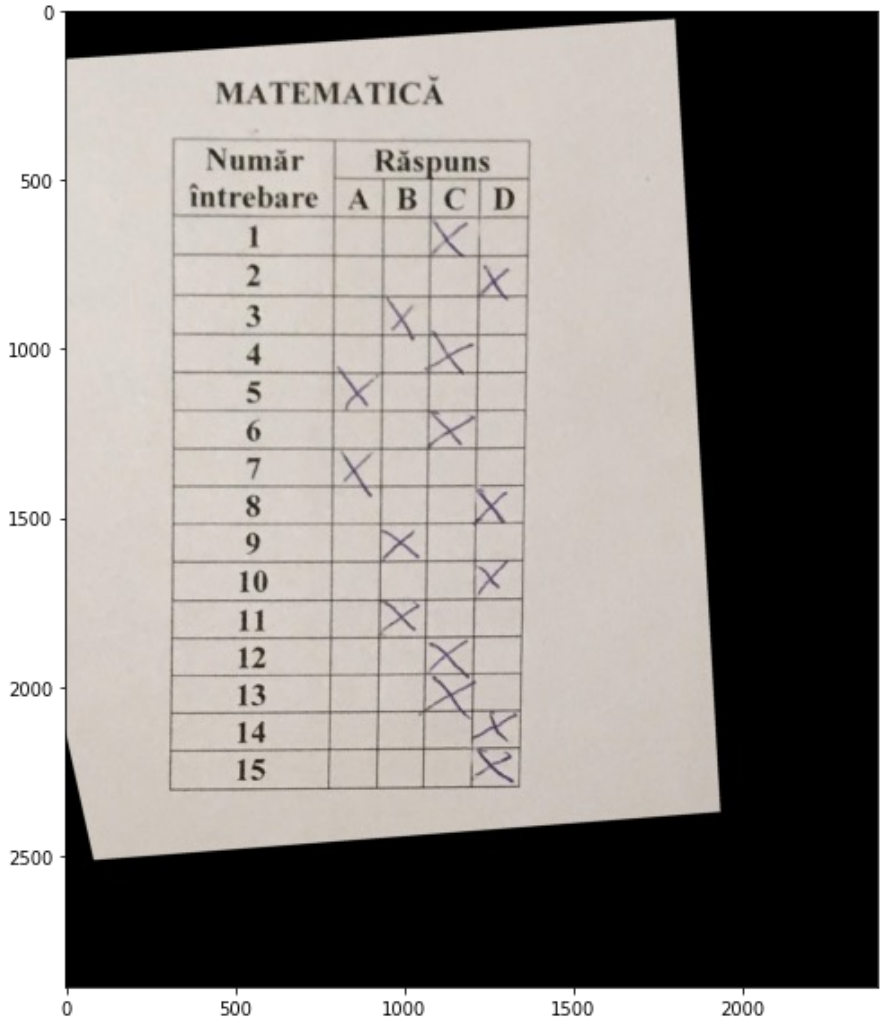
Matching local features – past P1

- matching local image feature = find correspondence points between images based on local features that look the same



Finding the homography transformation

- Find the matrix of the homography (perspective) transformation and use it to warp one image



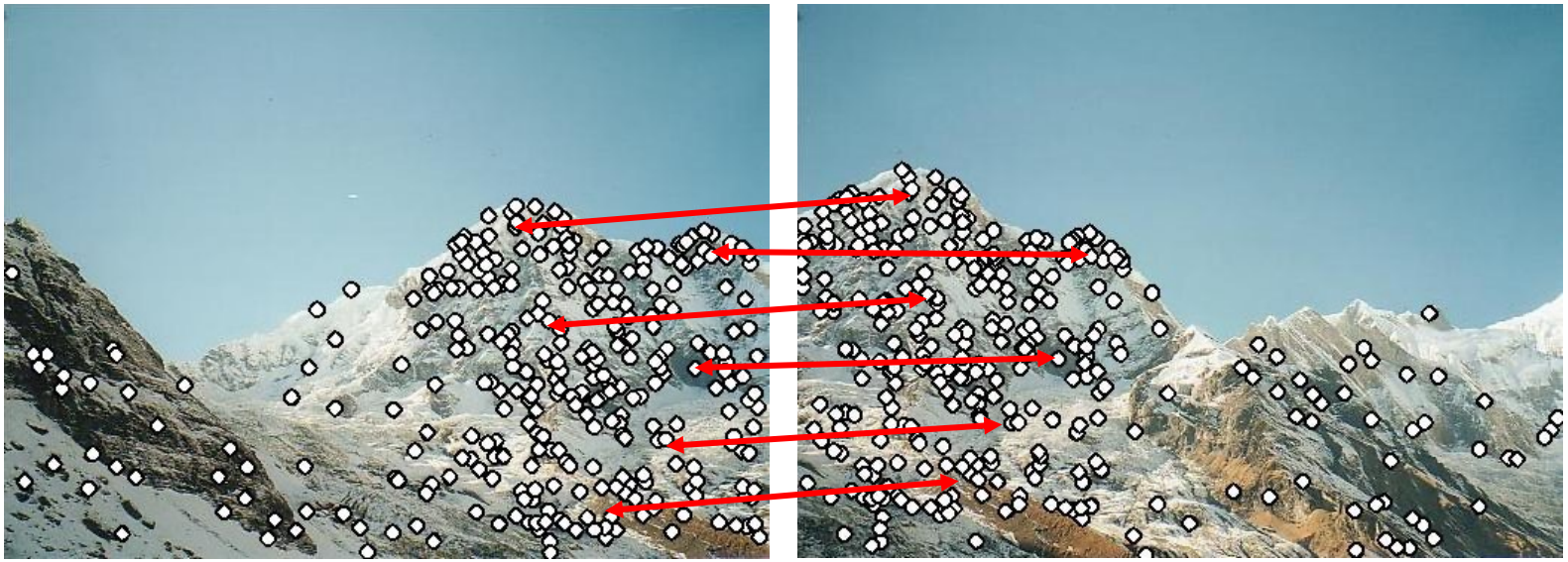
Why extract local features?

- Motivation: panorama stitching
  - We have two images – how do we combine them?

Step 1: extract local features

Why extract local features?

- Motivation: panorama stitching
  - We have two images – how do we combine them?



Step 1: extract local features

Step 2: match local features

Why extract local features?

- Motivation: panorama stitching
  - We have two images – how do we combine them?

Step 1: extract local features

Step 2: match local features

Step 3: align images

Local features – lab class 4

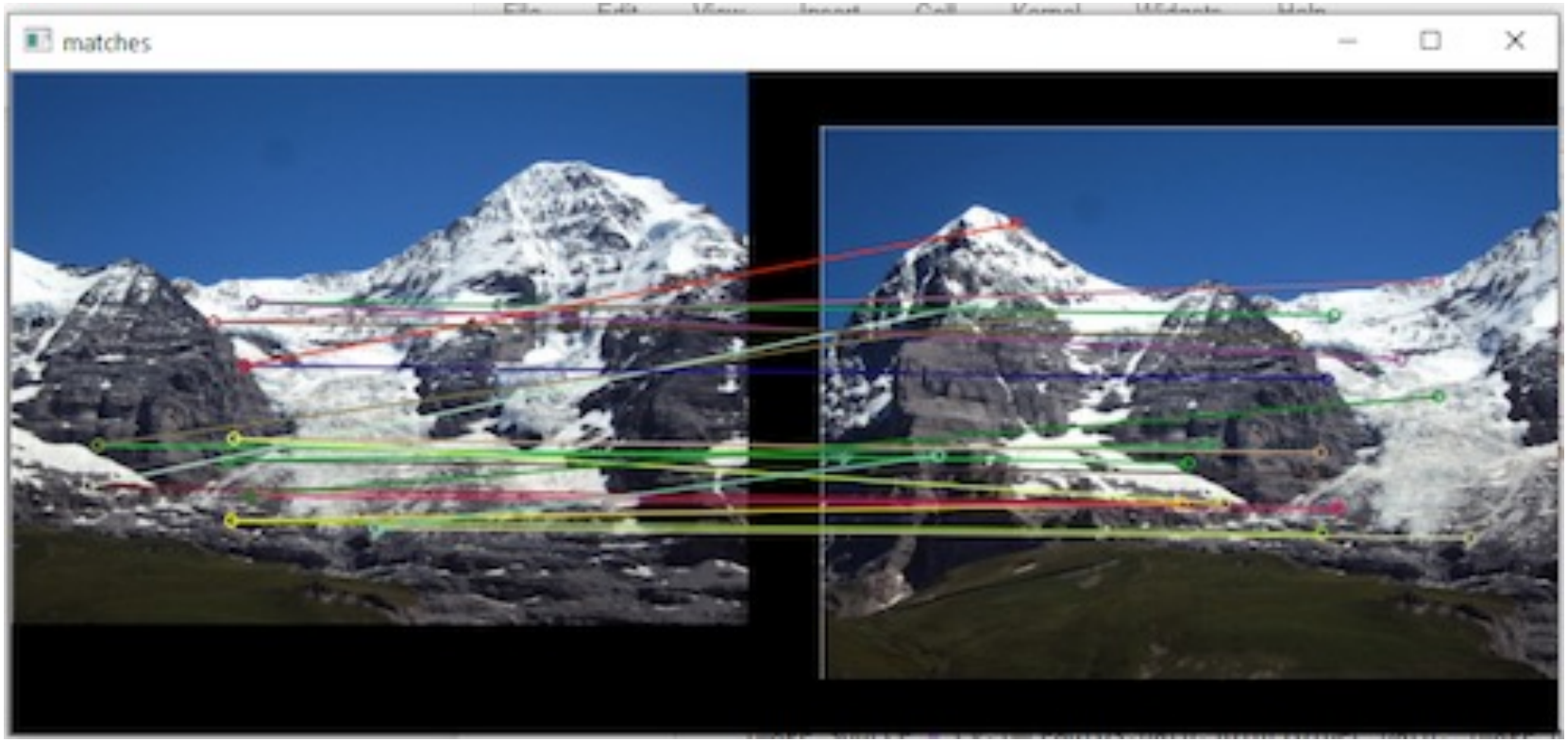
- a local feature = a distinctive, repeatable, and invariant image region that can be reliably matched across images.

Local features – lab class 4

- a local feature = a distinctive, repeatable, and invariant image region that can be reliably matched across images.

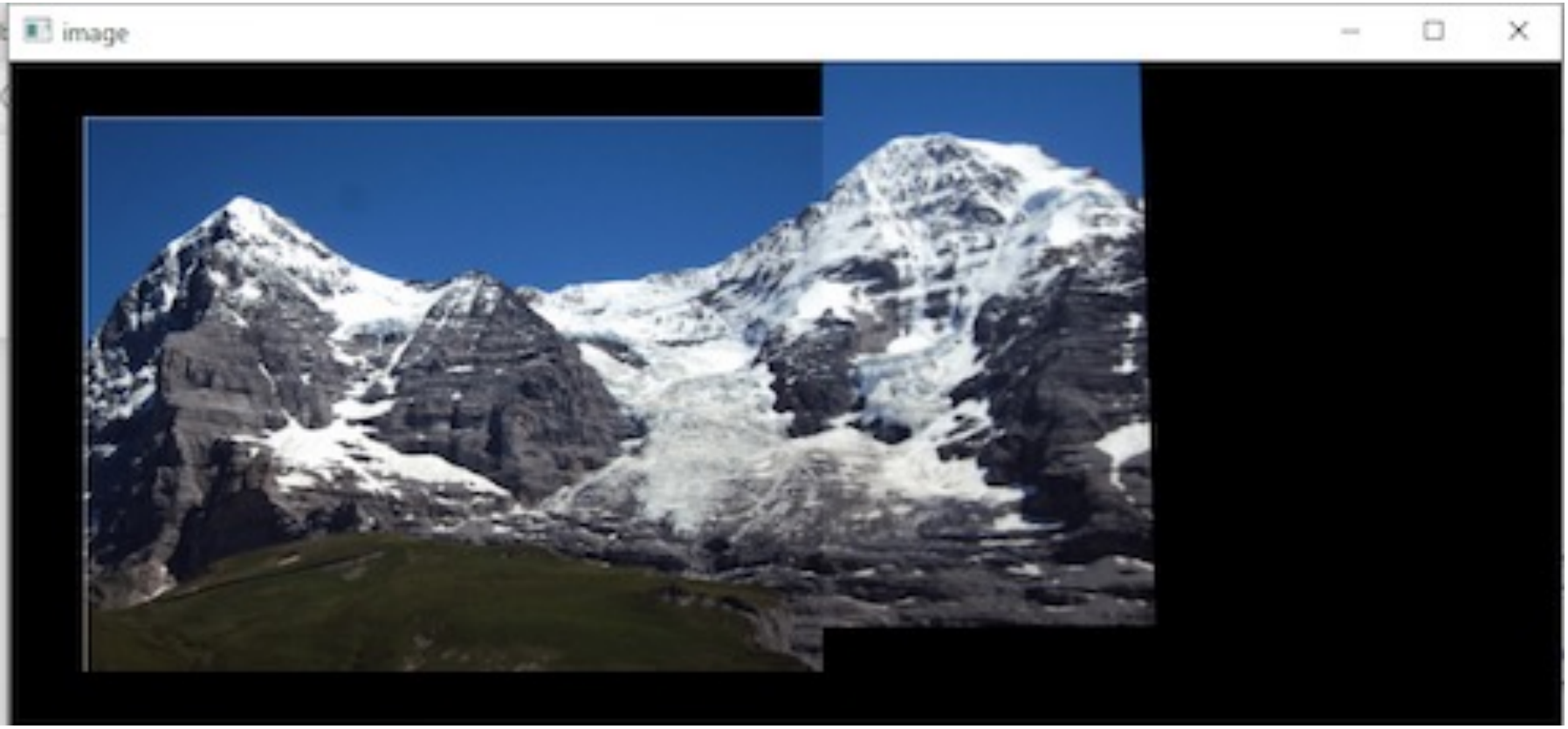
Matching local features – lab class 4

- matching local image feature = find correspondence points between images based on local features that look the same



Panorama stitching – lab class 4

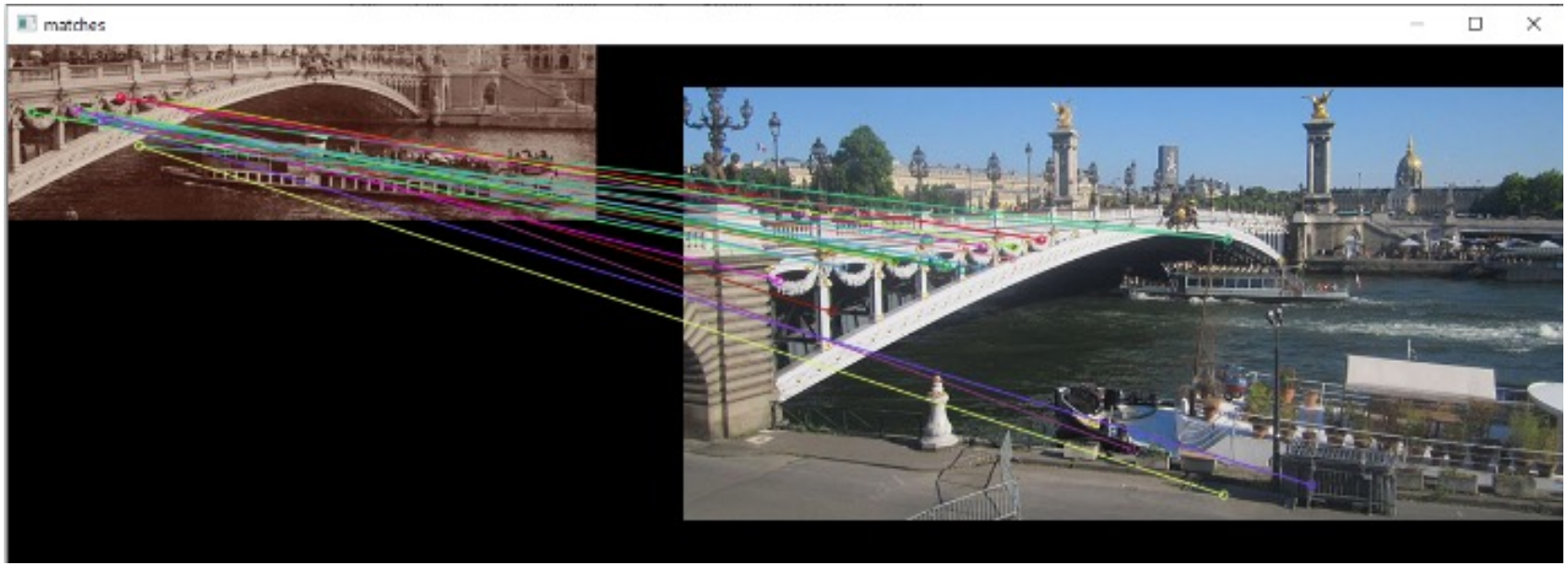
- align images in order to obtain panorama images



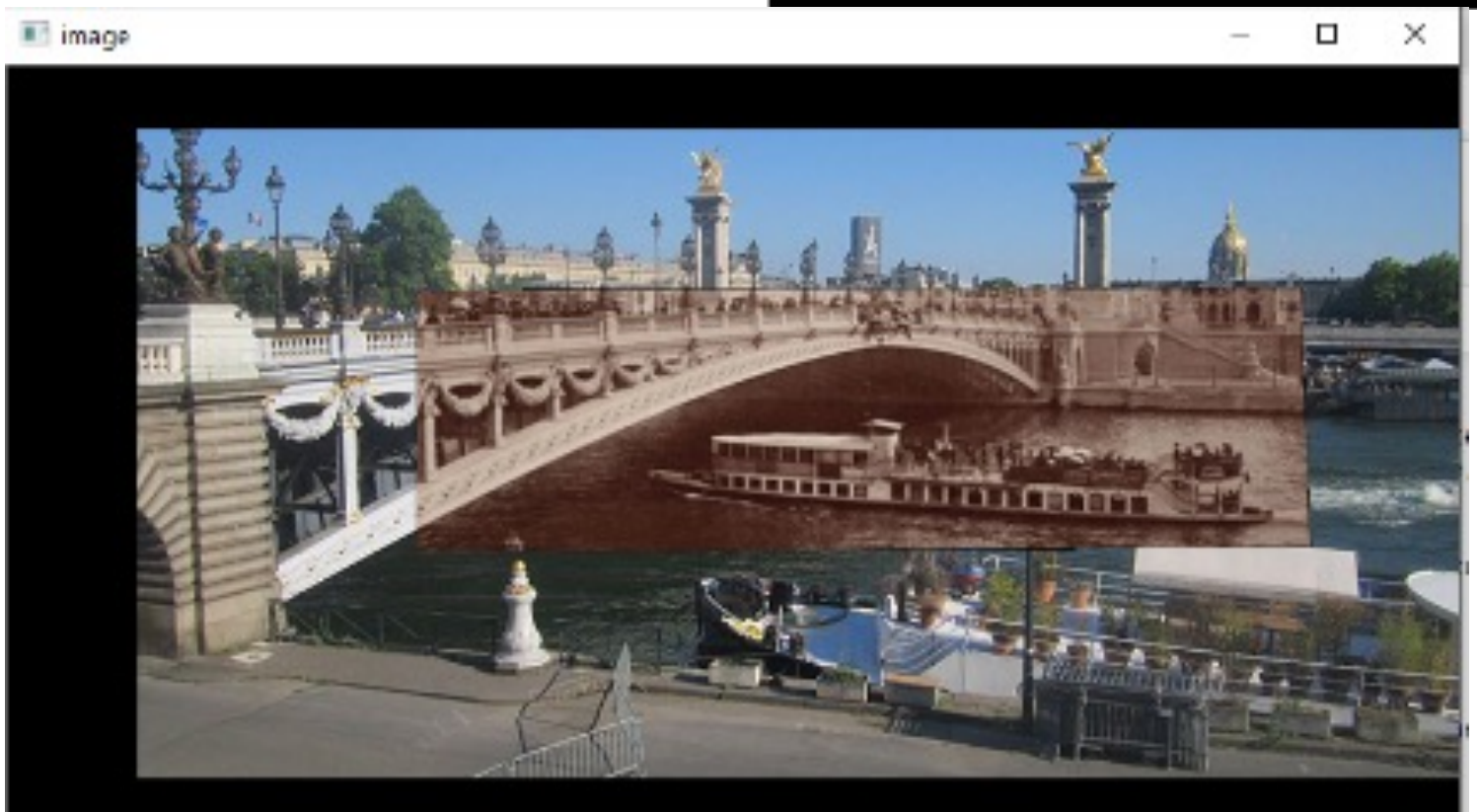
Panorama stitching – lab class 4

- align images in order to obtain panorama images

Then and now – lab class 4



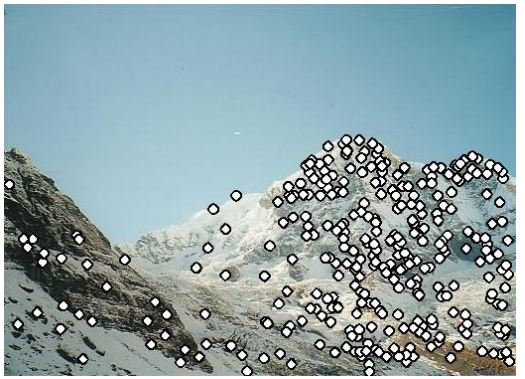
Then and now – lab class 4



Local features: pipeline

1. Detection: where?

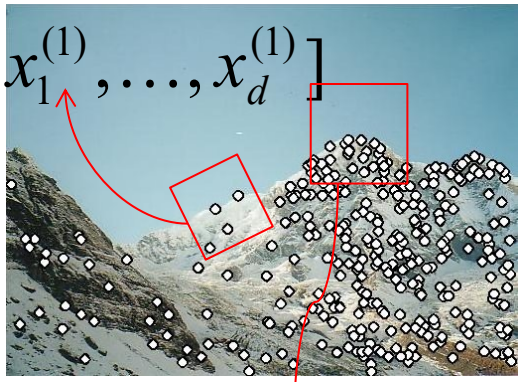
- Identify repeatable and distinctive points in the image
- Typically: corners, blobs, high-texture regions
- Goal: same points should be found across different views
- Examples: Harris, Harris-Laplace, DOG (SIFT), etc.



2. Description: what?

- For each keypoint, extract a feature vector
- Encodes the local appearance around the point
- Should be:
  - Invariant (scale, rotation, illumination)
  - Discriminative
- Examples: SIFT (128D), SURF, ORB, etc.

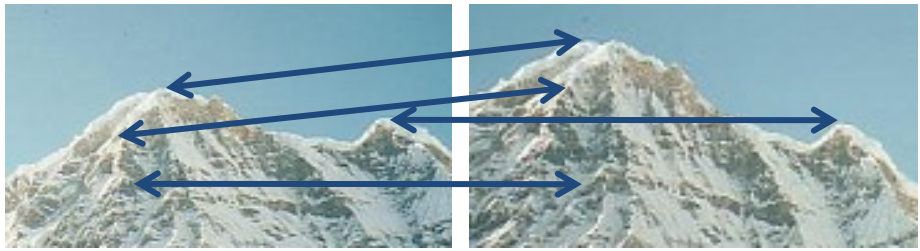
$$\mathbf{x}_1 = [x_1^{(1)}, \ldots, x_d^{(1)}]$$



$$\mathbf{x}_2 = [x_1^{(2)}, \ldots, x_d^{(2)}]$$

3. Matching: which correspondence to which?

- Compare descriptors across images
- Use a distance metric (e.g., Euclidean distance)
- Often combined with: Ratio test (Lowe), RANSAC (geometric verification)



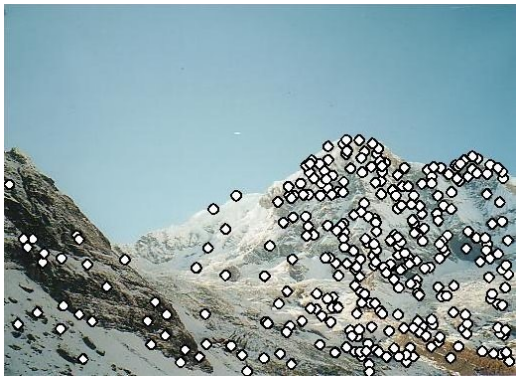
Applications

- Local features are used for:
  - Image alignment
  - 3D reconstruction
  - Motion tracking
  - Robot navigation
  - Database indexing and retrieval
  - Object recognition (before Deep Learning)

Local features: pipeline

1. Detection: where?

- Identify repeatable and distinctive points in the image
- Typically: corners, blobs, high-texture regions
- Goal: same points should be found across different views
- Examples: Harris, Harris-Laplace, DOG (SIFT), etc.



2. Description: what?

- For each keypoint, extract a feature vector
- Encodes the local appearance around the point
- Should be:
  - Invariant (scale, rotation, illumination)
  - Discriminative
- Examples: SIFT (128D), SURF, ORB, etc.

3. Matching: which correspondence to which?

- Compare descriptors across images
- Use a distance metric (e.g., Euclidean distance)
- Often combined with: Ratio test (Lowe), RANSAC (geometric verification)

- What points would you choose?


1
2
3
4
5
6

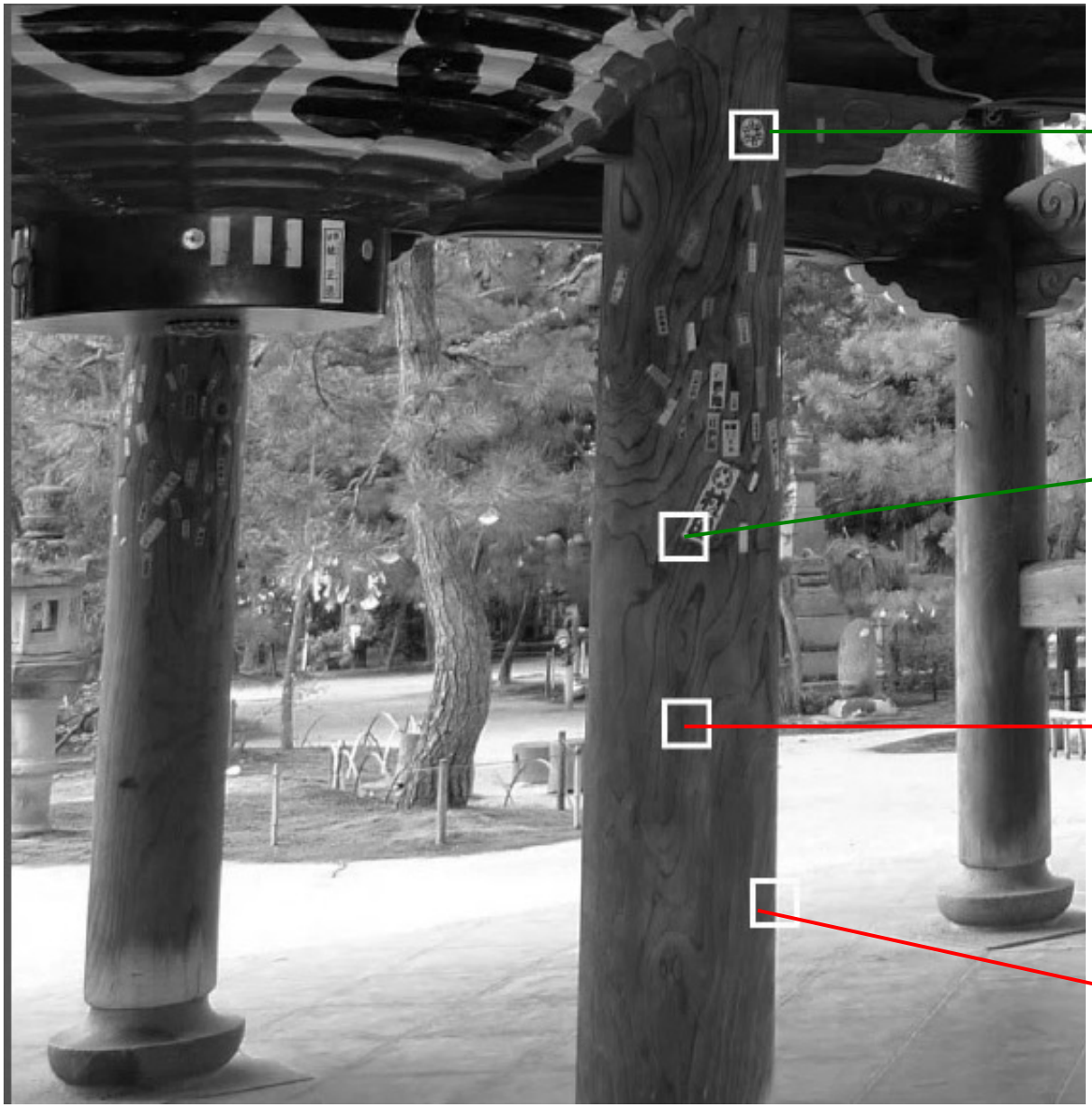
Detecting local invariant features

- Goal: Find distinctive points for reliable matching

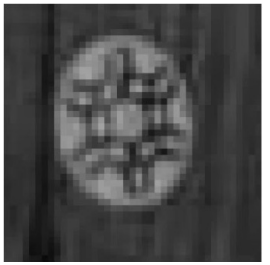
Uniform regions
Ambiguous for matching

Edges
Ambiguous for matching

Corners
Good for matching

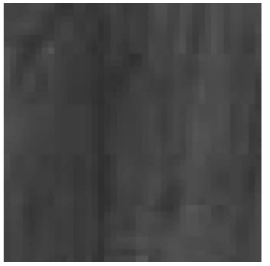


circular region (blob)

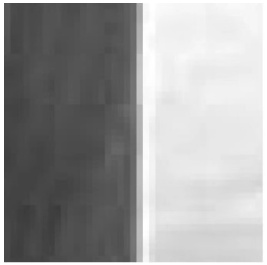


corner

uniform region



edge



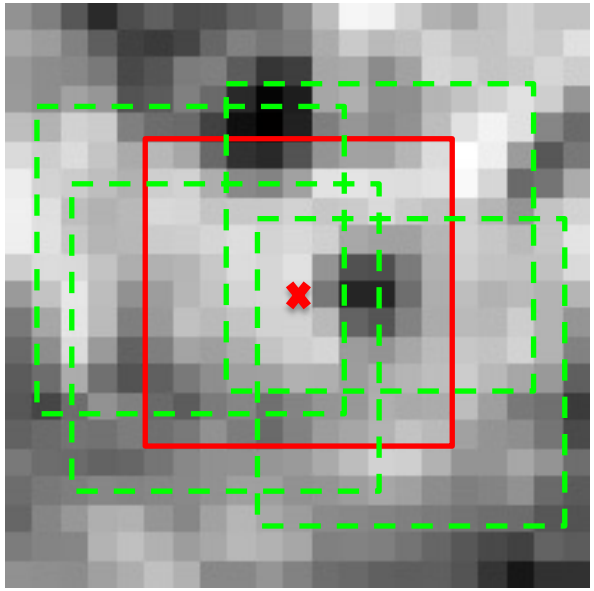
Detecting local invariant features

- Detection of keypoints (Where?)
  - identify repeatable and distinctive locations
  - typical methods:
    - Harris Corner Detector → corners (intensity variation in all directions)
    - LoG (Laplacian of Gaussian) → blob detection (scale-invariant)

- Description of local patches (What?)
  - represent each keypoint by a feature vector
  - encodes the appearance of the local neighborhood
  - should be:
    - invariant (scale, rotation, illumination)
    - discriminative

Detecting local invariant features

- How do we mathematically formulate what is a good local feature?

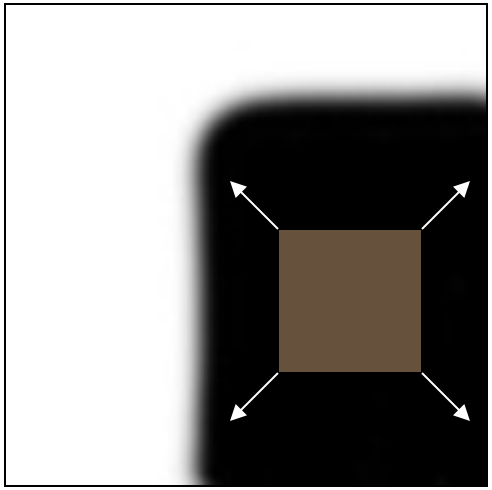


- Characterize the point by a small window (patch) centered in it

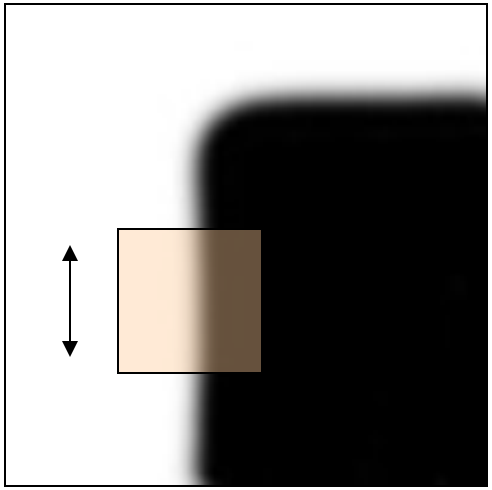
- Good local feature: shifting a window in *any direction* should give *a large change* in intensity (sum of squared distances - SSD)

Corner detection: Basic idea

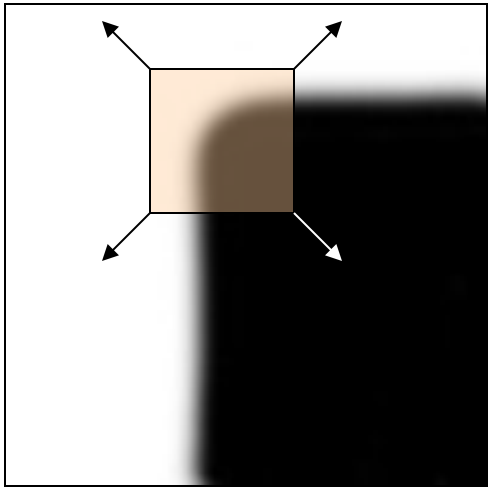
- We should easily recognize the point by looking through a small window
- Shifting a window in *any direction* should give *a large change* in intensity



"flat" region:
no change in all directions



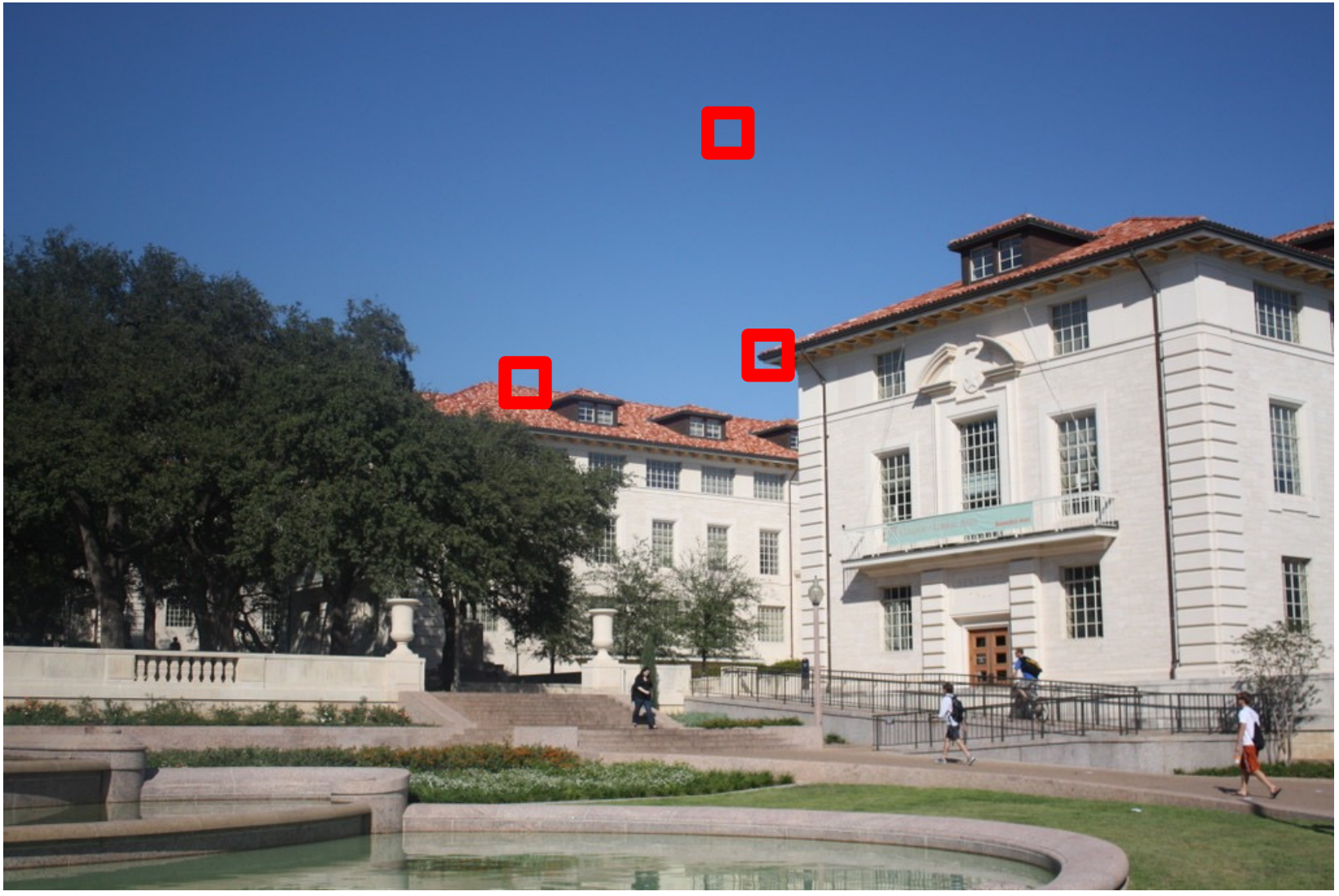
"edge":
no change along the edge direction



"corner":
significant change in all directions

Patch Self-Similarity

Self-similarity = compare a patch with a shifted version of itself using SSD.

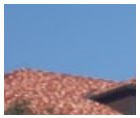


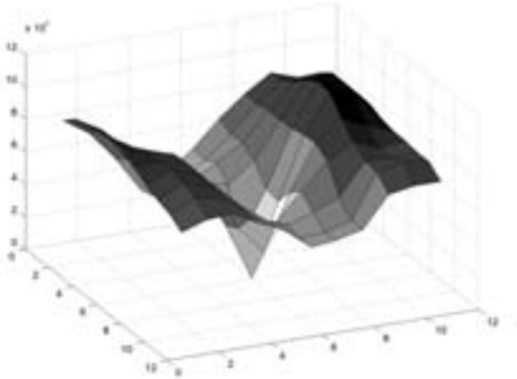
Patch Self-Similarity

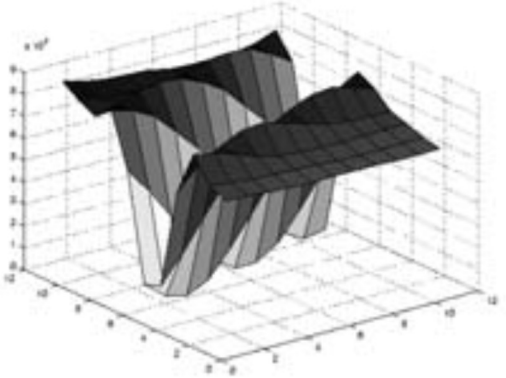
Sky (flat region): low change in all directions

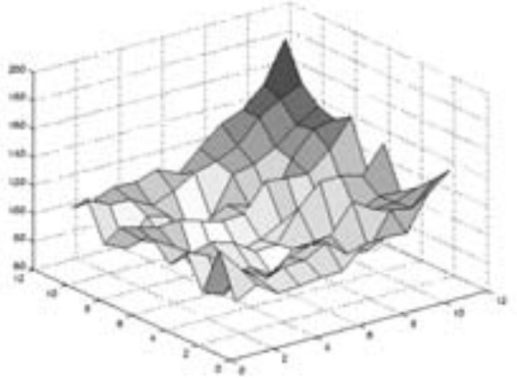
Edge: low change along the edge direction

Corner: high change in all directions





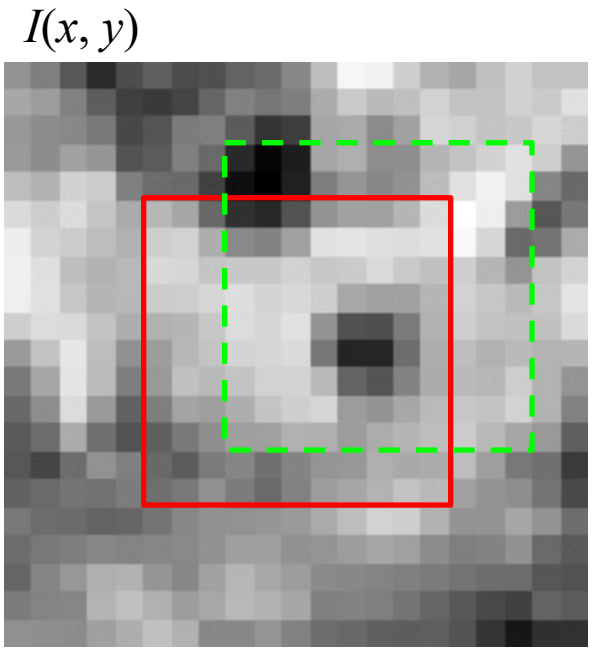


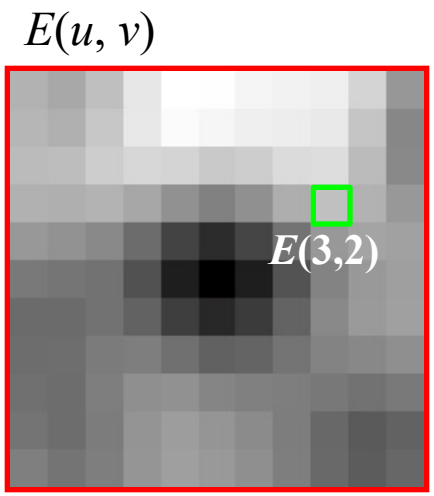


Measuring Patch Change under Small Shifts

Consider a small image window W and shift it by (u,v). Measure change using Sum of Squared Differences (SSD):

$$E(u,v) = \sum_{(x,y)\in W} (I(x+u, y+v) - I(x,y))^2$$





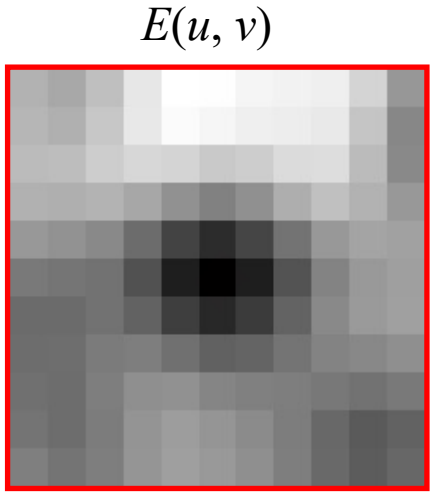
Corners = large change in all directions

Analyze Behavior for Small Shifts

Consider a small image window W and shift it by (u,v). Measure change using Sum of Squared Differences (SSD):

$$E(u,v) = \sum_{(x,y)\in W} (I(x+u, y+v) - I(x,y))^2$$

We are interested in studying the behaviour for small displacements (u,v)



Approximating Intensity with Taylor Expansion

Consider a small image window W and shift it by (u,v). Measure change using Sum of Squared Differences (SSD):

- **First-order Taylor approximation for small motions (*u*, *v*):**

$$I(x+u, y+v) \approx I(x,y) + u \cdot \frac{\partial I}{\partial x}(x,y) + v \cdot \frac{\partial I}{\partial y}(x,y)$$

$$I(x+u, y+v) \approx I(x,y) + u \cdot I_x + v \cdot I_y$$

- gradients tell us how intensity changes locally

- we approximate the image as locally linear

- **Let's plug this into *E*(*u*,*v*):**

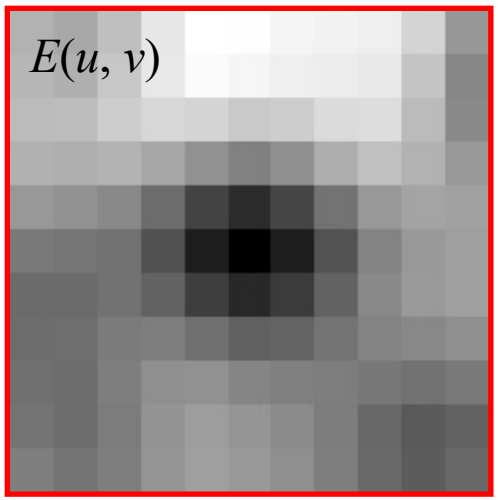
$$E(u,v) = \sum_{(x,y)\in W} \left(I(x+u, y+v) - I(x,y)\right)^2$$

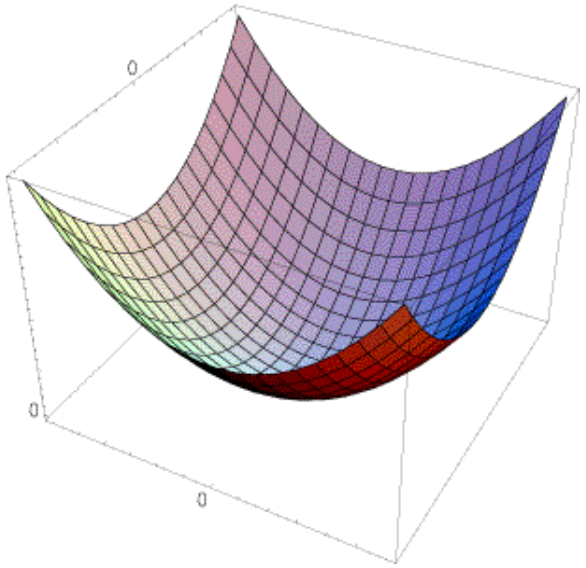
$$E(u,v) \approx \sum_{(x,y)\in W} (u \cdot I_x + v \cdot I_y)^2 = \sum_{(x,y)\in W} u^2 \cdot I_x^2 + 2uv \cdot I_x \cdot I_y + v^2 \cdot I_y^2$$

E(u,v) becomes a quadratic function

•$E(u,v)$ can be locally approximated by a quadratic function of (u,v) (it has the shape of a paraboloid):

$$E(u,v) \approx u^2 \sum_{x,y} I_x^2 + 2uv \sum_{x,y} I_x I_y + v^2 \sum_{x,y} I_y^2$$





•In which directions does $E(u,v)$ change the fastest and the slowest?

Matrix Formulation

•$E(u,v)$ can be locally approximated by a quadratic function of (u,v) (it has the shape of a paraboloid):

$$E(u,v) \approx u^2 \sum_{x,y} I_x^2 + 2uv \sum_{x,y} I_x I_y + v^2 \sum_{x,y} I_y^2$$

$$= \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} \sum_{x,y} I_x^2 & \sum_{x,y} I_x I_y \\ \sum_{x,y} I_x I_y & \sum_{x,y} I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

Second moment matrix M

•These directions are given by the eigenvectors of matrix M, and the amount of change by the eigenvalues.

Second Moment Matrix

$$E(u,v) \approx u^2 \sum_{x,y} I_x^2 + 2uv \sum_{x,y} I_x I_y + v^2 \sum_{x,y} I_y^2$$

$$= \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} \sum_{x,y} I_x^2 & \sum_{x,y} I_x I_y \\ \sum_{x,y} I_x I_y & \sum_{x,y} I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

Second moment matrix M

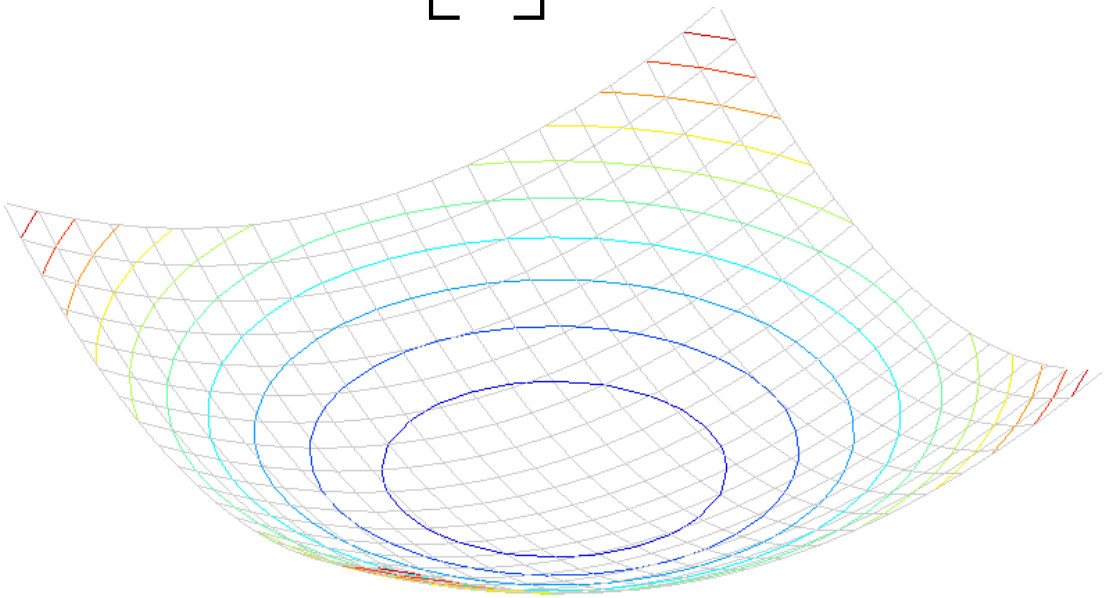
- M encodes gradient distribution in the window
- It summarizes:
  - strength of gradients
  - their orientation

M captures how intensity changes in different directions

Geometric Interpretation of E(u,v)

A horizontal "slice" of $E(u, v)$ is given by the equation of an ellipse:

$$E(u,v) = \text{const} \quad \Rightarrow \quad [u \;\; v]\; M \begin{bmatrix} u \\ v \end{bmatrix} = \text{const}$$



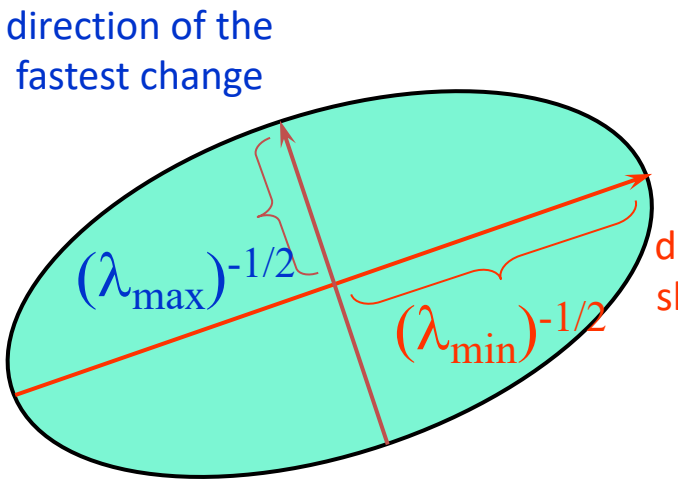
Ellipse shows how fast E increases in each direction

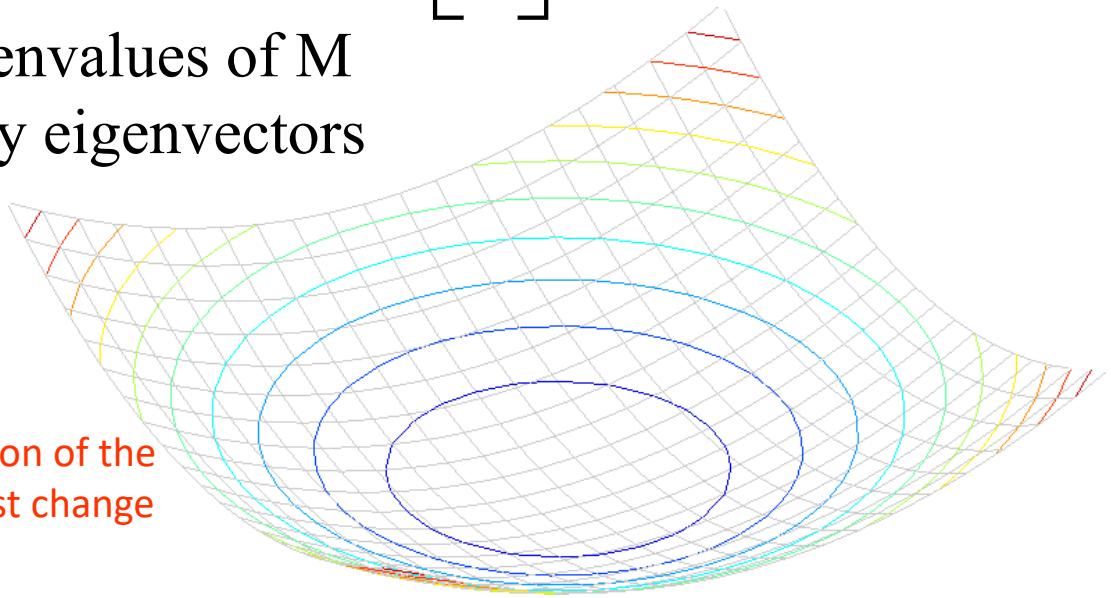
Ellipse Interpretation

A horizontal "slice" of $E(u, v)$ is given by the equation of an ellipse:

$$E(u,v) = \text{const} \quad \Rightarrow \quad [u \;\; v]\; M \begin{bmatrix} u \\ v \end{bmatrix} = \text{const}$$

- Shape determined by eigenvalues of M
- Orientation determined by eigenvectors





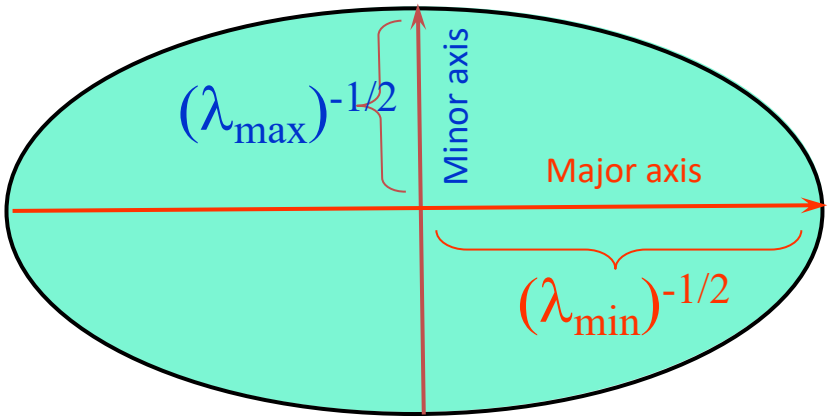
Ellipse shows how fast E increases in each direction

Ellipse Interpretation

Consider the axis-aligned case (gradients - horizontal or vertical):

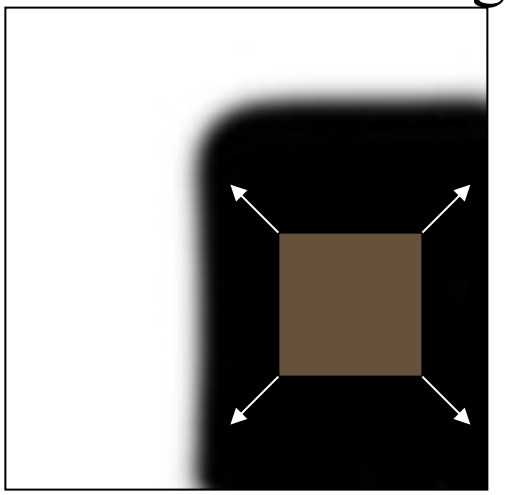
$$M = \begin{bmatrix} \sum_{x,y} I_x^2 & \sum_{x,y} I_x I_y \\ \sum_{x,y} I_x I_y & \sum_{x,y} I_y^2 \end{bmatrix} =$$

$$[u \;\; v] \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = 1$$



Eigenvalues cases

Consider the axis-aligned case (gradients - horizontal or vertical):



"flat" region:
no change in all directions

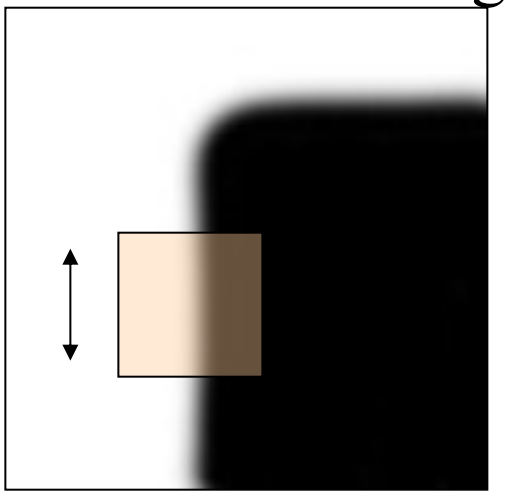
$$M = \begin{bmatrix} \sum_{x,y} I_x I_x & \sum_{x,y} I_x I_y \\ \sum_{x,y} I_x I_y & \sum_{x,y} I_y I_y \end{bmatrix}$$

$$M \approx \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

The eigenvalues of M = solutions of the equation $\det(M - \lambda I_2) = 0$ are $\lambda_1 \approx \lambda_2 \approx 0$

Eigenvalues cases

Consider the axis-aligned case (gradients - horizontal or vertical):



$$M = \begin{bmatrix} \sum_{x,y} I_x I_x & \sum_{x,y} I_x I_y \\ \sum_{x,y} I_x I_y & \sum_{x,y} I_y I_y \end{bmatrix}$$

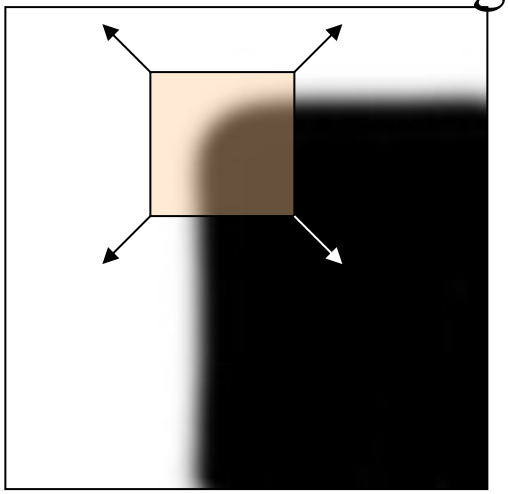
"edge":
no change along
the edge direction

$$M \approx \begin{bmatrix} \lambda & 0 \\ 0 & 0 \end{bmatrix}$$

The eigenvalues of M = solutions of the equation $\det(M - \lambda I_2) = 0$ are $\lambda_1 >> 0$, $\lambda_2 \approx 0$

Eigenvalues cases

Consider the axis-aligned case (gradients - horizontal or vertical):



$$M = \begin{bmatrix} \sum_{x,y} I_x I_x & \sum_{x,y} I_x I_y \\ \sum_{x,y} I_x I_y & \sum_{x,y} I_y I_y \end{bmatrix}$$

"corner":
significant change in all directions

$$M \approx \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$$

The eigenvalues of M = solutions of the equation $\det(M - \lambda I_2) = 0$ are $\lambda_1, \lambda_2 >> 0$

Eigenvalues cases

Consider the axis-aligned case (gradients - horizontal or vertical):

$$M = \begin{bmatrix} \sum_{x,y} I_x^2 & \sum_{x,y} I_x I_y \\ \sum_{x,y} I_x I_y & \sum_{x,y} I_y^2 \end{bmatrix} = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$$

This means dominant gradient directions align with x or y axis

Look for locations where **both** λ's are large.

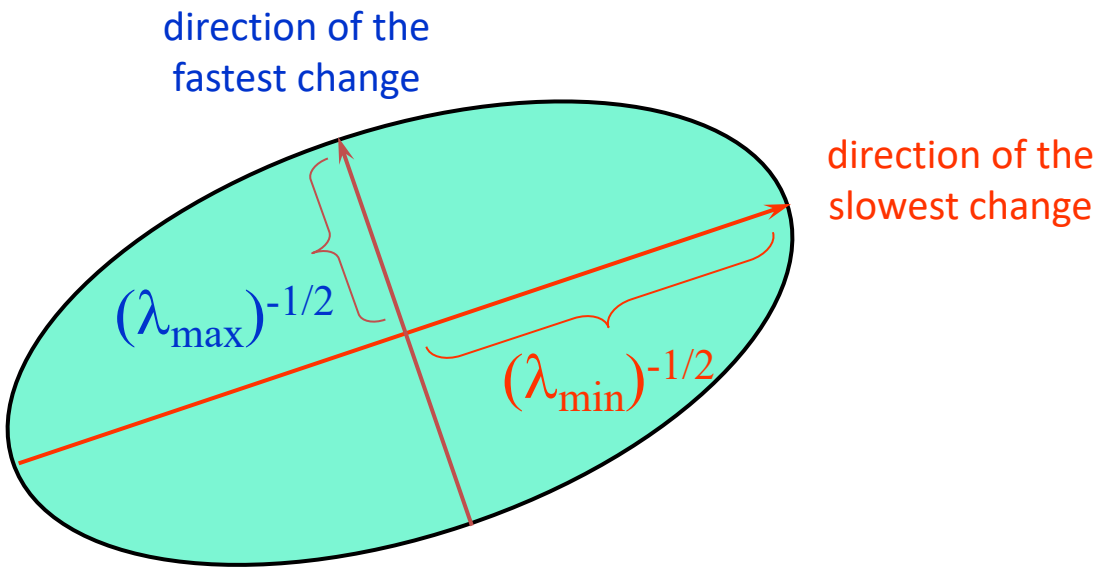
If either λ_1 or λ_2 is close to 0, then this is **not** corner-like.

General Case: Rotation Invariance

In the general case, need to *diagonalize* M:

$$M = R^{-1}\begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R$$

The axis lengths of the ellipse are determined by the eigenvalues and the orientation is determined by R:



General Case: Rotation Invariance

In the general case, need to *diagonalize* M:

$$M = R^{-1}\begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R$$

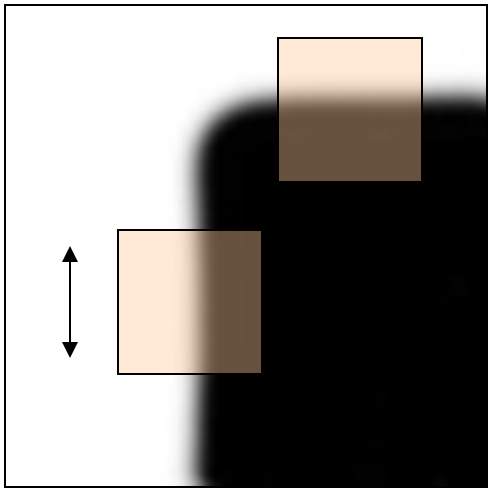
$$Mx_i = \lambda_i x_i$$

The *eigenvalues* of *M* reveal the amount of intensity change in the two principal orthogonal gradient directions in the window.

Eigenvalues: magnitude of variation
Eigenvectors: direction of variation

We don't care about orientation → only eigenvalues matter

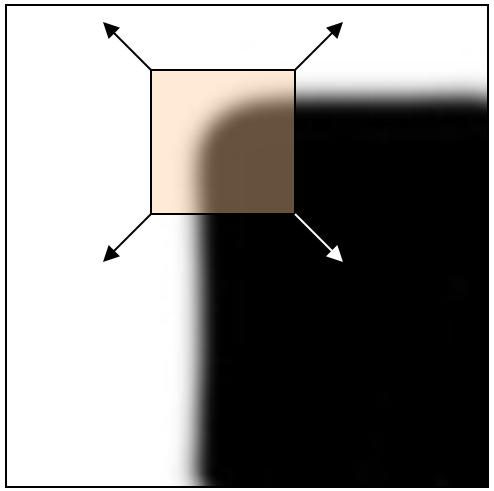
From Eigenvalues to Harris Score



"edge":

$\lambda_1 >> \lambda_2$

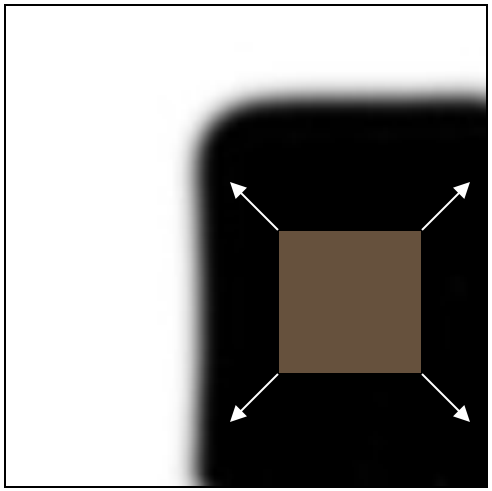
$\lambda_2 >> \lambda_1$



"corner":

λ_1 and λ_2 are large,

$\lambda_1 \sim \lambda_2$;



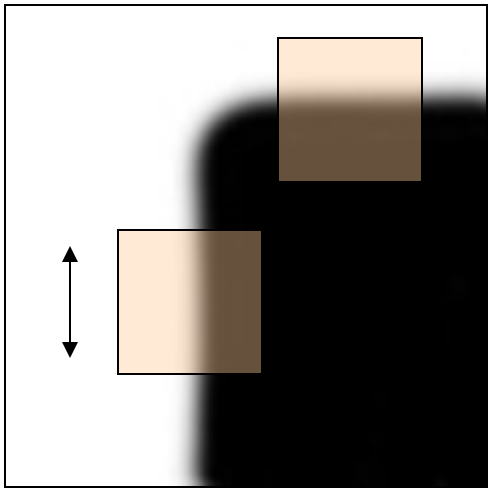
"flat" region

λ_1 and λ_2 are small

We want:

- large λ_1, λ_2
- avoid explicit eigenvalue computation

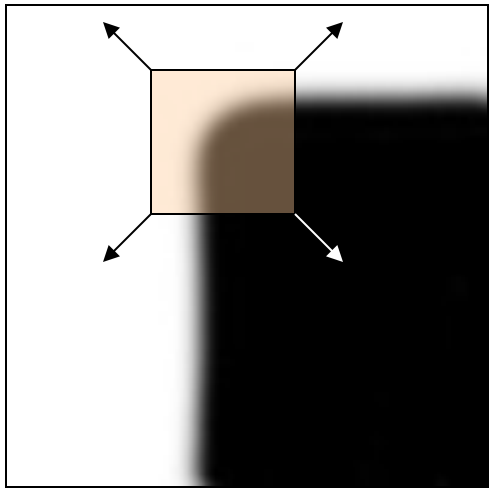
Harris corner response function



"edge":

$\lambda_1 >> \lambda_2$

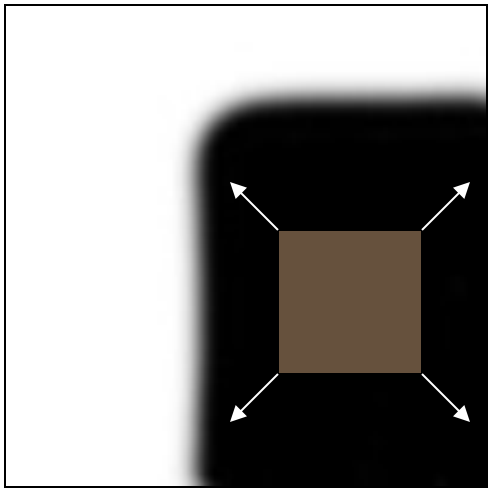
$\lambda_2 >> \lambda_1$



"corner":

λ_1 and λ_2 are large,

$\lambda_1 \sim \lambda_2$;



"flat" region

λ_1 and λ_2 are small

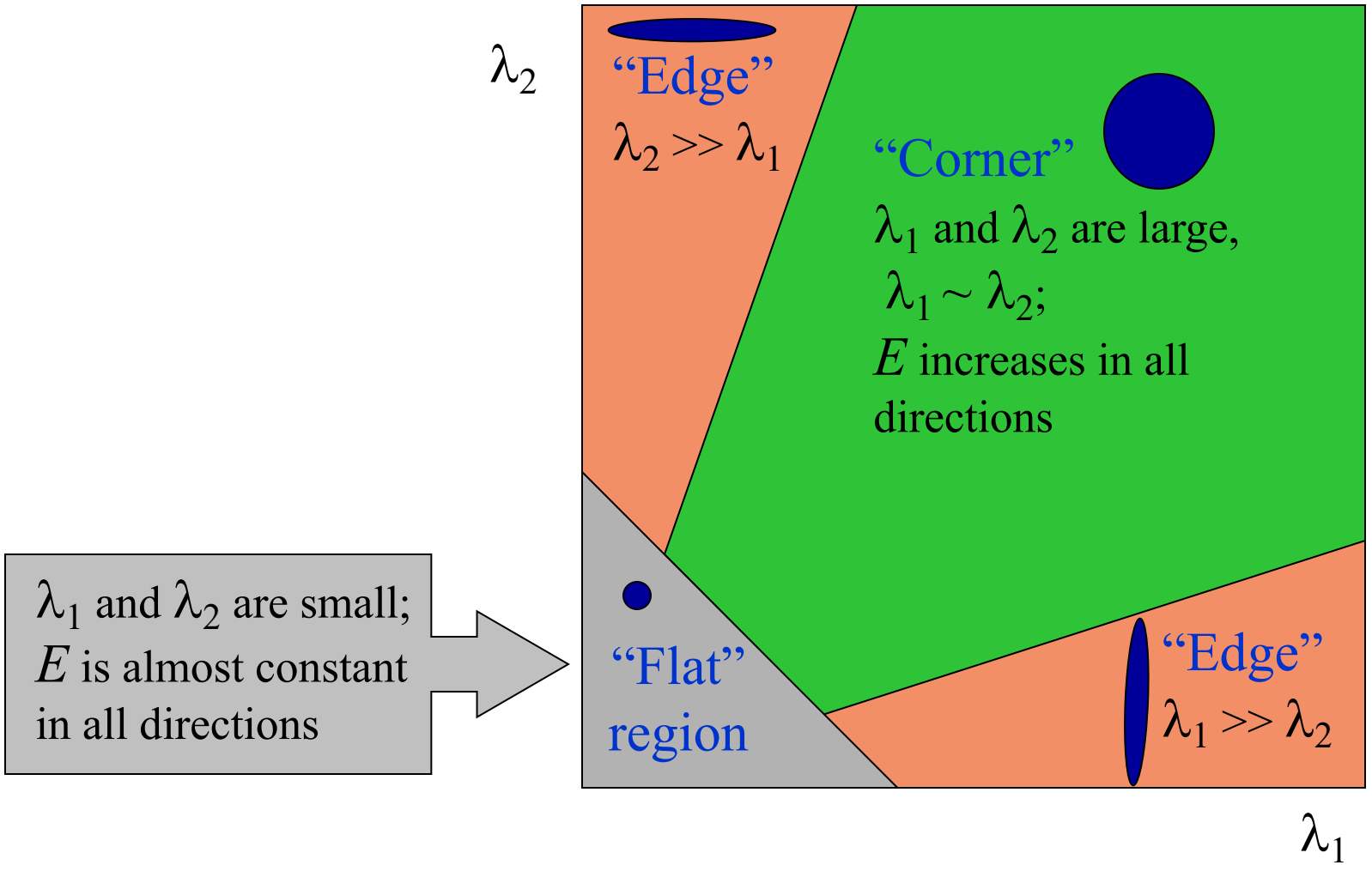
Cornerness score
(other variants possible)

$$f = \frac{\lambda_1 \lambda_2}{\lambda_1 + \lambda_2}$$

$$f = det(M) - \alpha \cdot trace(M)^2 = \lambda_1 \cdot \lambda_2 - \alpha * (\lambda_1 + \lambda_2)^2$$

Harris corner response function

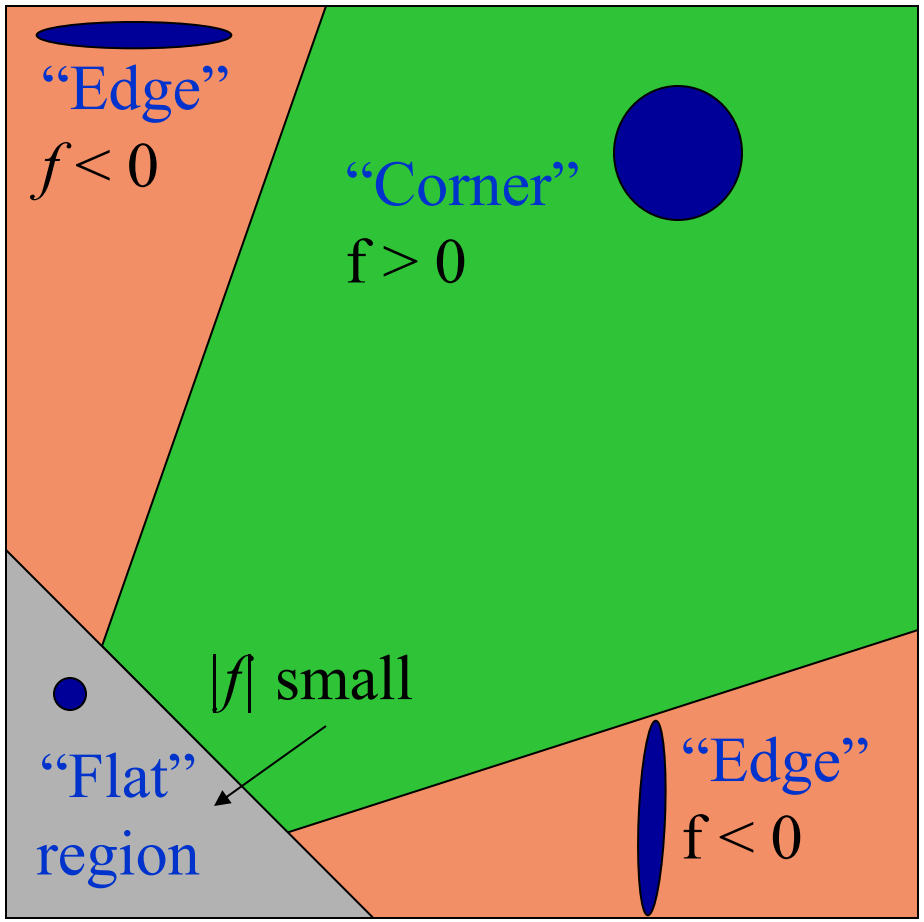
Classification of image points using eigenvalues of M:



Harris corner response function

$$f = \det(M) - \alpha \,\mathrm{trace}(M)^2 = \lambda_1\lambda_2 - \alpha(\lambda_1 + \lambda_2)^2$$

α: constant (0.04 to 0.06)



Harris corner detector algorithm

1) Compute image gradients I_x, I_y

2) Compute M matrix for each image window to get their *cornerness* scores $f = det(M) - \alpha \cdot trace(M)^2$.

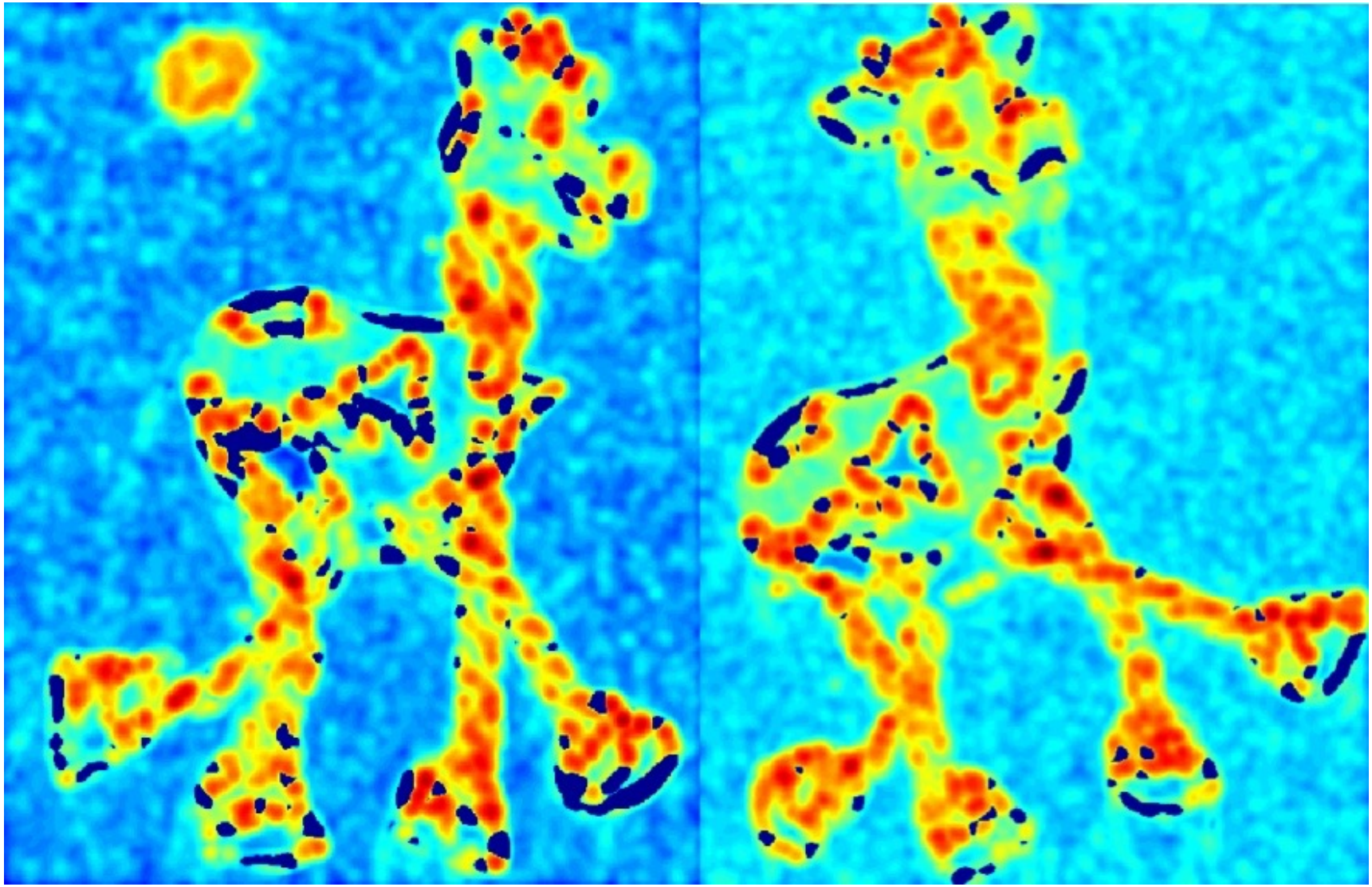
3) Find points whose surrounding window give large corner response (f > threshold)

4) Take the points of local maxima, i.e., perform non-maximum suppression

Harris Detector: Steps

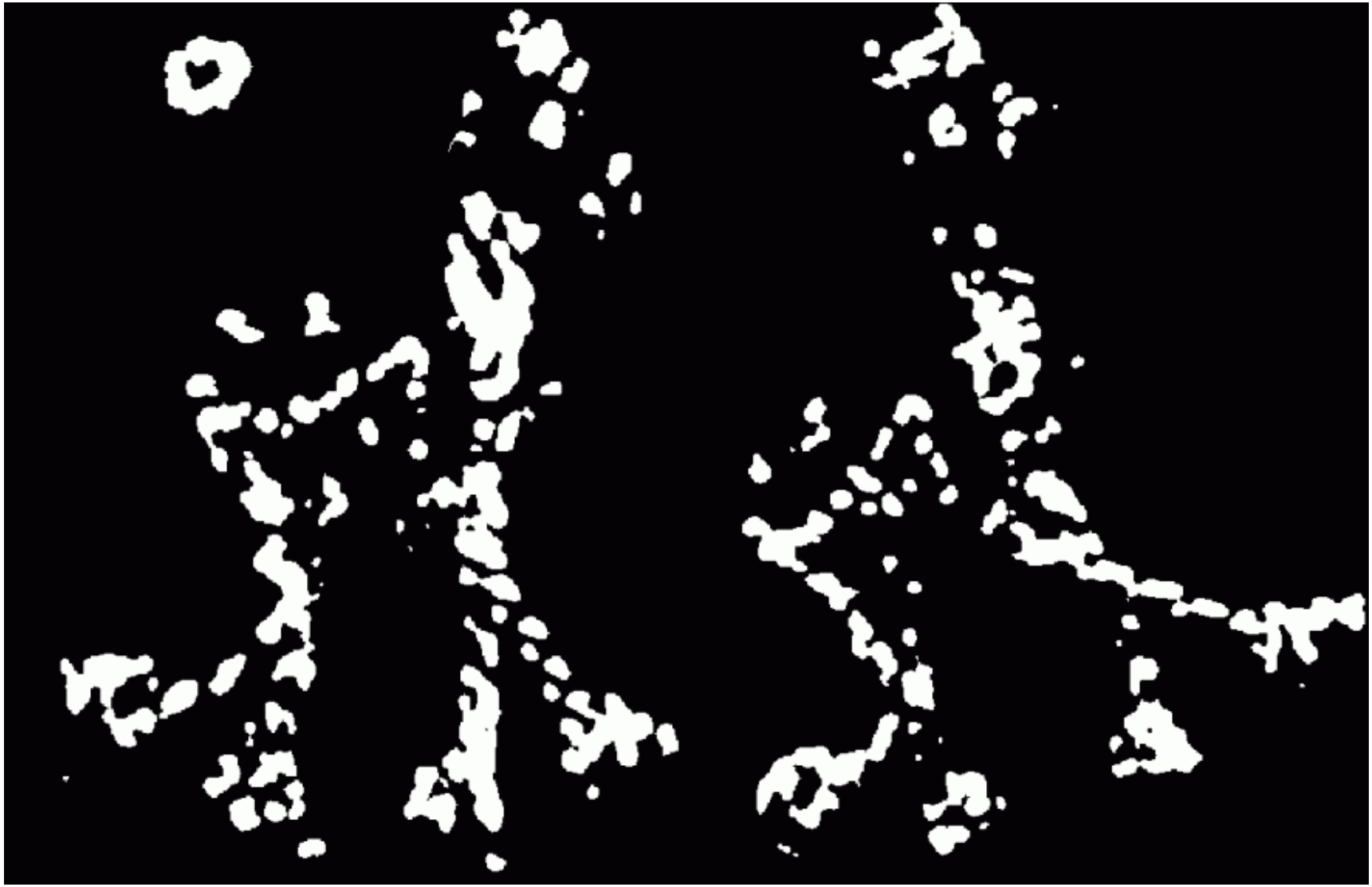
Harris Detector: Steps

Compute corner response f



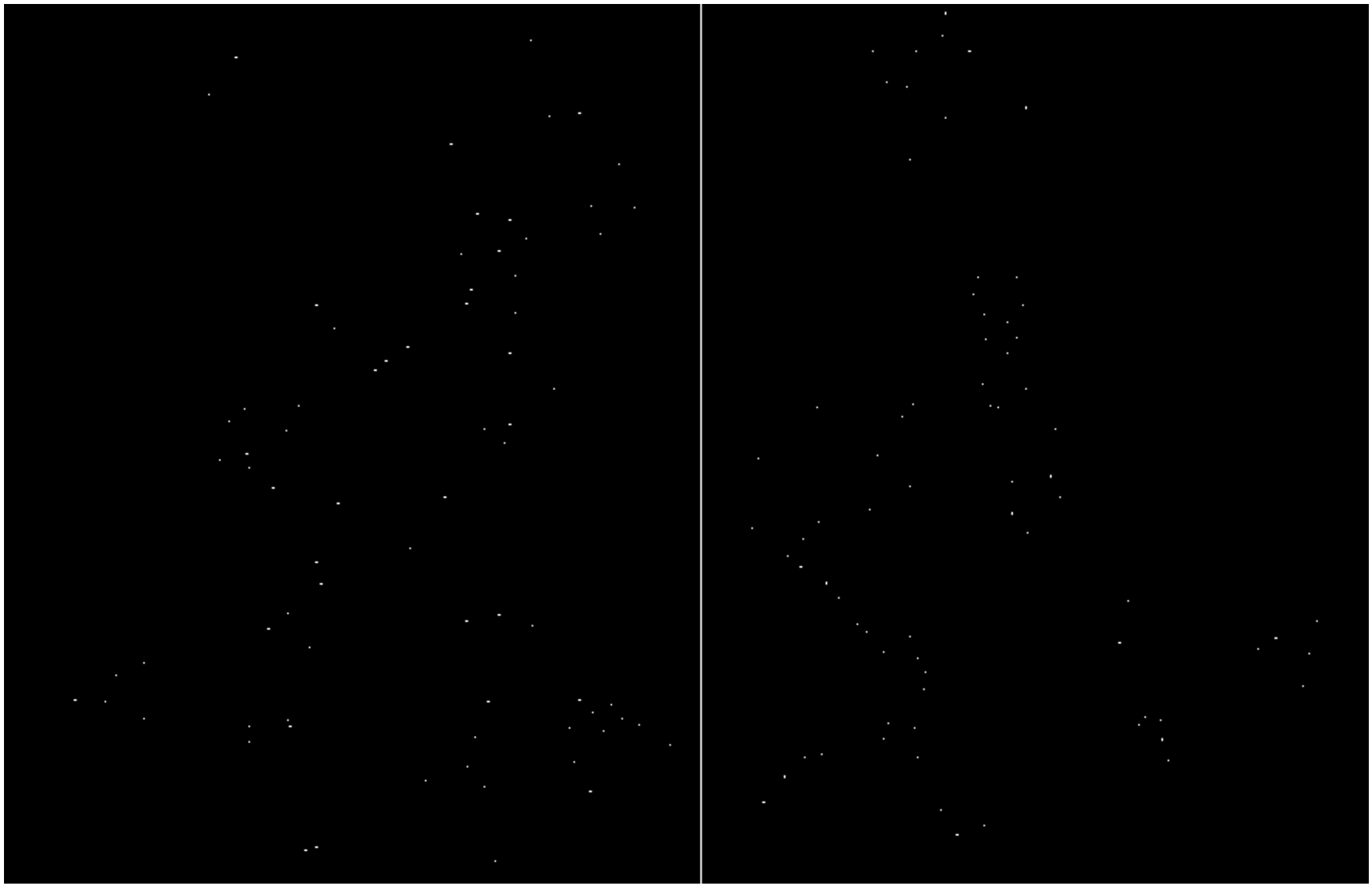
Harris Detector: Steps

Find points with large corner response: $f >$ threshold

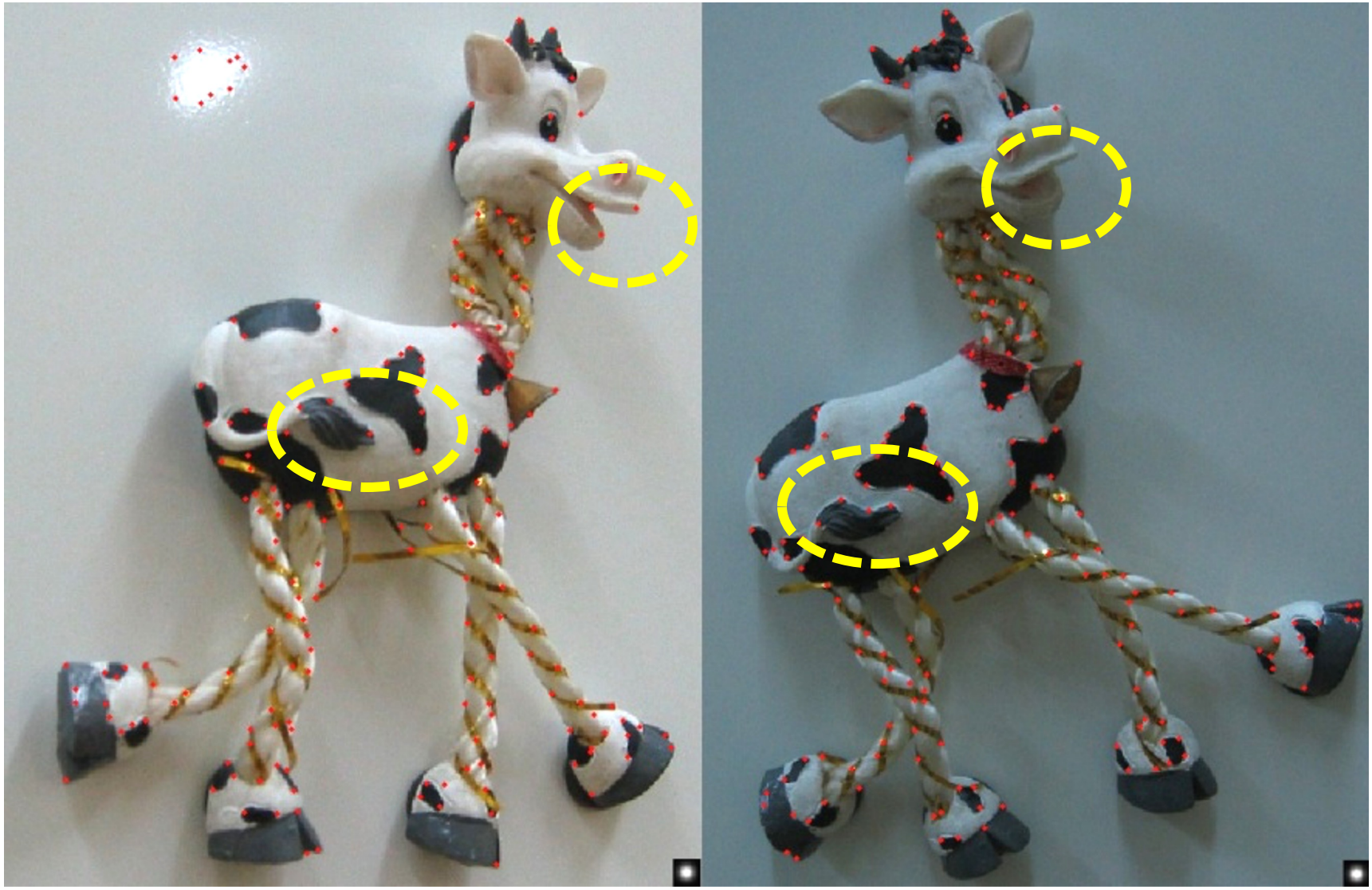


Harris Detector: Steps

Take only the points of local maxima of f



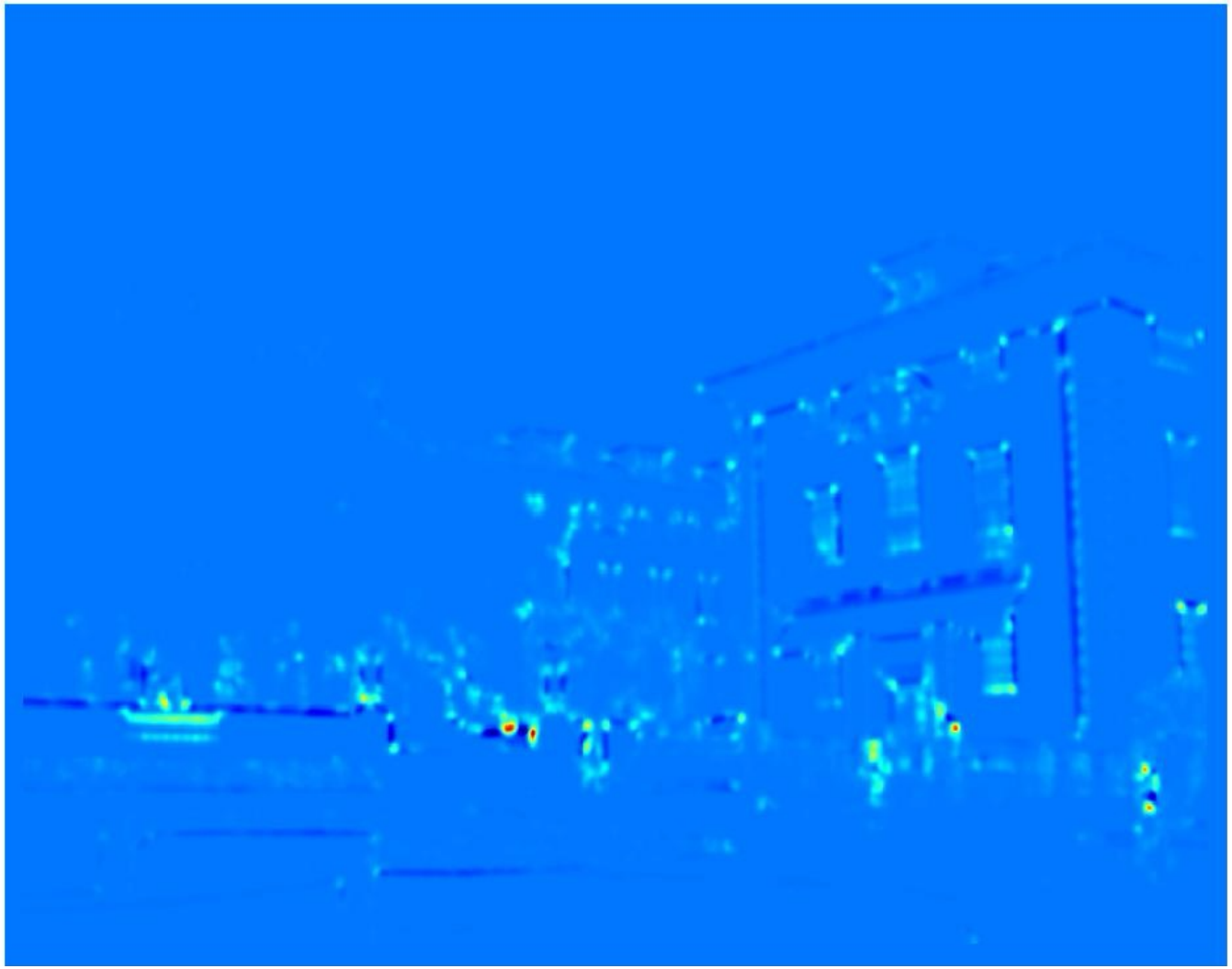
Harris Detector: Steps



Harris Detector: Steps

Harris Detector: Steps

Compute corner response f



Harris Detector: Steps

Take only the points of local maxima of f

Robustness of corner features

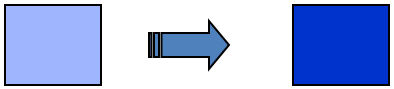
- What happens to corner features when the image undergoes geometric or photometric transformations?

Invariance and covariance

We want to find corners that are:

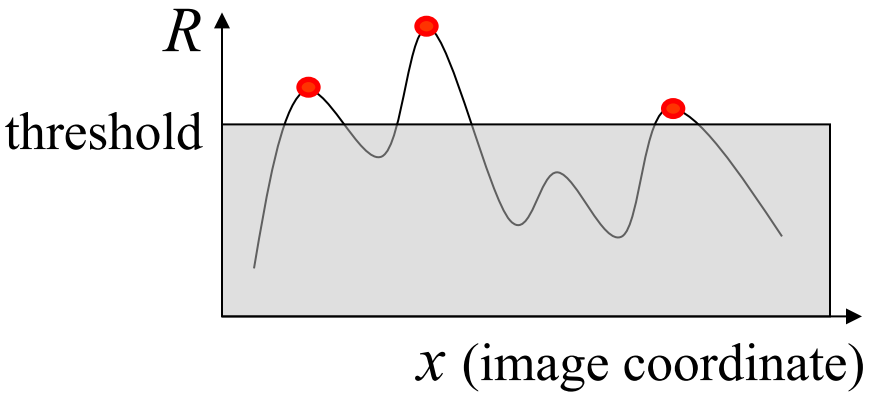
- ***invariant*** to ***photometric transformations***
- ***covariant*** to ***geometric transformation***

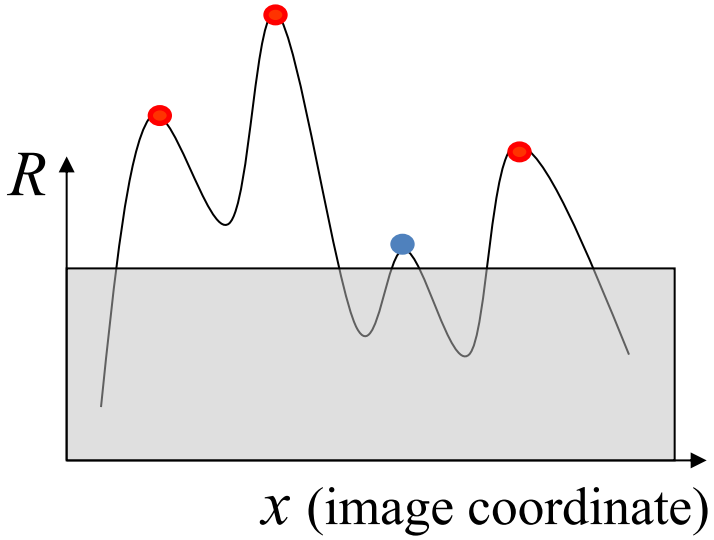
Photometric transformations: affine intensity changes



$I \rightarrow a\,I + b$

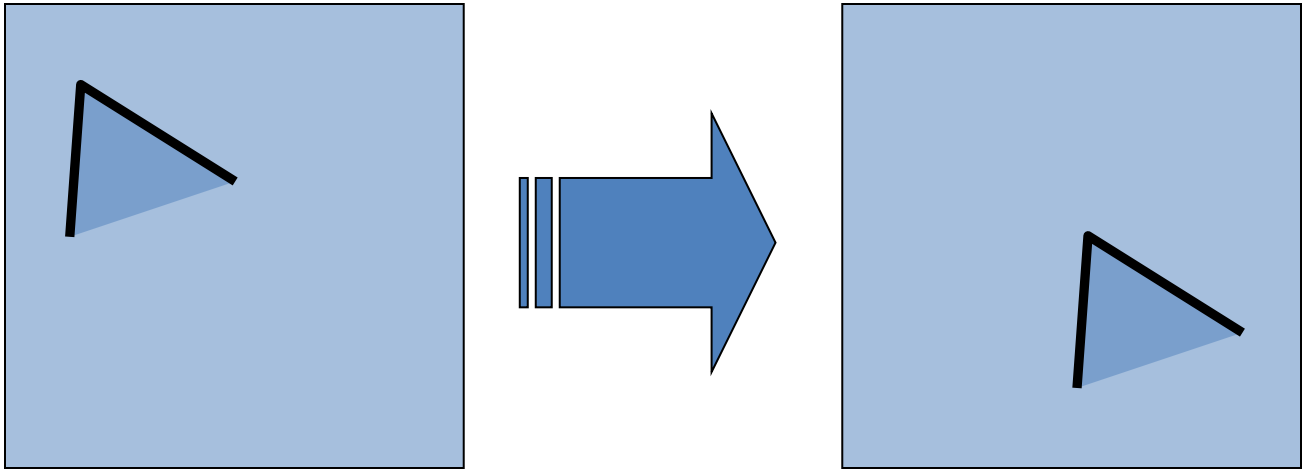
- Only derivatives are used, so invariant to intensity shift $I \rightarrow I + b$
- Intensity scaling: $I \rightarrow a\,I$





Partially invariant to affine intensity change

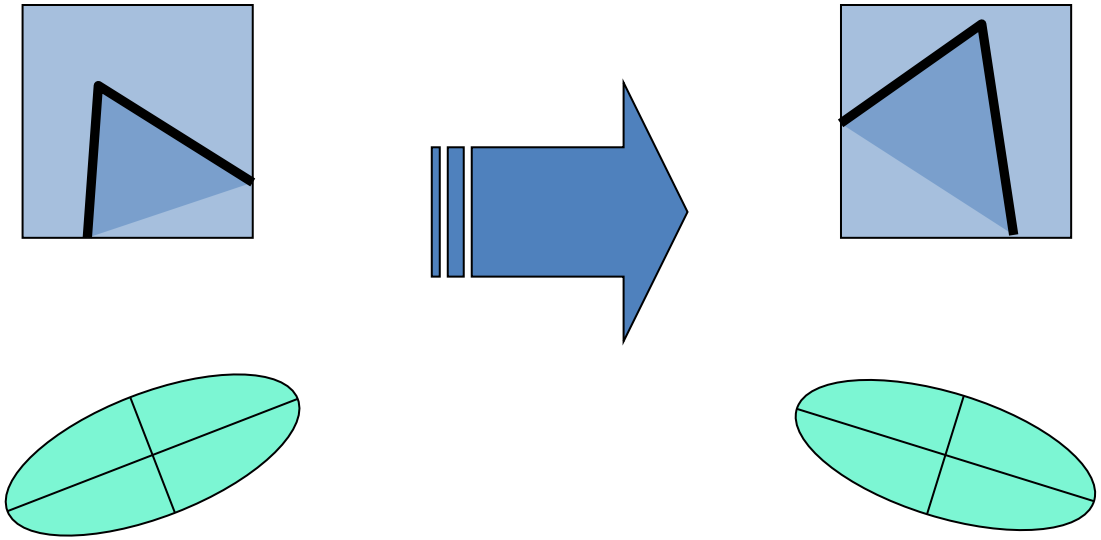
Geometric transformations: translations



- Derivatives and window function are shift-invariant

Corner location is *covariant* w.r.t. translation

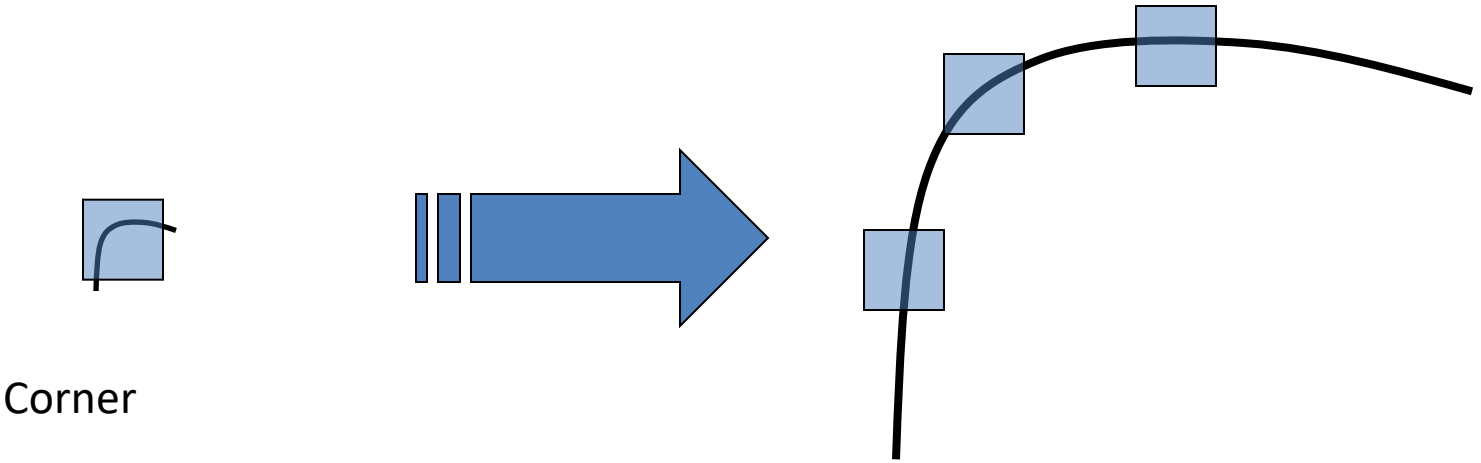
Geometric transformations: rotations



Second moment ellipse rotates but its shape (i.e. eigenvalues) remains the same

Corner location is covariant w.r.t. rotation

Geometric transformations: scaling



Corner

All points will be classified as edges

Corner location is not covariant w.r.t. scaling!

Robustness of corner features

- What happens to corner features when the image undergoes geometric or photometric transformations?
- We want to find corners that are:
  - ***invariant*** to ***photometric transformations***
  - ***covariant*** to ***geometric transformation***

Properties of Harris corner detector

Partially invariant to affine intensity change

Corner location is *covariant* w.r.t. translation

Corner location is covariant w.r.t. rotation

Corner location is not covariant w.r.t. scaling!

Harris is not scale invariant

- Everything depends on a fixed window W with a fixed scale.
- But what happens if the image is scaled?
  - small corner → becomes large structure
  - same window → now sees only part of it
- Harris detector is not invariant to scale changes
- We need to detect features at the appropriate scale

Scale invariant keypoints

How can we detect keypoints such that they are repeatable across different image scales?

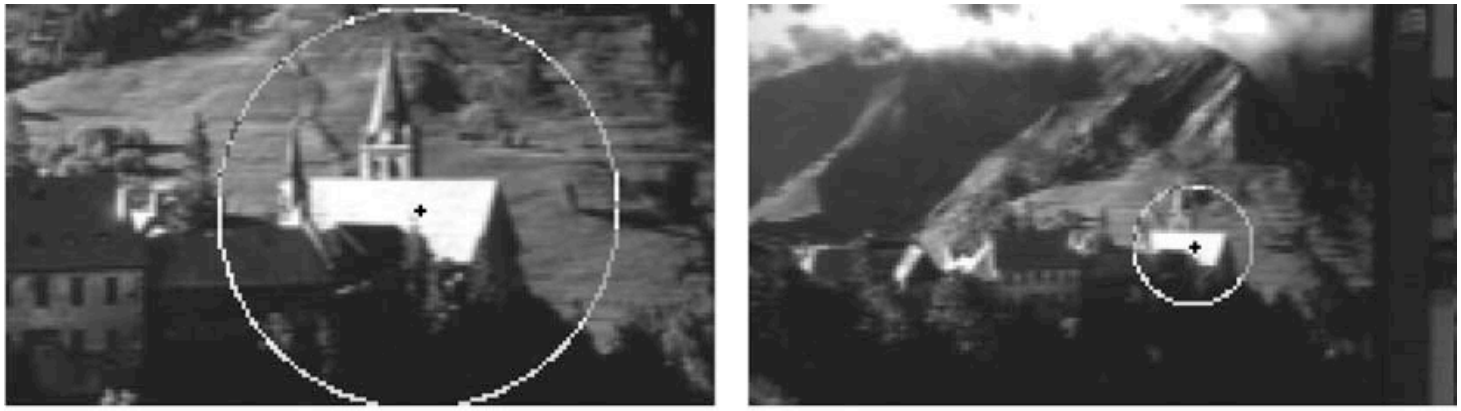
Detecting local invariant features

- Detection of keypoints (Where?)
  - identify repeatable and distinctive locations
  - typical methods:
    - Harris Corner Detector → corners (intensity variation in all directions)
    - LoG (Laplacian of Gaussian) → blob detection (scale-invariant)

- Description of local patches (What?)
  - represent each keypoint by a feature vector
  - encodes the appearance of the local neighborhood
  - should be:
    - invariant (scale, rotation, illumination)
    - discriminative

Keypoint detection with scale selection

- We want to extract keypoints with *characteristic scales* that are *covariant* w.r.t. the image transformation



- The same structure appears at different sizes in different images. The same keypoint should be detected but with a different optimal scale. We want features that are:
  - repeatable across scales
  - covariant w.r.t. scaling

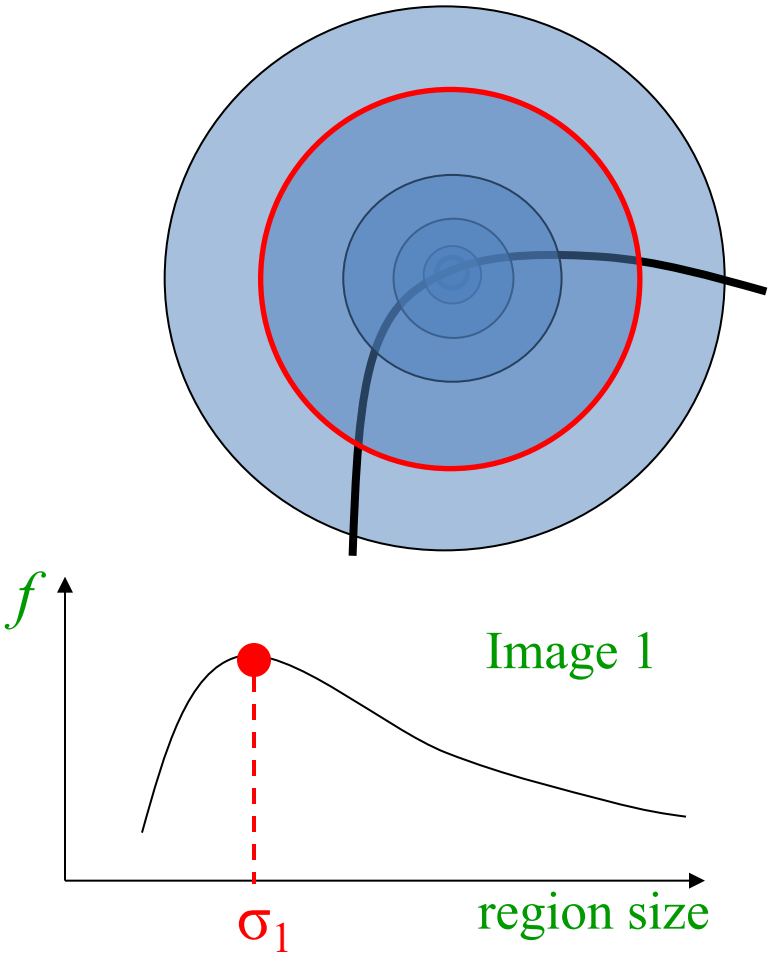
Automatic Scale Selection

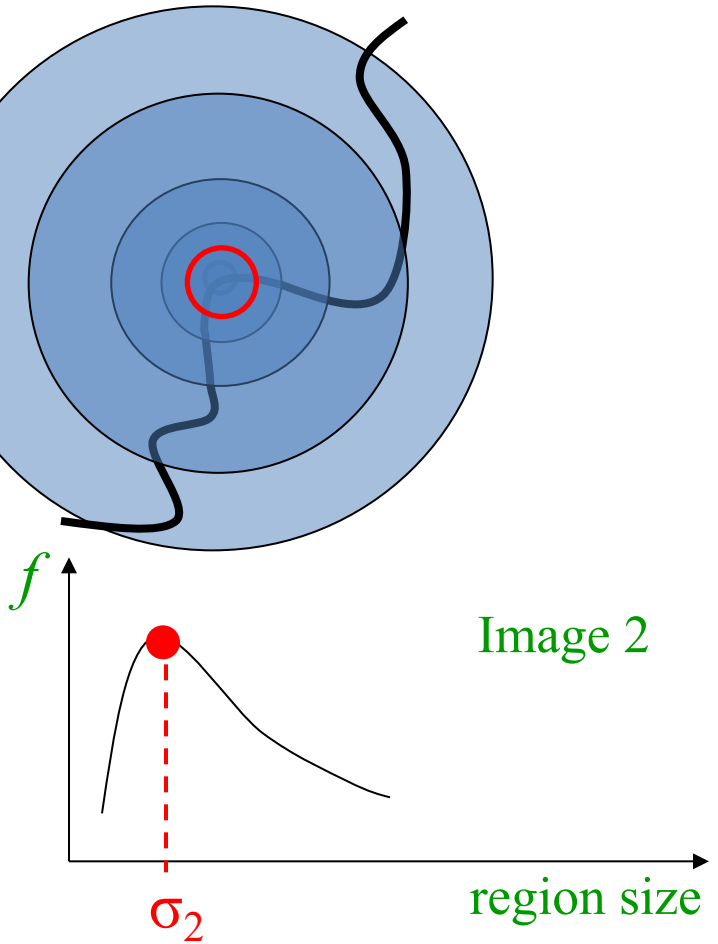
- How do we choose the right patch size for a feature when objects appear at different scales?
- The same physical point appears at different sizes. A fixed patch size will fail.
- Analyze the image at multiple scales σ. For each point → choose the scale where the response of some function f is maximal. This gives the characteristic scale.

Automatic Scale Selection

Intuition:

- Find scale that gives local maxima of some function *f* in both position x and scale σ.





Automatic Scale Selection

Intuition:

- Find scale that gives local maxima of some function *f* in both position x and scale σ.

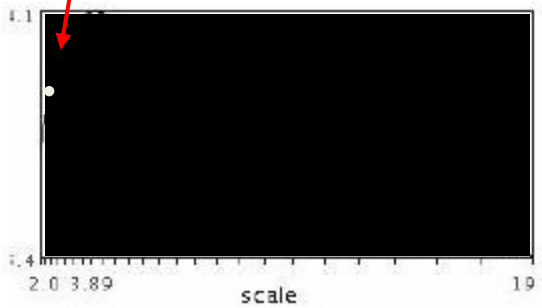
- Peaks correspond to the same structure at different image scales

- For each point, we don't fix a scale - we scan all scales. And we simply pick the scale where the response is strongest.

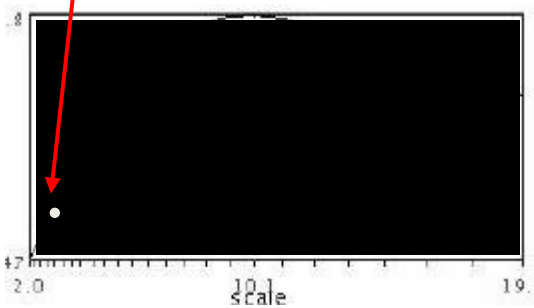
- Characteristic scale = scale where the response is maximal

Automatic Scale Selection

- scale signature: response of a point as a function of scale



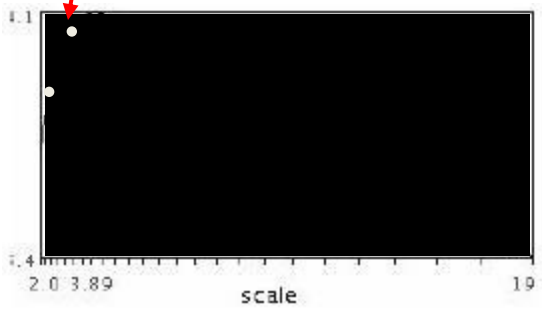
$f(I_{i_1 \ldots i_m}(x,\sigma))$



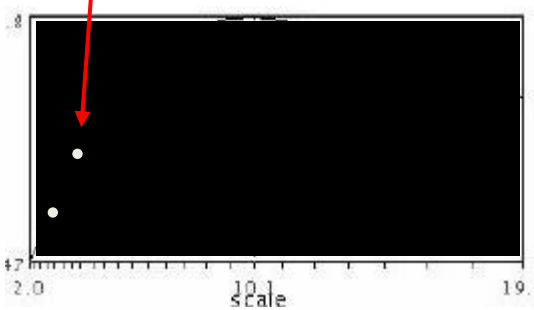
$f(I_{i_1 \ldots i_m}(x',\sigma))$

Automatic Scale Selection

- scale signature: response of a point as a function of scale



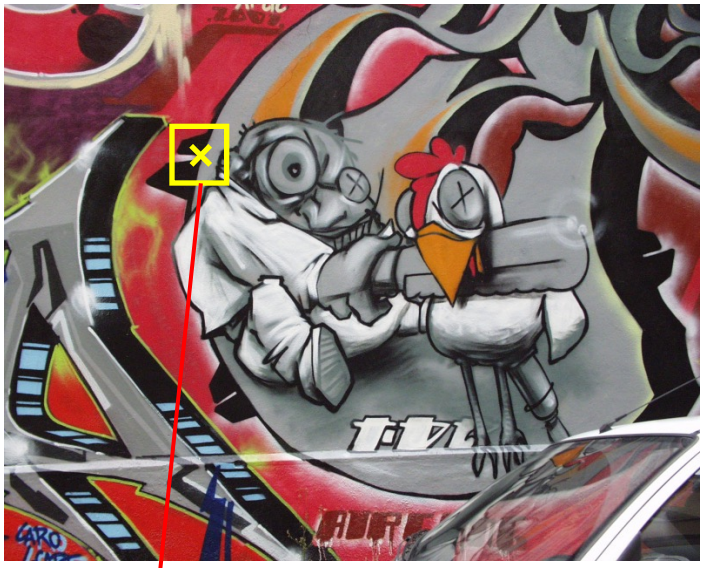
$f(I_{i_1 \ldots i_m}(x,\sigma))$

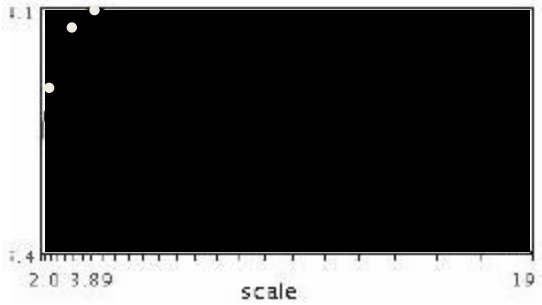


$f(I_{i_1 \ldots i_m}(x',\sigma))$

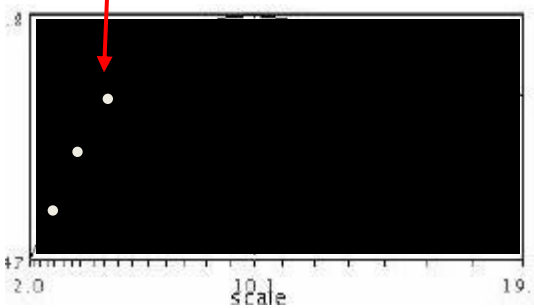
Automatic Scale Selection

- scale signature: response of a point as a function of scale





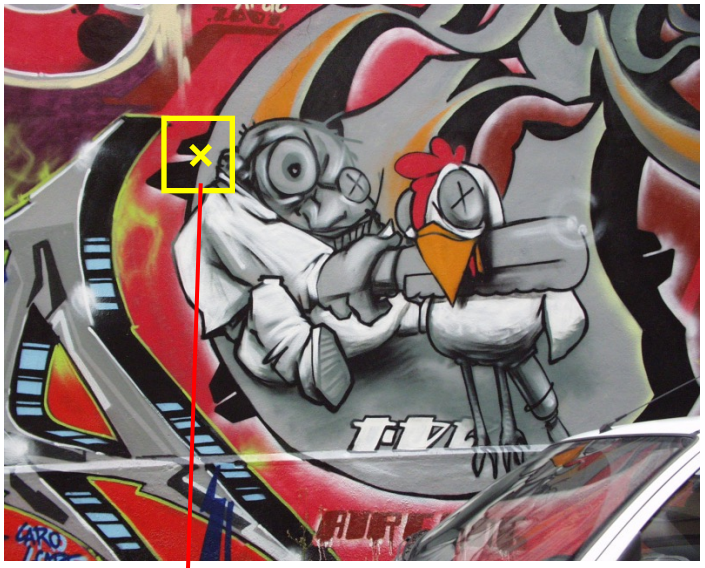
$f(I_{i_1 \ldots i_m}(x,\sigma))$

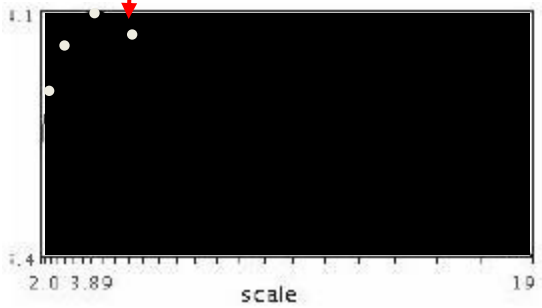


$f(I_{i_1 \ldots i_m}(x',\sigma))$

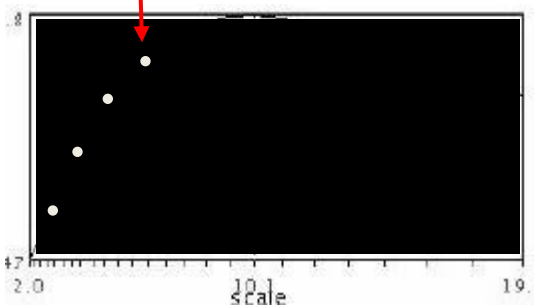
Automatic Scale Selection

- scale signature: response of a point as a function of scale





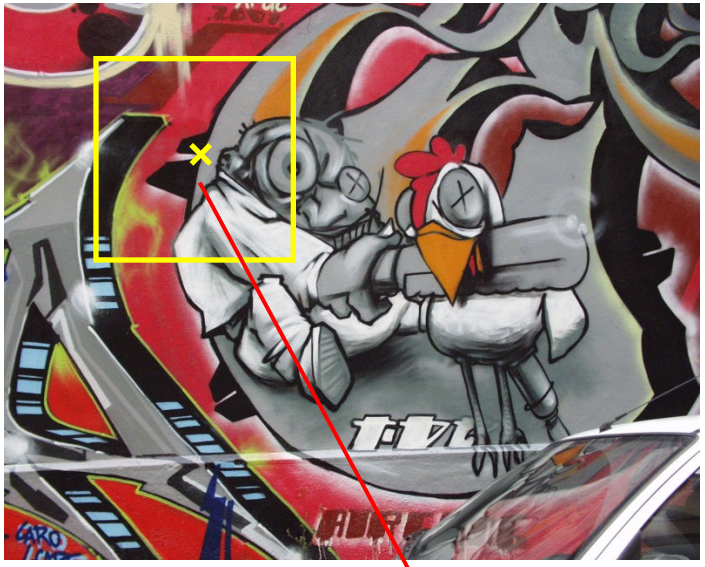
$f(I_{i_1 \ldots i_m}(x,\sigma))$

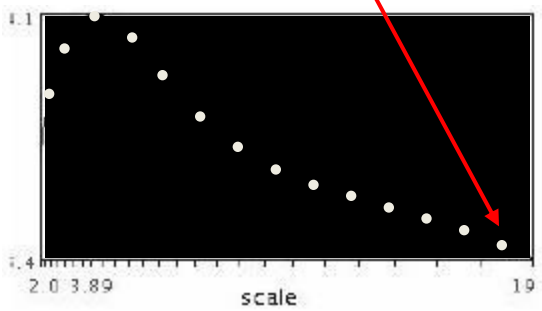


$f(I_{i_1 \ldots i_m}(x',\sigma))$

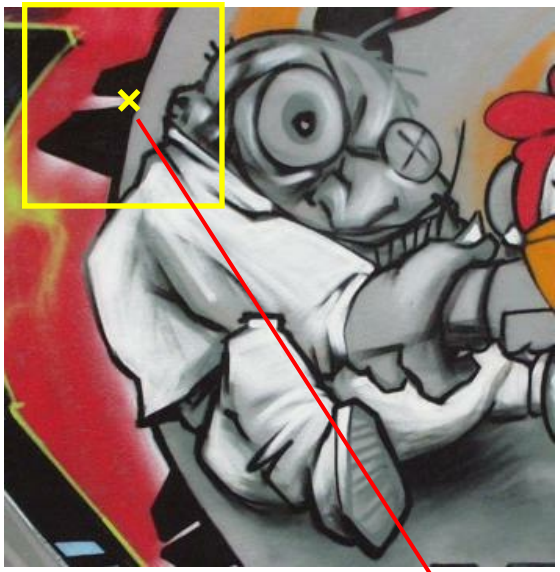
Automatic Scale Selection

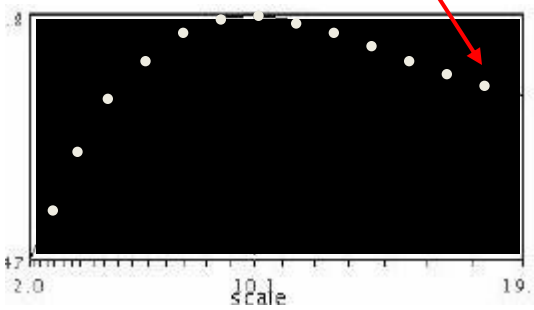
- scale signature: response of a point as a function of scale





$f(I_{i_1 \ldots i_m}(x,\sigma))$

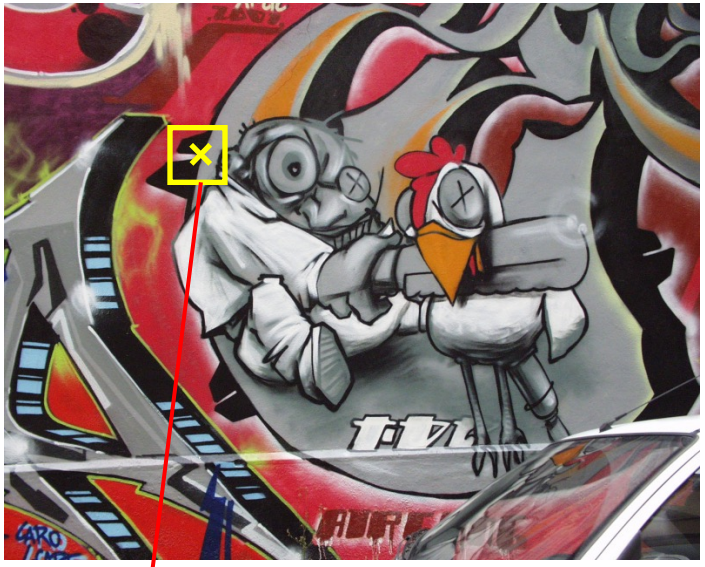


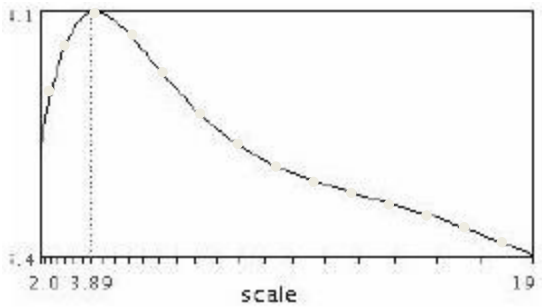


$f(I_{i_1 \ldots i_m}(x',\sigma))$

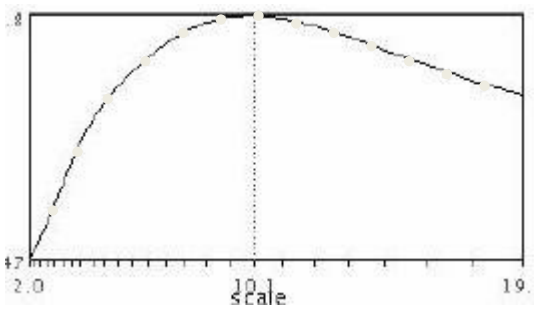
Automatic Scale Selection

- scale signature: response of a point as a function of scale





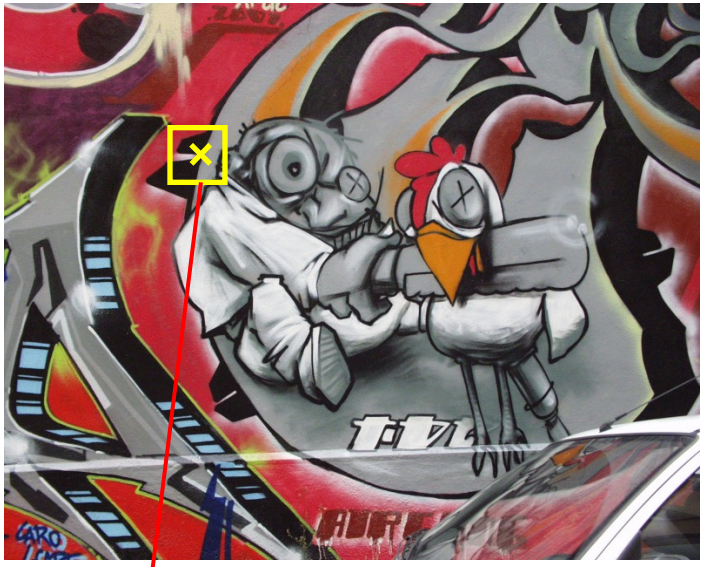
$f(I_{i_1 \ldots i_m}(x,\sigma))$

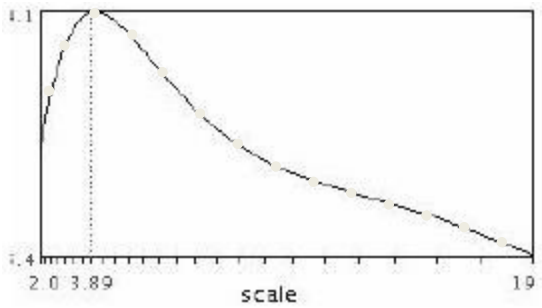


$f(I_{i_1 \ldots i_m}(x',\sigma'))$

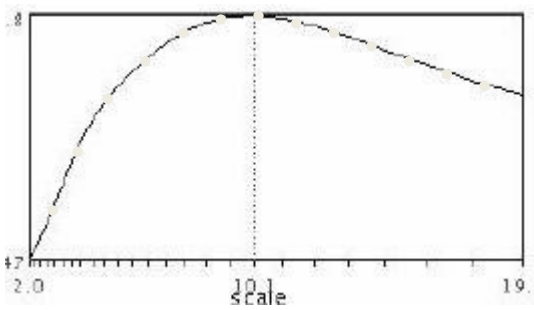
Automatic Scale Selection

- scale signature: response of a point as a function of scale





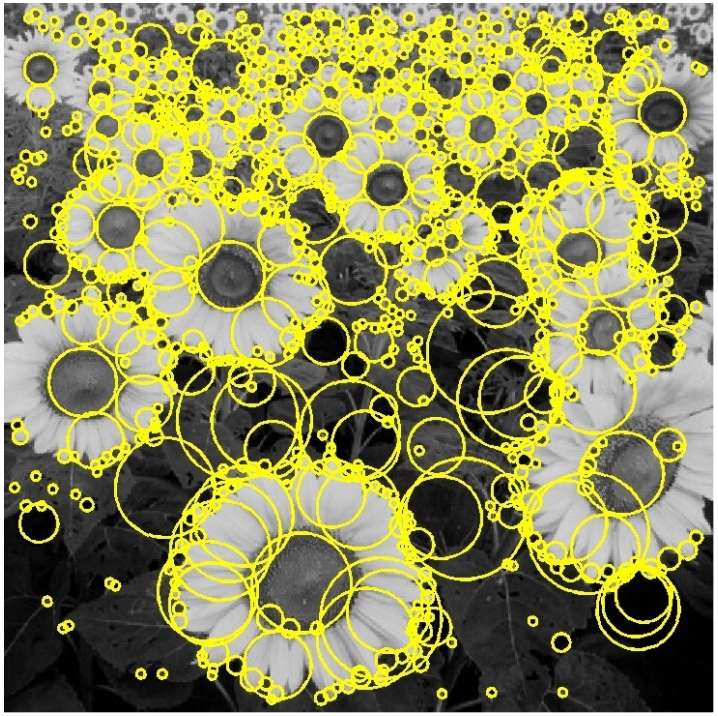
$f(I_{i_1 \ldots i_m}(x,\sigma))$

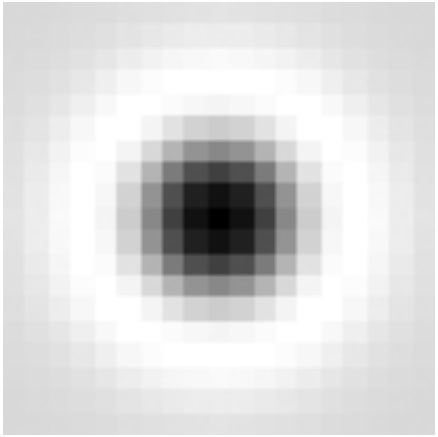


$f(I_{i_1 \ldots i_m}(x',\sigma'))$

Basic idea

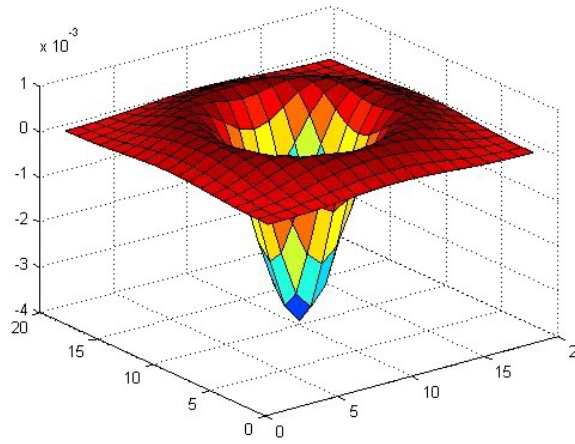
- Convolve the image with a "blob filter" at multiple scales and look for extrema of filter response in the resulting *scale space*





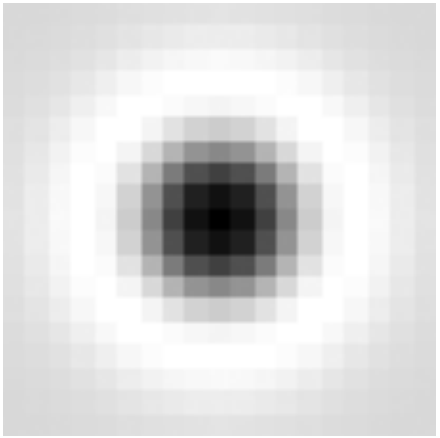
T. Lindeberg, Feature detection with automatic scale selection, *IJCV* 30(2), pp 77-116, 1998

Laplacian of Gaussian (LoG) for Blob Detection



- Detect blob-like structures in the image

$$\nabla^2 g = \frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2}$$

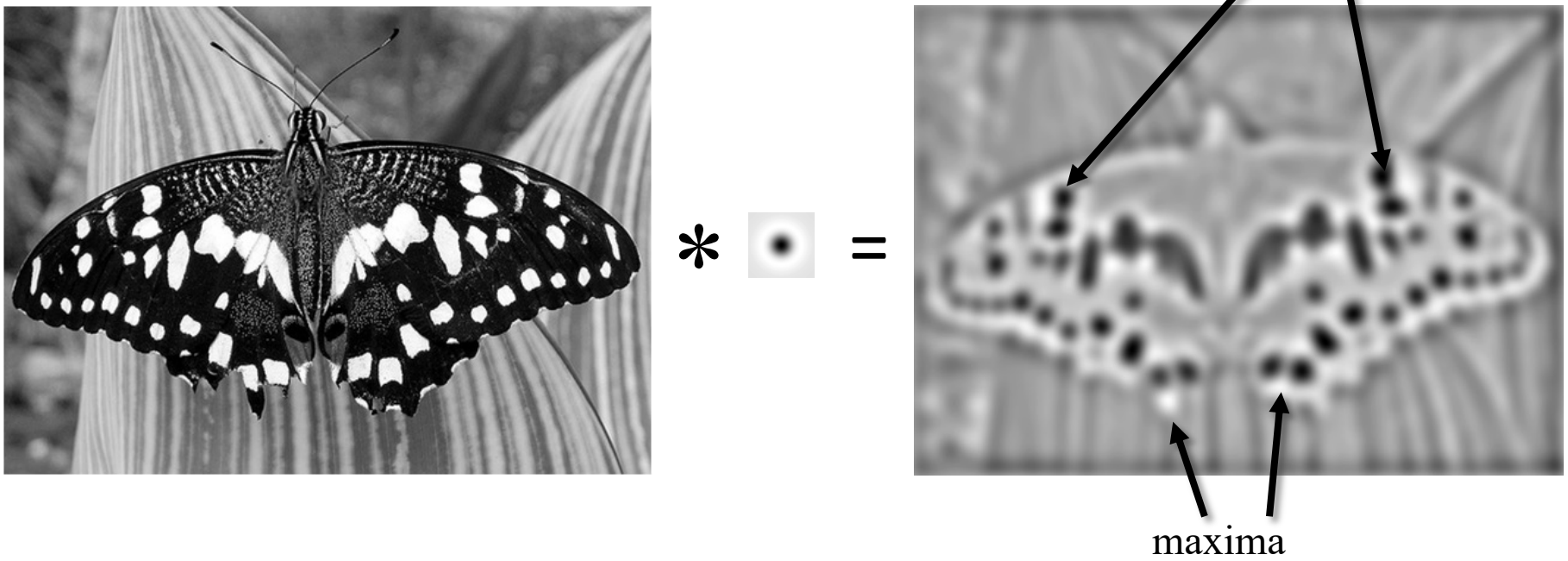


- Circularly symmetric → detects blobs (not edges)
- Responds to:
  - bright blobs (minima)
  - dark blobs (maxima)
- LoG acts like a "blob detector" (Mexican hat filter)

  → strong response when the filter matches the blob size

Blob detection with LoG

- Convolve the image with a blob detector (LoG)
- Detect extrema in space AND scale of the response
- Keypoints = local extrema of LoG response in scale-space (x,y,σ)
- Minima → bright blobs on dark background
- Maxima → dark blobs on bright background



Blob detection with LoG

- Laplacian of Gaussian: Circularly symmetric operator for blob detection in 2D

$$g(x, y; \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y2}{2\sigma^2}}$$

$$\nabla^2 g = \frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2}$$

$$\frac{\partial g}{\partial x} = \frac{1}{2\pi\sigma^2}(-\frac{2x}{2\sigma^2})e^{-\frac{x^2+y2}{2\sigma^2}} = \frac{-x}{2\pi\sigma^4} e^{-\frac{x^2+y2}{2\sigma^2}}$$

$$\frac{\partial^2 g}{\partial x^2} = \frac{-1}{2\pi\sigma^4} e^{-\frac{x^2+y2}{2\sigma^2}} + \frac{x^2}{2\pi\sigma^6} e^{-\frac{x^2+y2}{2\sigma^2}} = \frac{x^2-\sigma^2}{2\pi\sigma^6} e^{-\frac{x^2+y2}{2\sigma^2}}$$

Blob detection with LoG

- Laplacian of Gaussian: Circularly symmetric operator for blob detection in 2D

$$\nabla^2 g = \frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2} \qquad g(x, y; \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y2}{2\sigma^2}}$$

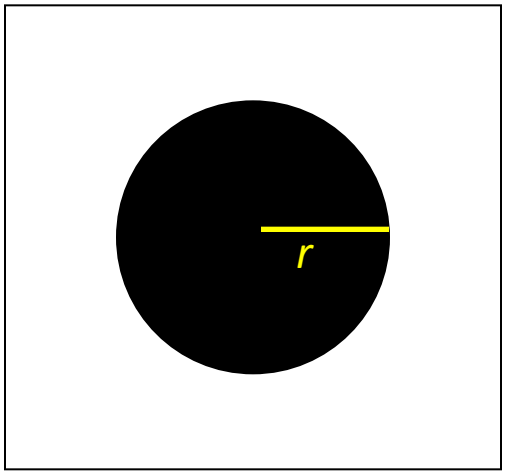
$$\frac{\partial^2 g}{\partial x^2} = \frac{-1}{2\pi\sigma^4} e^{-\frac{x^2+y2}{2\sigma^2}} + \frac{x^2}{2\pi\sigma^6} e^{-\frac{x^2+y2}{2\sigma^2}} = \frac{x^2-\sigma^2}{2\pi\sigma^6} e^{-\frac{x^2+y2}{2\sigma^2}}$$

$$\frac{\partial^2 g}{\partial y^2} = \frac{y^2-\sigma^2}{2\pi\sigma^6} e^{-\frac{x^2+y2}{2\sigma^2}}$$

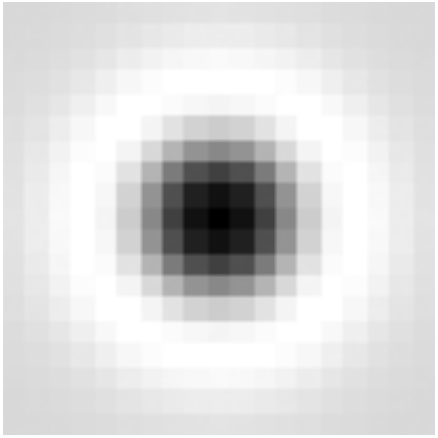
$$\nabla^2 g = \frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2} = \frac{x^2+y^2-\sigma^2}{2\pi\sigma^6} e^{-\frac{x^2+y2}{2\sigma^2}}$$

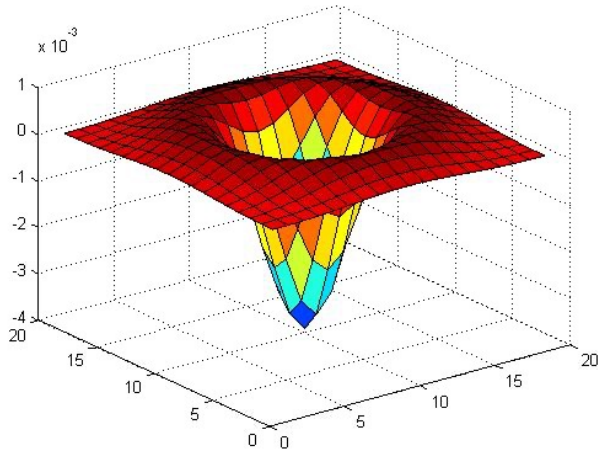
Blob detection with LoG

- At what scale does the Laplacian achieve a maximum response to a binary circle of radius r?



image





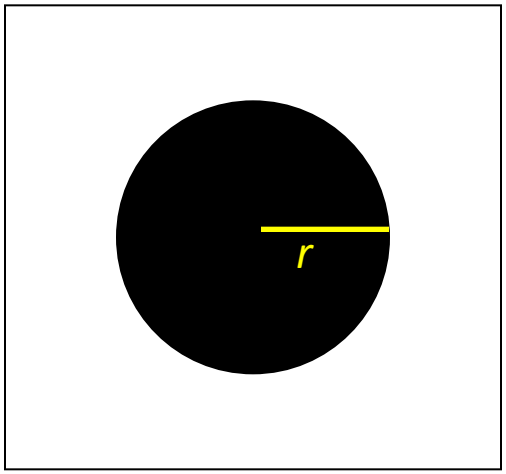
Laplacian

Blob detection with LoG

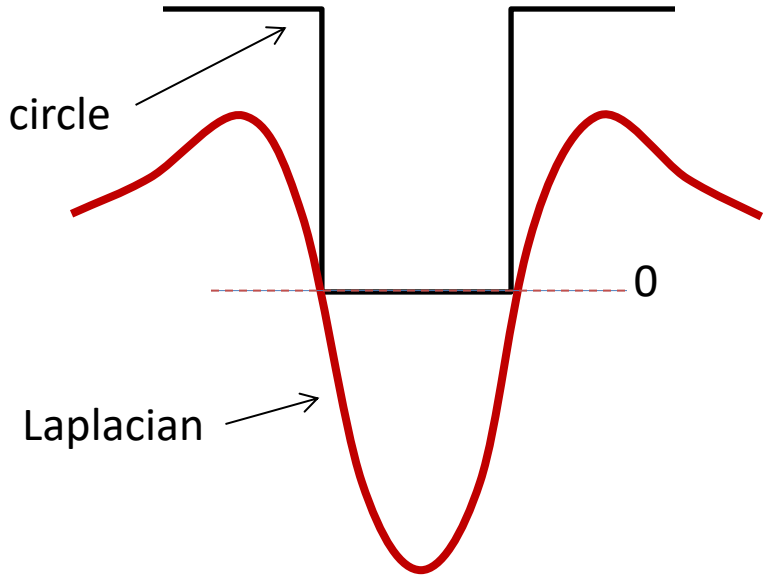
- At what scale does the Laplacian achieve a maximum response to a binary circle of radius r?
- To get maximum response, the zeros of the Laplacian have to be aligned with the circle
- The Laplacian is given by (up to scale):

$$(x^2 + y^2 - 2\sigma^2)\, e^{-(x^2+y^2)/2\sigma^2}$$

- Therefore, the maximum response occurs at $\sigma = r/\sqrt{2}.$

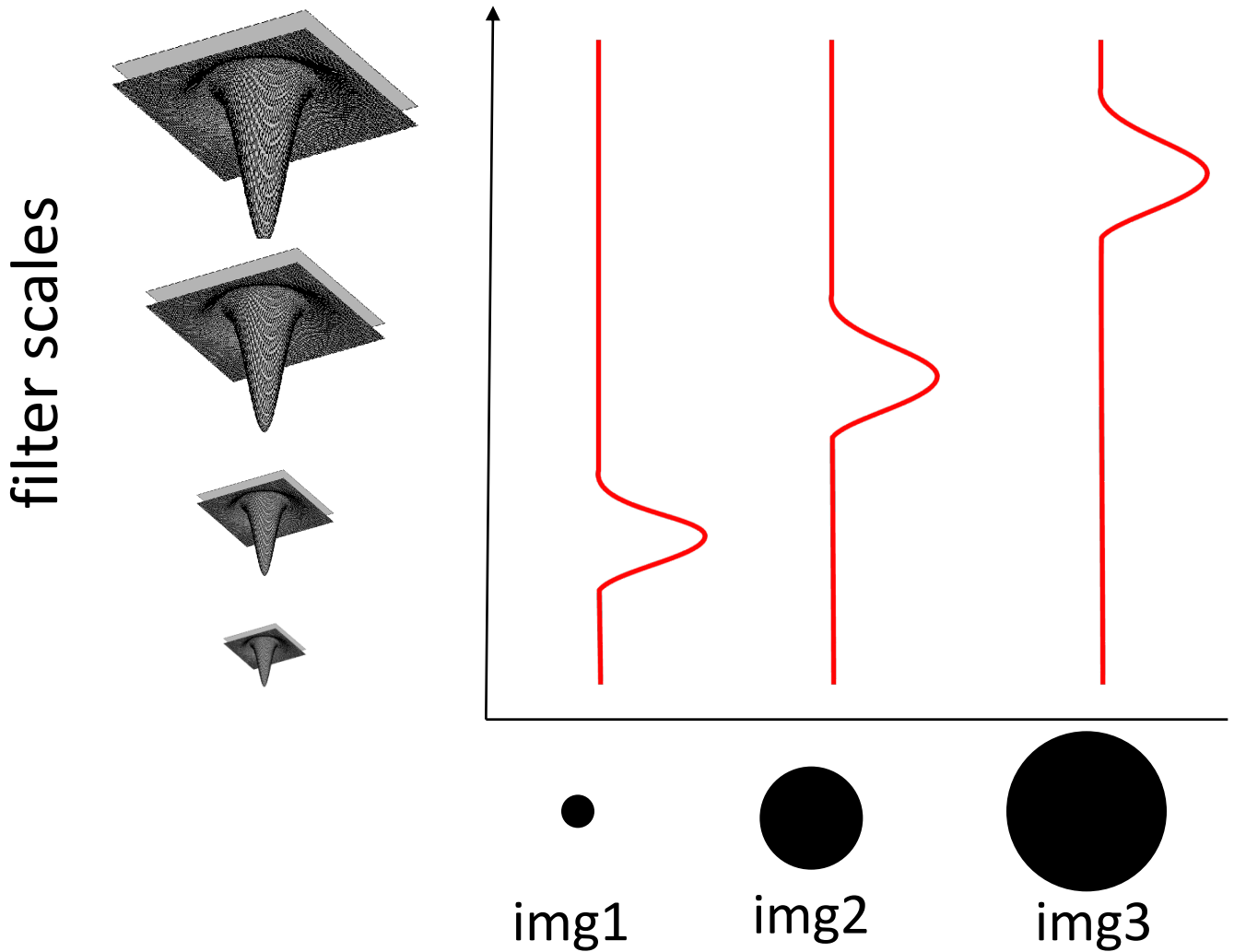


image



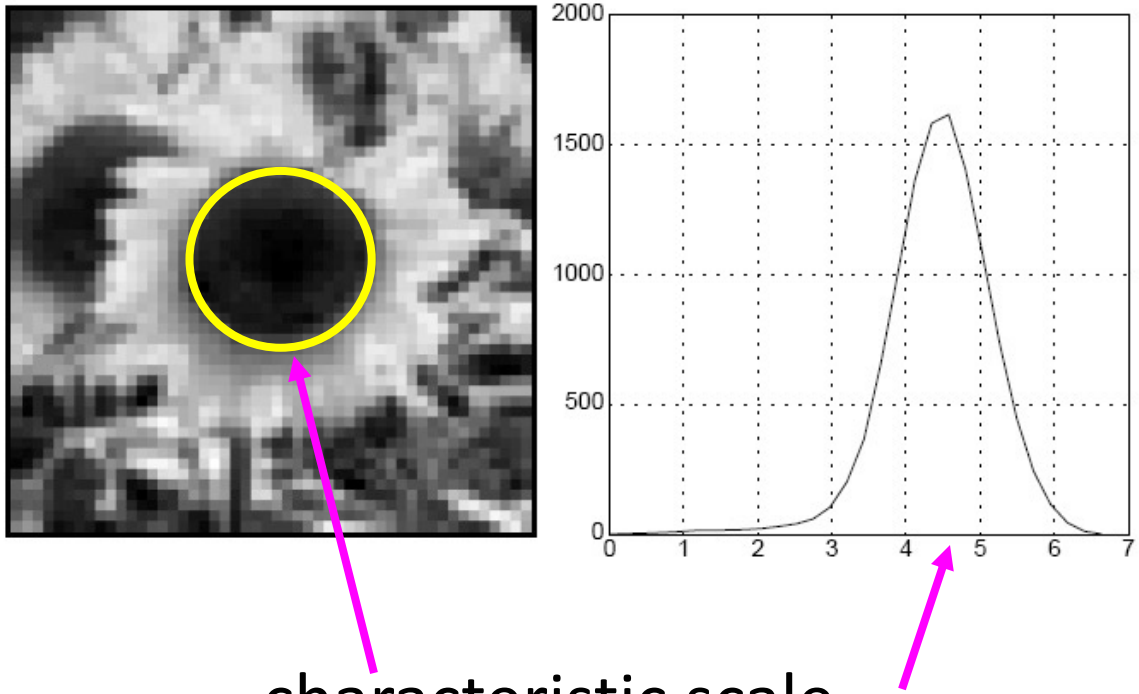
Blob detection with LoG: scale selection

- Laplacian-of-Gaussian = "blob" detector $\nabla^2 g = \frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2}$



Blob detection in 2D

- We define the *characteristic scale* as the scale that produces peak of Laplacian response



Slide credit: Lana Lazebnik

Blob Detection with Scale-Normalized LoG

- The LoG operator is: $\nabla^2 g(x,y,\sigma) * I$
- This is equivalent to: $\nabla^2 (g(x,y,\sigma) * I)$

This means:

- First: smooth the image $L(x,y,\sigma) = g(x,y,\sigma) * I$
- Then: compute Laplacian $\nabla^2 L(x,y,\sigma)$

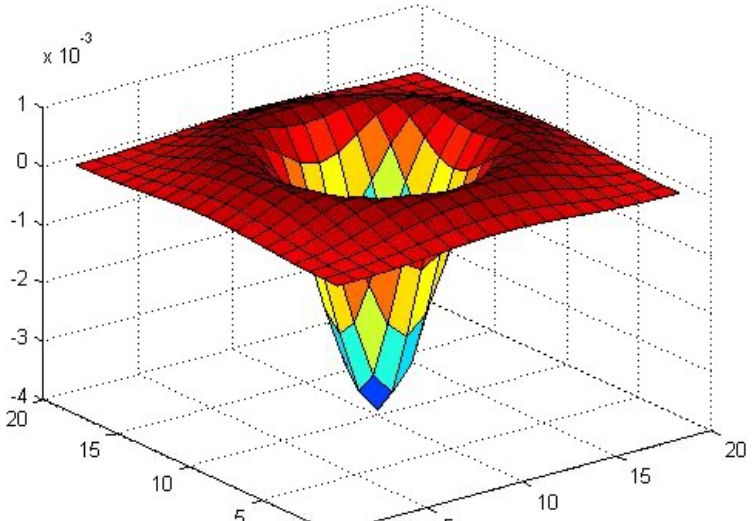
As σ increases: edges become blurred + peaks become flatter, so curvature decreases. As Laplacian (second derivative) measures curvature this means that we get smaller response at higher scales.

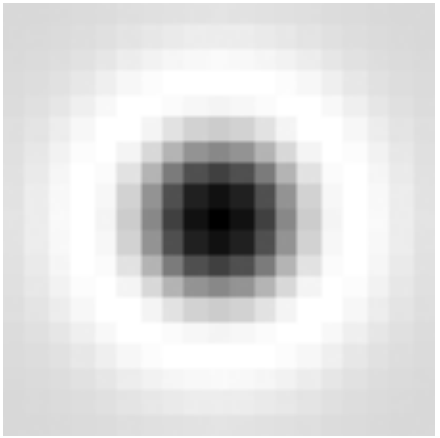
Blob Detection with Scale-Normalized LoG

- Better performance: *Scale-normalized* Laplacian of Gaussian:

$$\nabla^2_{\text{norm}} g = \sigma^2 \left(\frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2} \right)$$

Lindeberg showed that the normalization of the Laplacian with the factor σ^2 is required for true scale invariance





T. Lindeberg, Feature detection with automatic scale selection, *IJCV* 30(2), pp 77-116, 1998

Scale-space blob detector

1. Convolve image with scale-normalized Laplacian at several scales

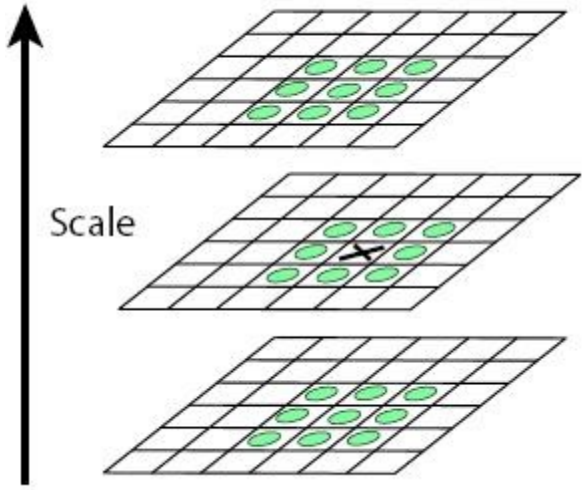
Scale-space blob detector: Example

Scale-space blob detector: Example

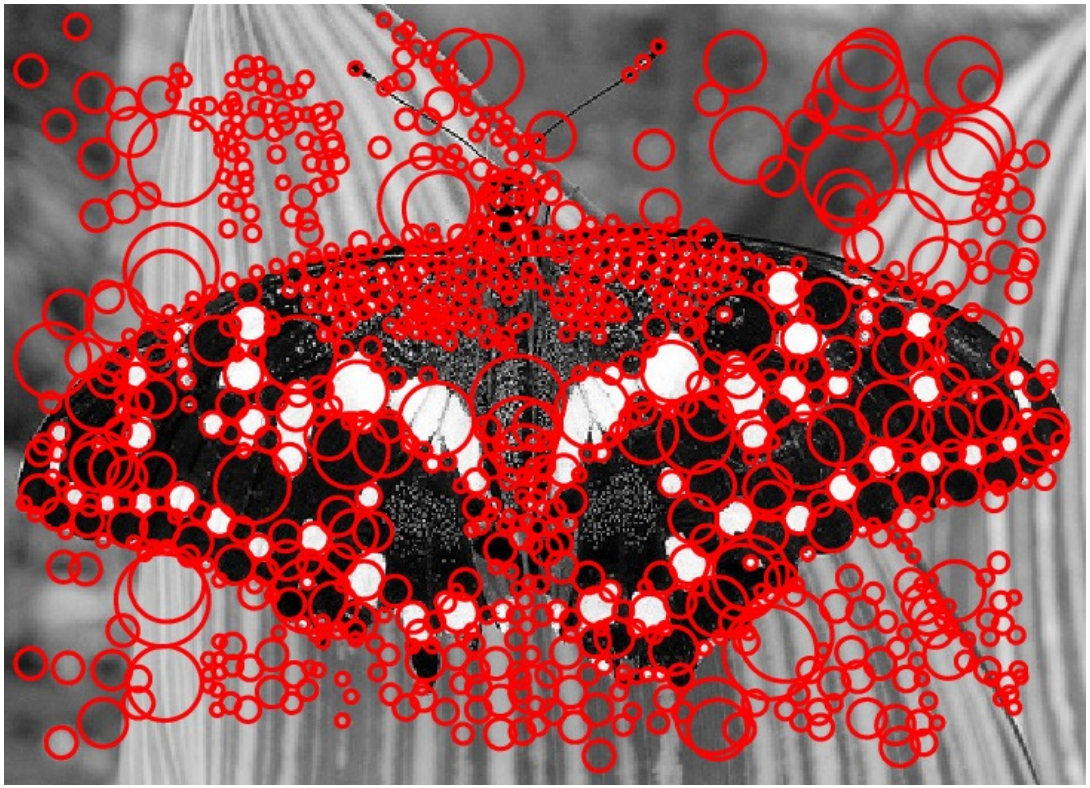
sigma = 11.9912

Scale-space blob detector

1. Convolve image with scale-normalized Laplacian at several scales
2. Find maxima of squared Laplacian response in scale-space



LoG Response: Problem with Edges



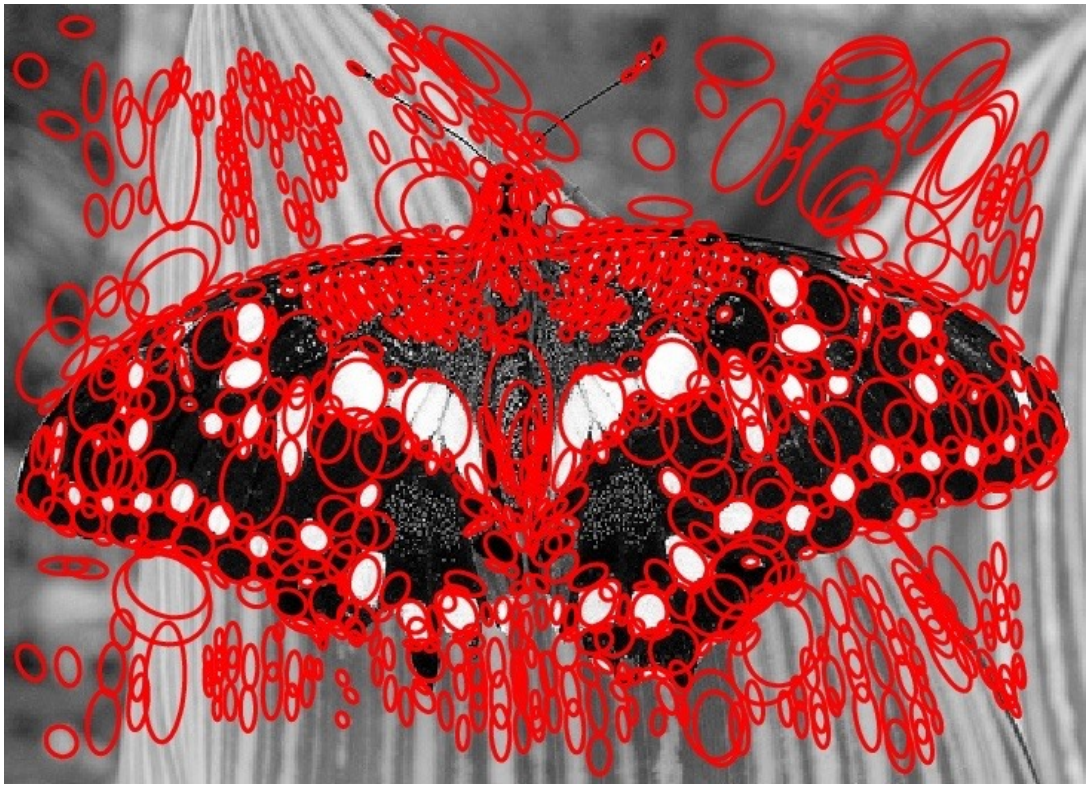
The Laplacian responds strongly to:

- blobs
- edges - these are not stable, not distinctive, not good keypoints

Use additional criteria to:

- suppress edge responses
- keep well-localized structures

Removing Edge Responses



Use Harris cornerness measure. Analyze local curvature:

- if strong in one direction only → edge ✗
- if strong in both directions → blob ✓

Efficient implementation

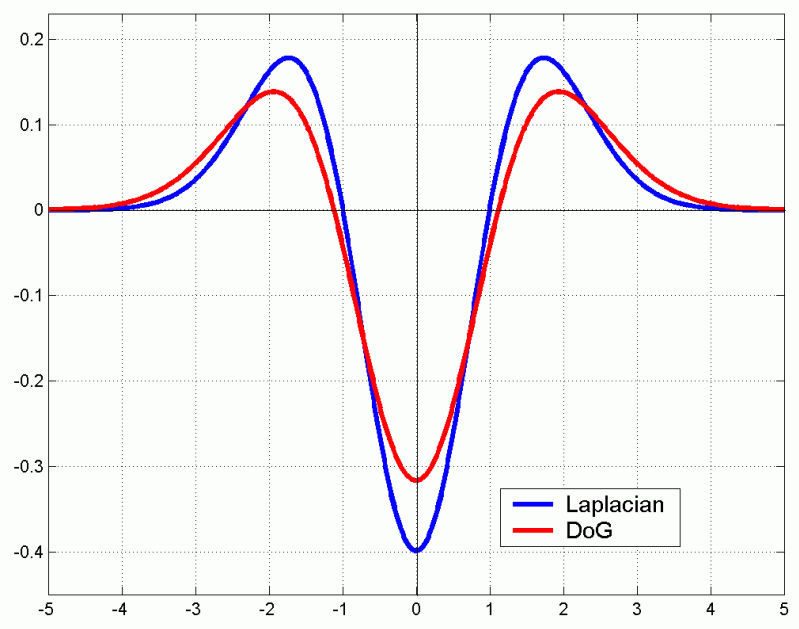
- Approximating the scale-normalized Laplacian with a difference of Gaussians:

$$L = \sigma^2 \left(G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma) \right)$$

(Scale normalized Laplacian)

$$DoG = G(x, y, k\sigma) - G(x, y, \sigma)$$

(Difference of Gaussians)



Efficient implementation

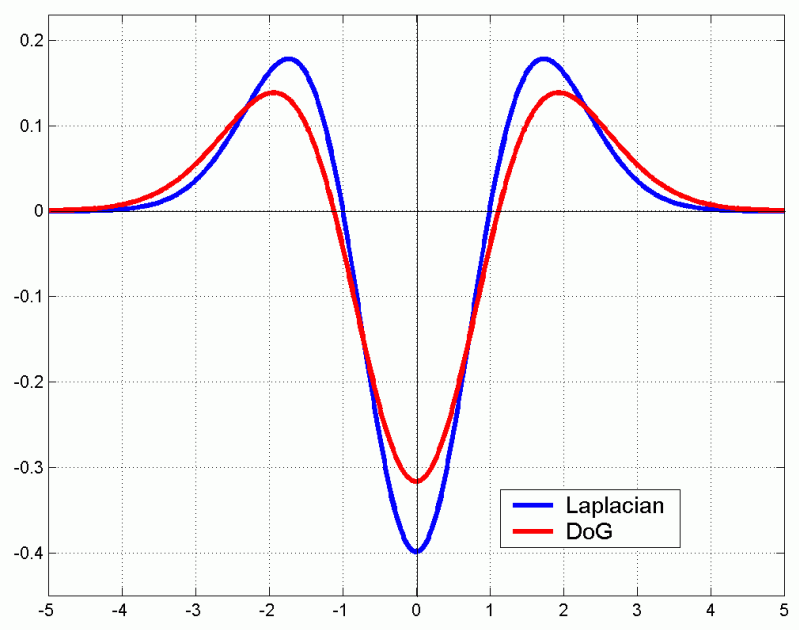
- Approximating the scale-normalized Laplacian with a difference of Gaussians:

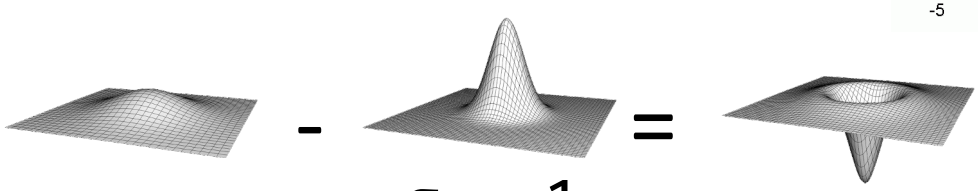
$$L = \sigma^2 \left(G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma) \right)$$

(Scale normalized Laplacian)

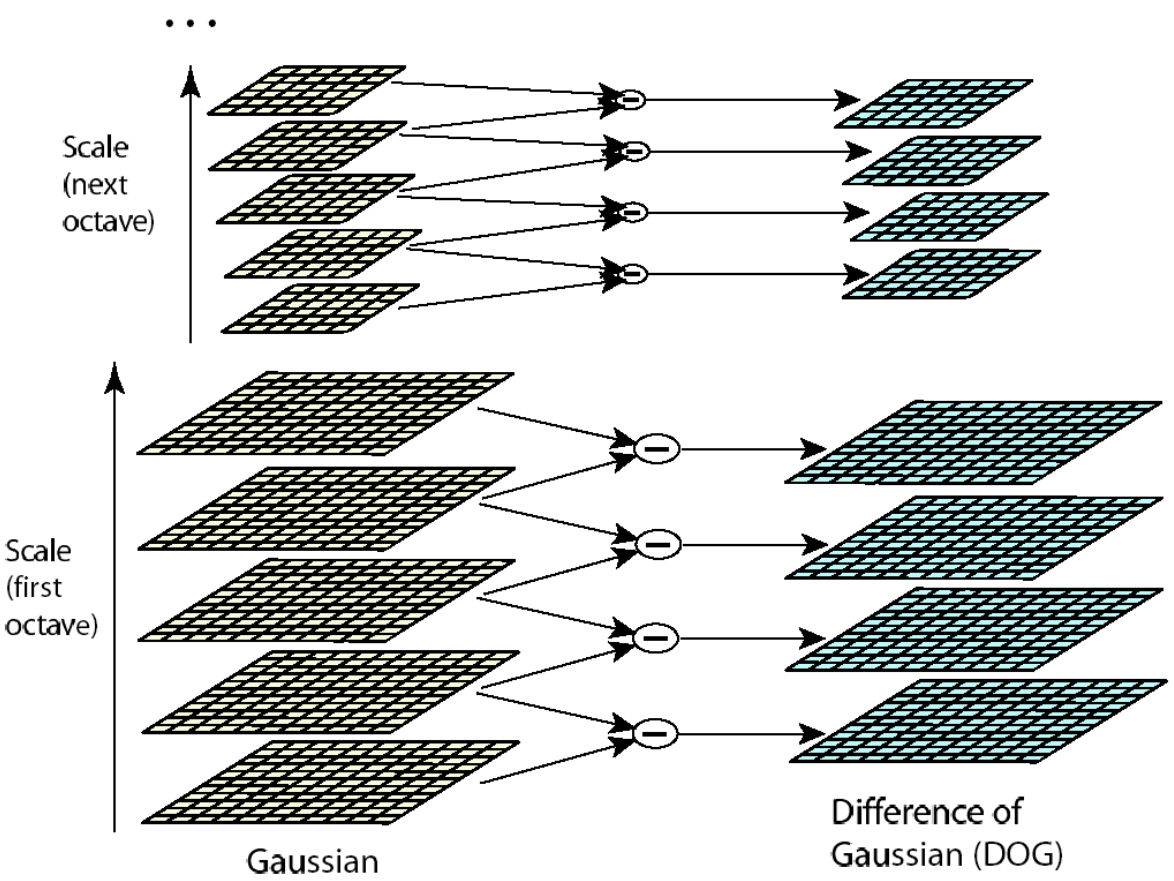
$$DoG = G(x, y, k\sigma) - G(x, y, \sigma)$$

(Difference of Gaussians)





Efficient implementation



David G. Lowe. **"Distinctive image features from scale-invariant keypoints."** *IJCV* 60 (2), pp. 91-110, 2004.

Example

Original image at
¾ the size

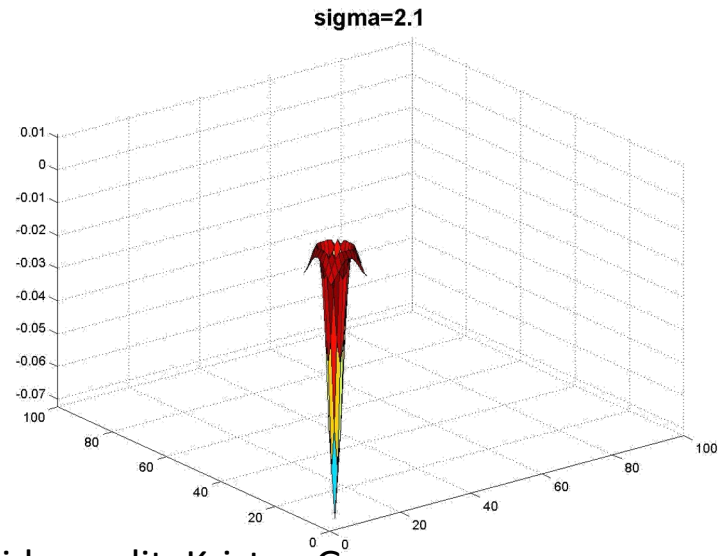
Slide credit: Kristen Grauman

Example

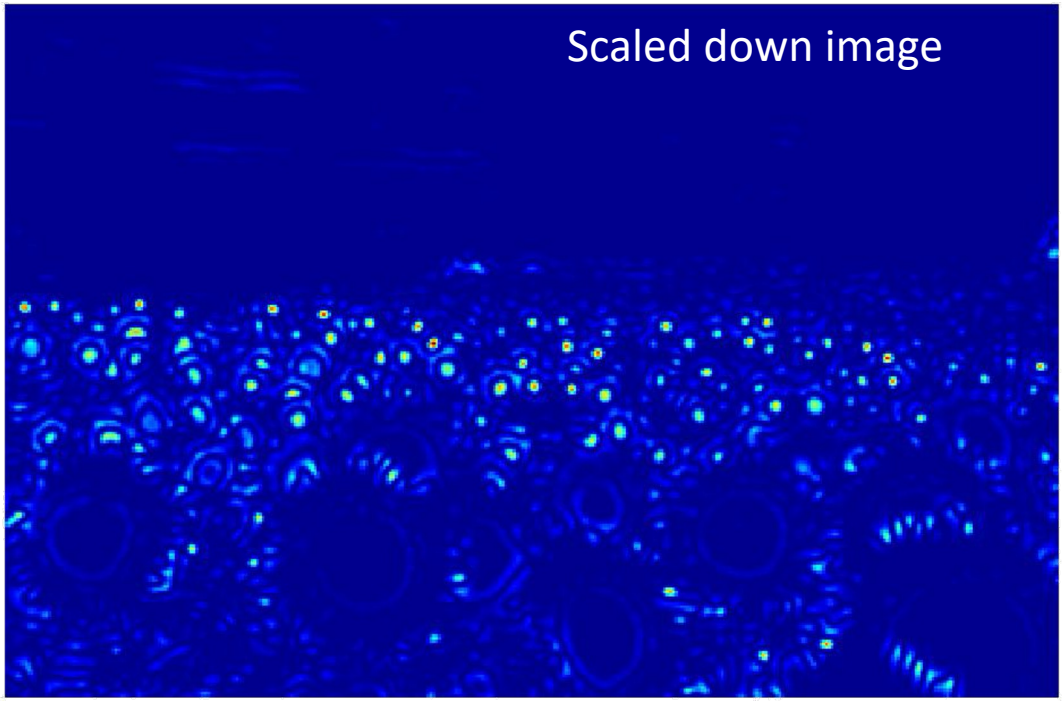
Original image at
¾ the size

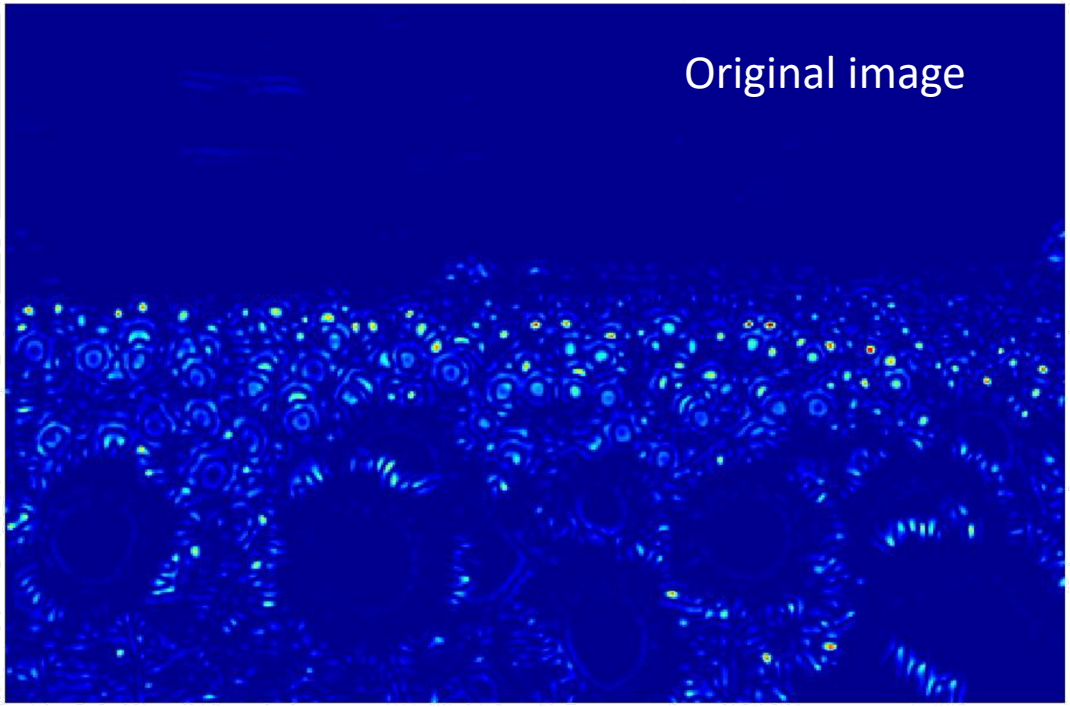
Slide credit: Kristen Grauman

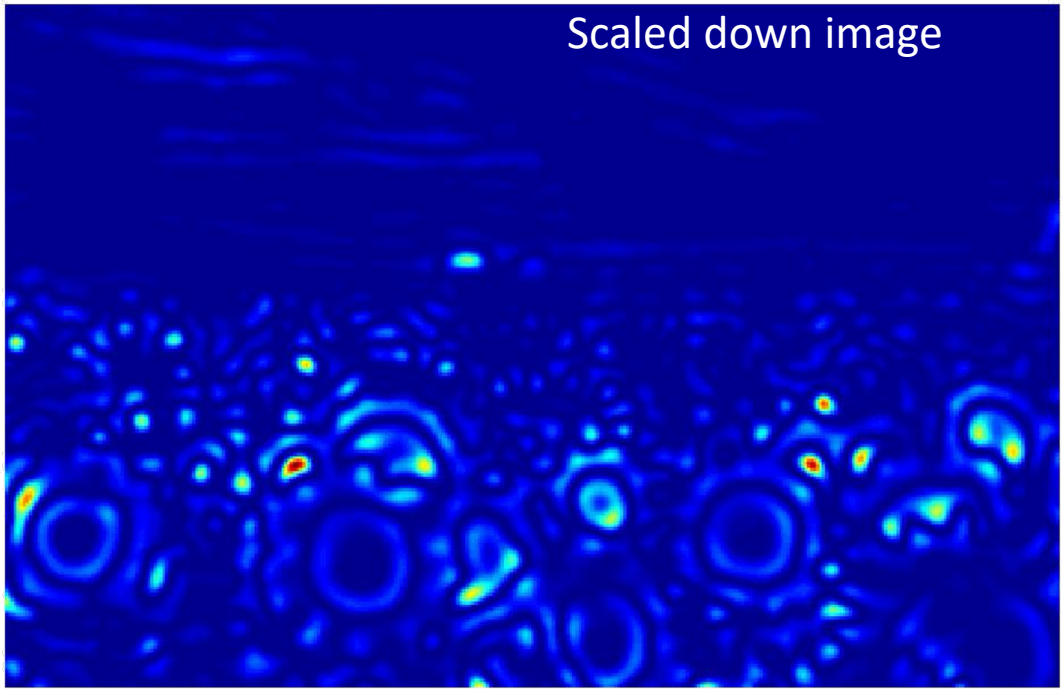
Original image at
¾ the size

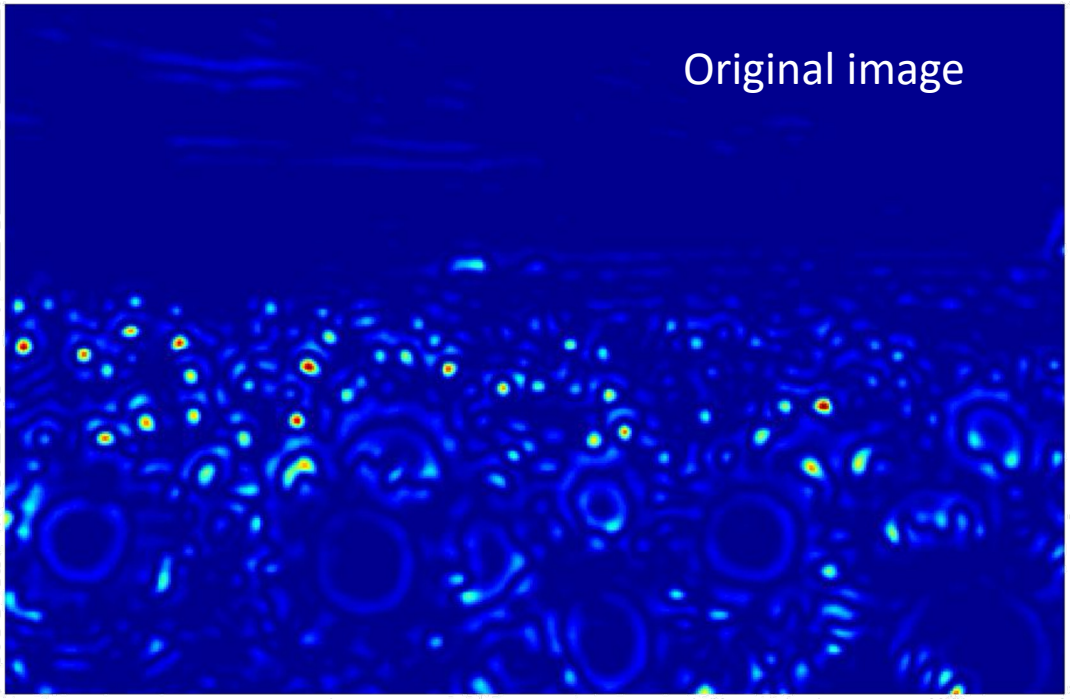


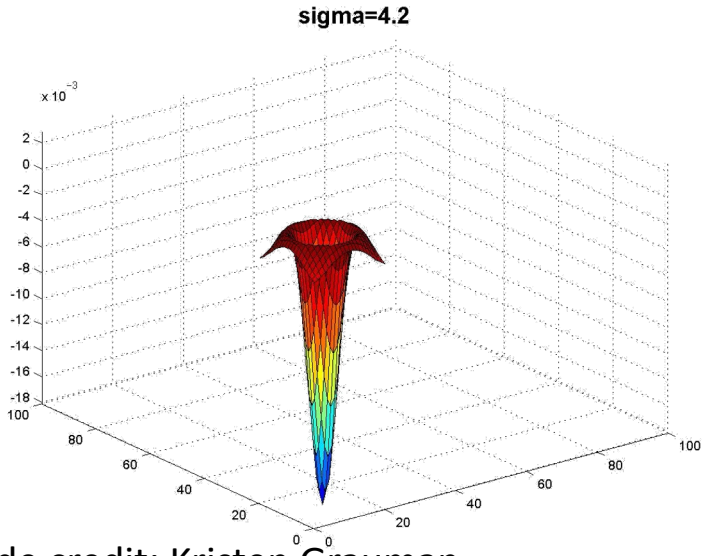
Slide credit: Kristen Grauman



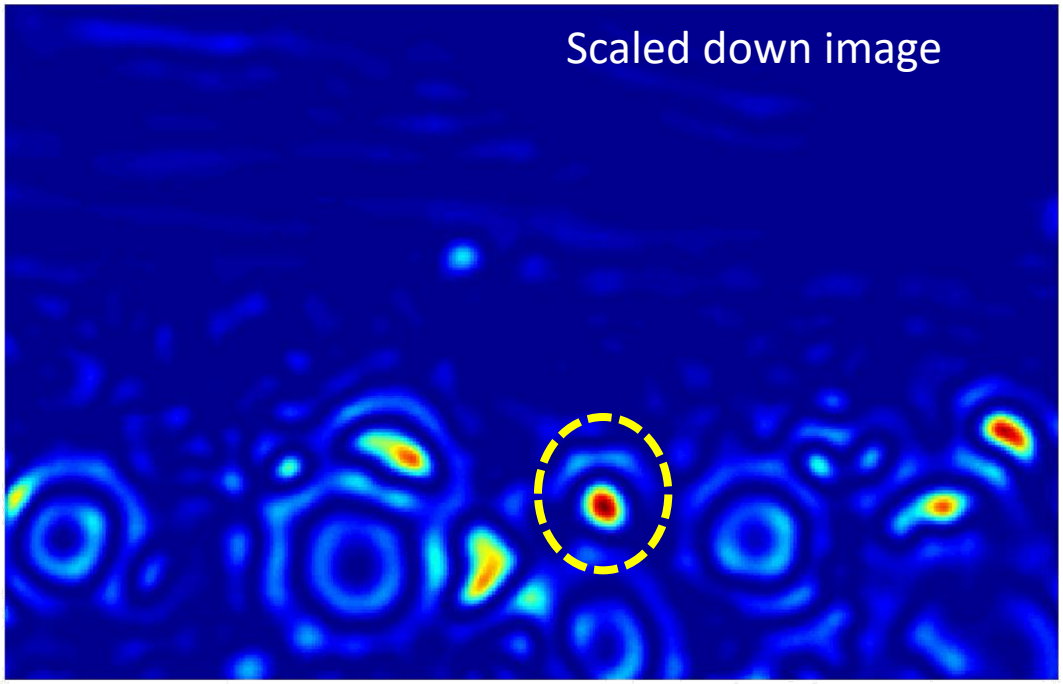


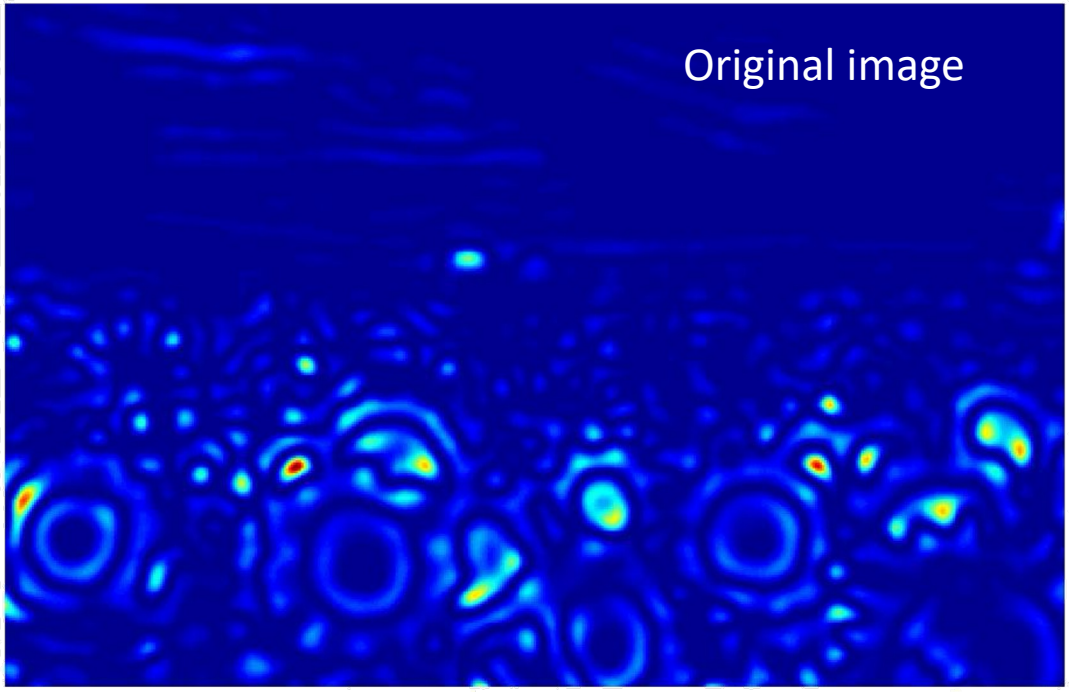


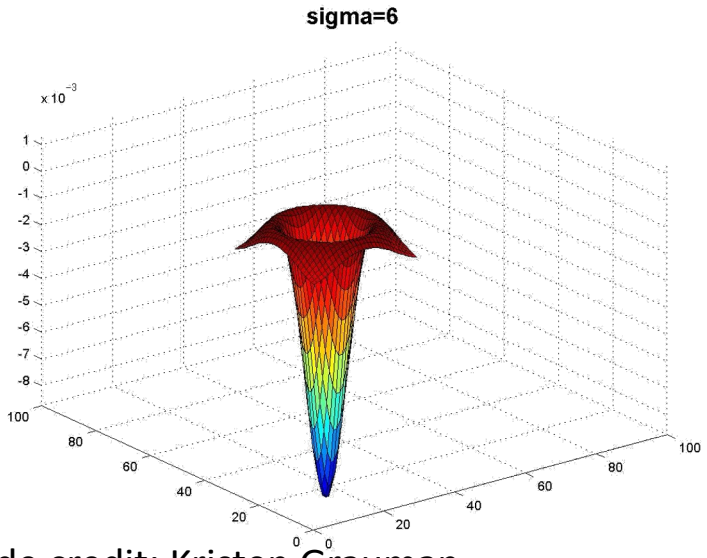




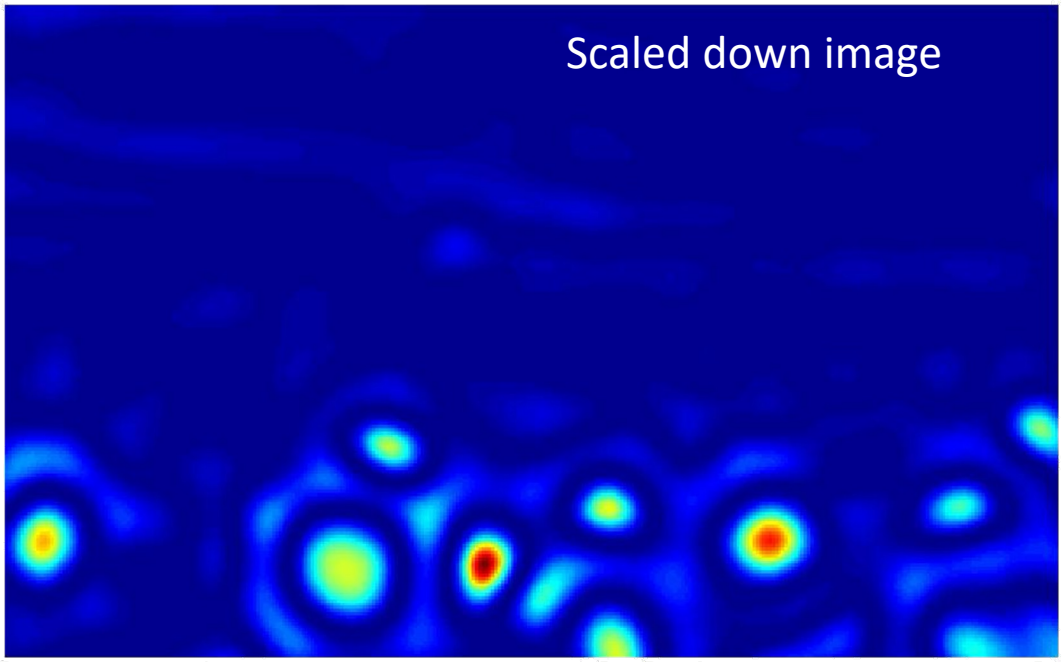
Slide credit: Kristen Grauman

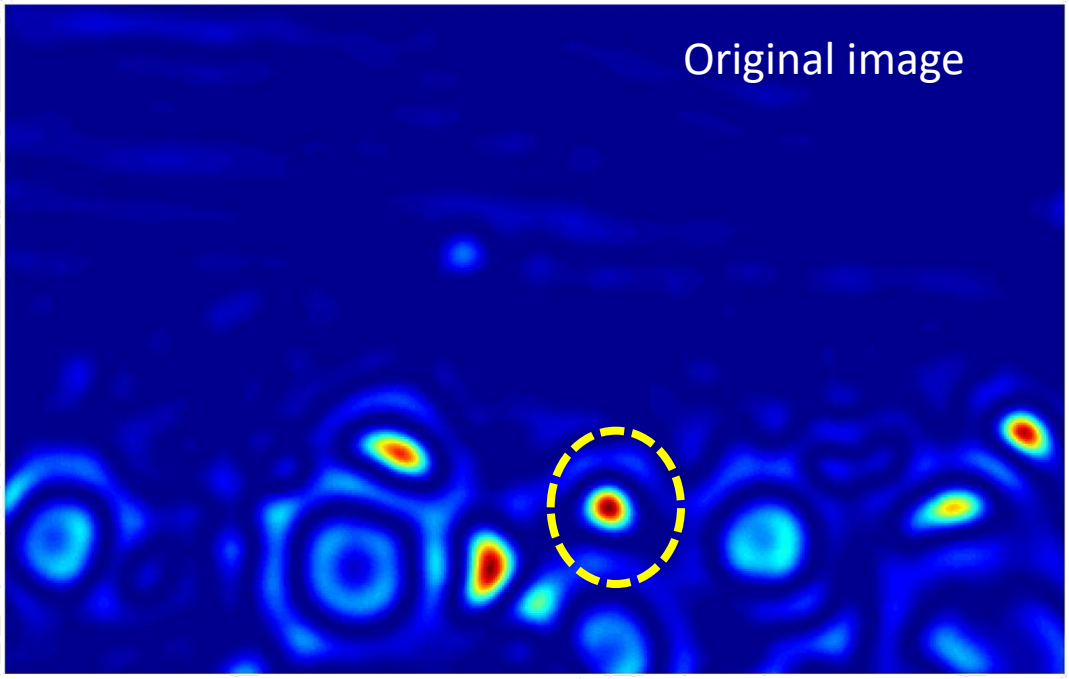


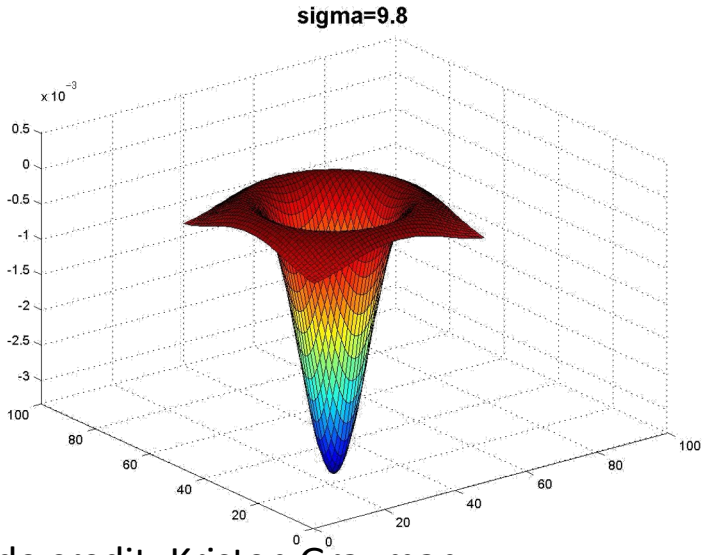




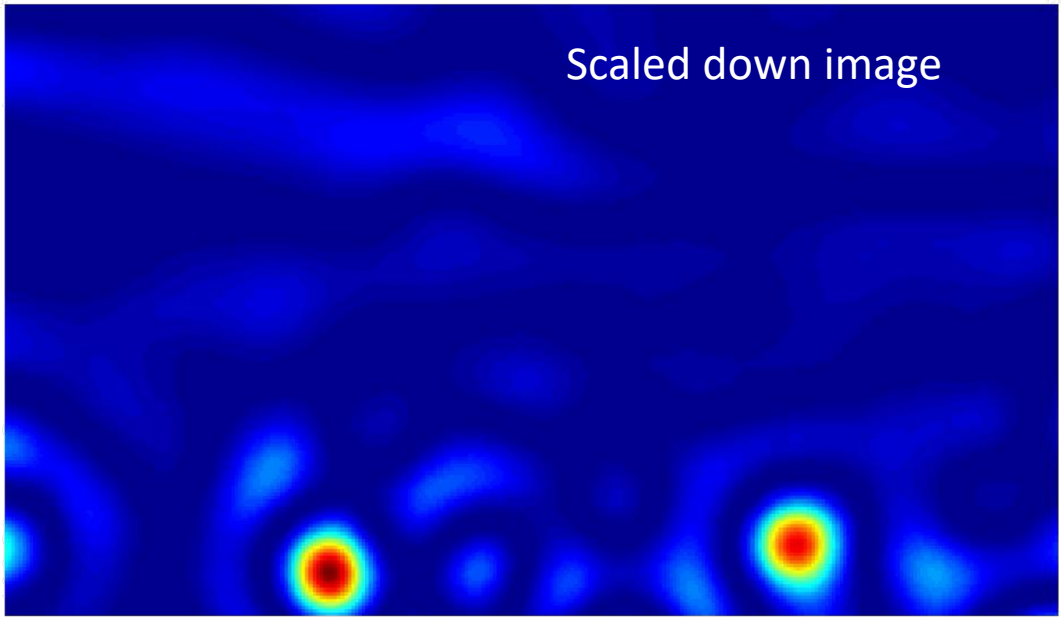
Slide credit: Kristen Grauman

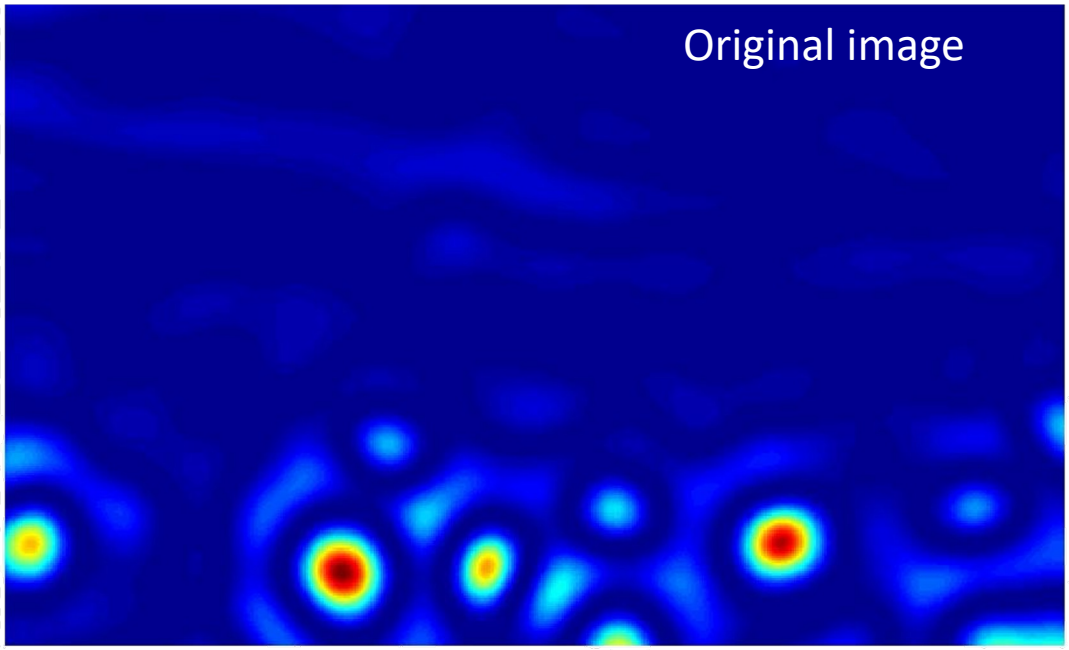


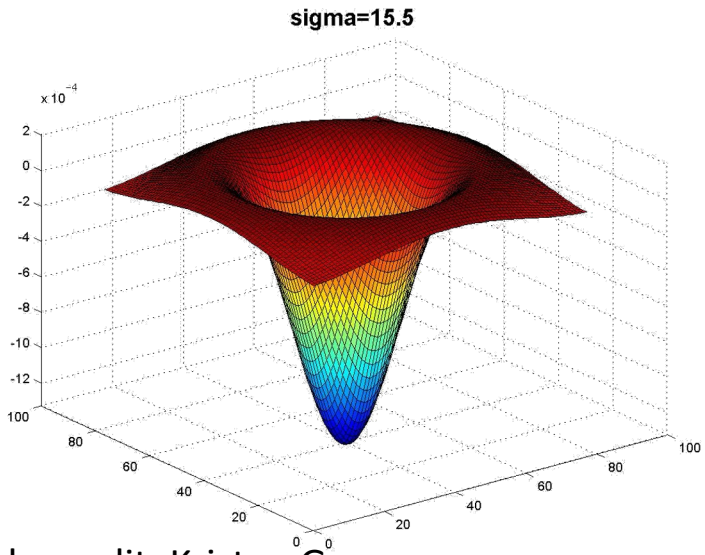




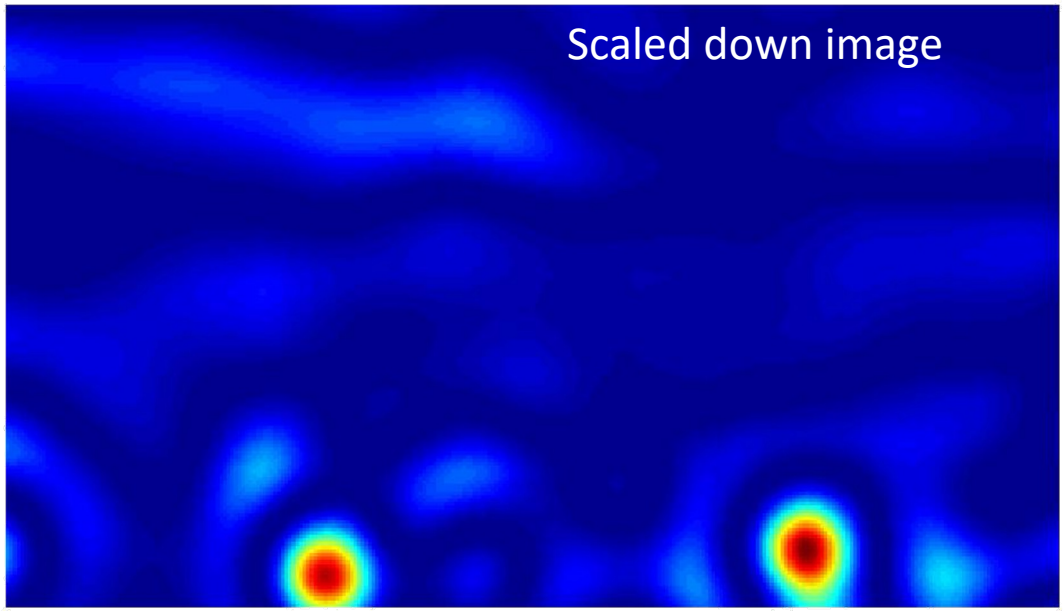
Slide credit: Kristen Grauman

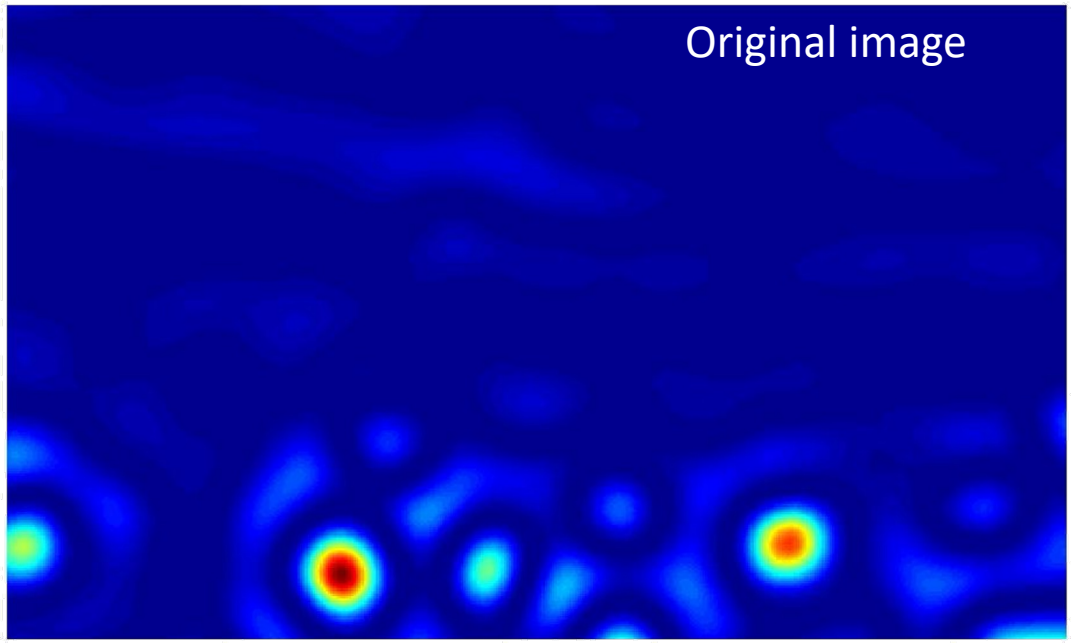


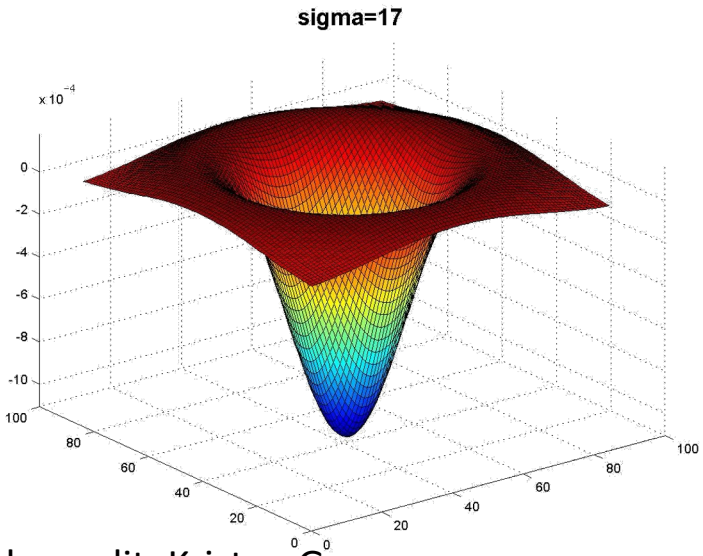




Slide credit: Kristen Grauman





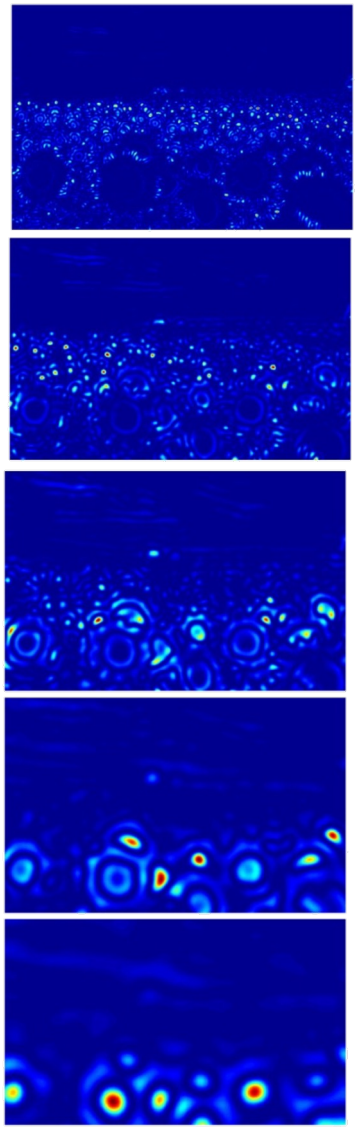


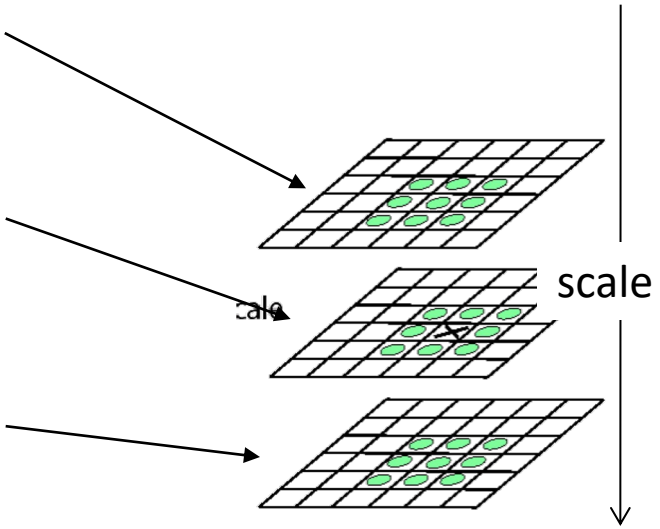
Slide credit: Kristen Grauman

Scale invariant interest points

Interest points are local maxima in both position and scale.

$$L_{xx}(\sigma) + L_{yy}(\sigma)$$





⇒ **List of**
(x, y, σ)

Squared filter response maps

Slide credit: Kristen Grauman

Comparison of Feature Detectors

Table 7.1 Overview of feature detectors.

Feature Detector	Corner	Blob	Region	Rotation invariant	Scale invariant	Affine invariant	Repeatability	Localization accuracy	Robustness	Efficiency
Harris	√			√			+++	+++	+++	++
Hessian		√		√			++	++	++	+
SUSAN	√			√			++	++	++	+++
Harris-Laplace	√	(√)		√	√		+++	+++	++	+
Hessian-Laplace	(√)	√		√	√		+++	+++	+++	+
DoG	(√)	√		√	√		++	++	++	++
SURF	(√)	√		√	√		++	++	++	+++
Harris-Affine	√	(√)		√	√	√	+++	+++	++	++
Hessian-Affine	(√)	√		√	√	√	+++	+++	+++	++
Salient Regions	(√)	√		√	√	(√)	+	+	++	+
Edge-based	√			√	√	√	+++	+++	+	+
MSER			√	√	√	√	+++	+++	++	+++
Intensity-based			√	√	√	√	++	++	++	++
Superpixels			√	√	(√)	(√)	+	+	+	+

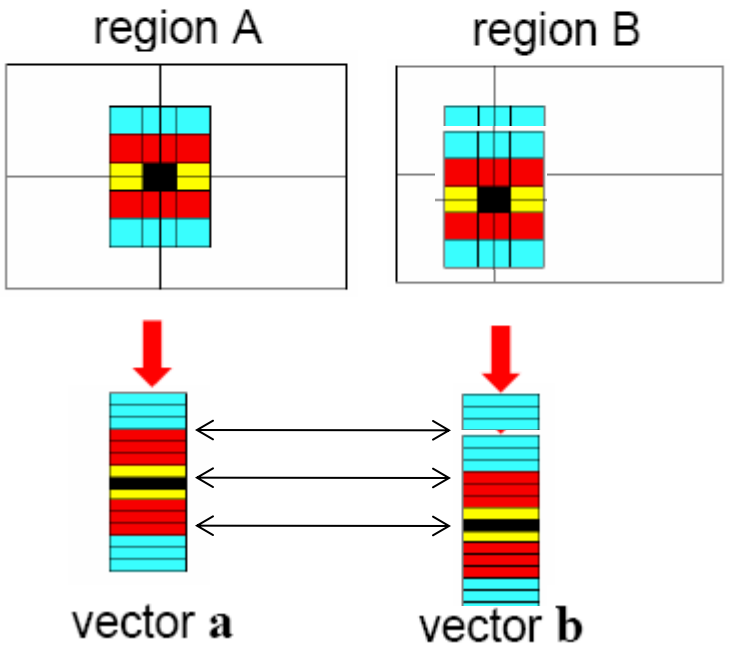
Tuytelaars Mikolajczyk 2008

Detecting local invariant features

- Detection of keypoints (Where?)
  - identify repeatable and distinctive locations
  - typical methods:
    - Harris Corner Detector → corners (intensity variation in all directions)
    - LoG (Laplacian of Gaussian) → blob detection (scale-invariant)

- Description of local patches (What?)
  - represent each keypoint by a feature vector
  - encodes the appearance of the local neighborhood
  - should be:
    - invariant (scale, rotation, illumination)
    - discriminative

Raw patches as local descriptors



The simplest way to describe the neighborhood around an interest point is to write down the list of intensities to form a feature vector.

But this is very sensitive to even small shifts, rotations.

Figure: Andrew Zisserman

Local Descriptors

- The ideal descriptor should be
  - Robust
  - Distinctive
  - Compact
  - Efficient

- Most available descriptors focus on edge/gradient information
  - Capture texture information
  - Color rarely used

SuperPoint: Self-Supervised Interest Point Detection and Description

Daniel DeTone
Magic Leap
Sunnyvale, CA
ddetone@magicleap.com

Tomasz Malisiewicz
Magic Leap
Sunnyvale, CA
tmalisiewicz@magicleap.com

Andrew Rabinovich
Magic Leap
Sunnyvale, CA
arabinovich@magicleap.com

Abstract

*This paper presents a self-supervised framework for training interest point detectors and descriptors suitable for a large number of multiple-view geometry problems in computer vision. As opposed to patch-based neural networks, our fully-convolutional model operates on full-sized images and jointly computes pixel-level interest point locations and associated descriptors in one forward pass. We introduce Homographic Adaptation, a multi-scale, multi-homography approach for boosting interest point detection repeatability and performing cross-domain adaptation (*e.g.*, synthetic-to-real). Our model, when trained on the MS-COCO generic image dataset using Homographic Adaptation, is able to repeatedly detect a much richer set of interest points than the initial pre-adapted deep model and any other traditional corner detector. The final system gives rise to state-of-the-art homography estimation results on HPatches when compared to LIFT, SIFT and ORB.*

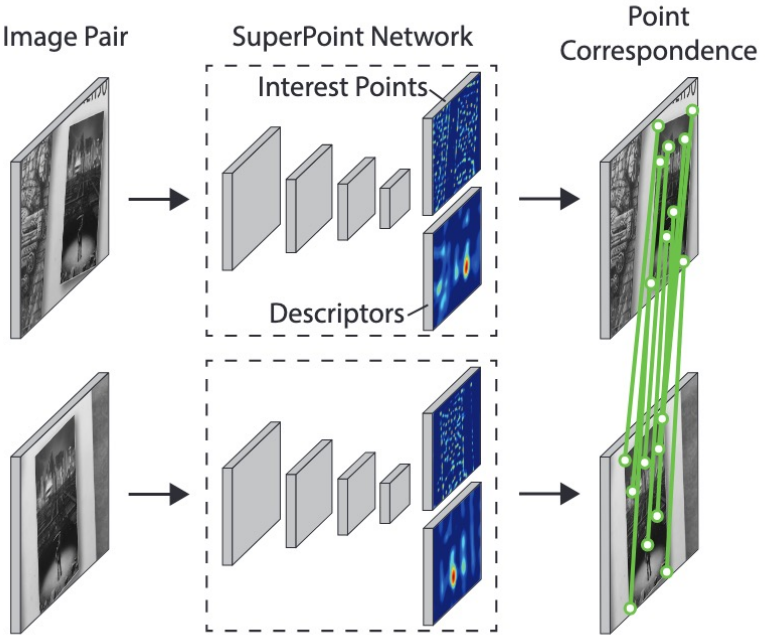


Figure 1. **SuperPoint for Geometric Correspondences.** We present a fully-convolutional neural network that computes SIFT-like 2D interest point locations and descriptors in a single forward pass and runs at 70 FPS on 480×640 images with a Titan X GPU.

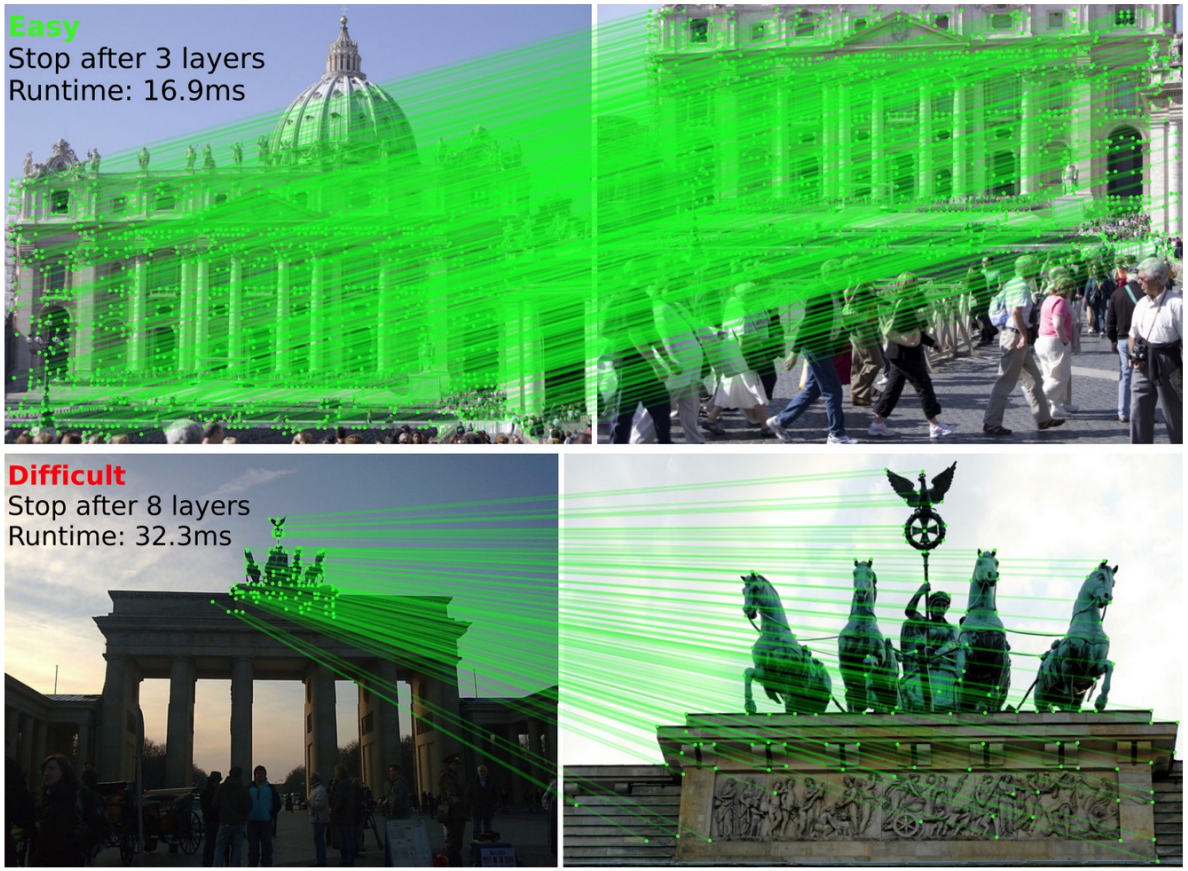
1. Introduction

LightGlue ⚡
Local Feature Matching at Light Speed

Philipp Lindenberger · Paul-Edouard Sarlin · Marc Pollefeys

ICCV 2023

Paper | Colab | Poster | Train your own!



LightGlue is a deep neural network that matches sparse local features across image pairs.
An adaptive mechanism makes it fast for easy pairs (top) and reduces the computational complexity for difficult ones (bottom).

David Lowe

FOLLOW

Professor Emeritus, Computer Science Dept., University of British Columbia
Verified email at cs.ubc.ca - Homepage

Computer Vision Object Recognition

TITLE	CITED BY	YEAR
Distinctive image features from scale-invariant keypoints DG Lowe International journal of computer vision 60 (2), 91-110	81375	2004
Object recognition from local scale-invariant features DG Lowe International Conference on Computer Vision, 1999, 1150-1157	27264	1999
Fast Approximate Nearest Neighbors with Automatic Algorithm Configuration. M Muja, DG Lowe VISAPP (1) 2, 331-340	4496	2009
Automatic panoramic image stitching using invariant features M Brown, DG Lowe International Journal of Computer Vision 74 (1), 59-73	3925	2007
Unsupervised learning of depth and ego-motion from video T Zhou, M Brown, N Snavely, DG Lowe Proceedings of the IEEE Conference on Computer Vision and Pattern …	3718	2017
Perceptual Organization and Visual Recognition DG Lowe Kluwer Academic Publishers, Boston	2226 *	1985
Three-dimensional object recognition from single two-dimensional images DG Lowe Artificial intelligence 31 (3), 355-395	2119	1987